

Zero-Knowledge IOPPs for Constrained Interleaved Codes

Alessandro Chiesa
alessandro.chiesa@epfl.ch
EPFL

Giacomo Fenzi
giacomo.fenzi@epfl.ch
EPFL

Guy Weissenberg
guy.weissenberg@epfl.ch
EPFL

February 25, 2026

Abstract

Succinct arguments based on interactive oracle proofs (IOPs) have achieved remarkable efficiency improvements and are now widely adopted in applications. State-of-the-art IOPs involve protocols for testing proximity to constrained interleaved linear codes, and enjoy essentially optimal parameters. However, recent IOP constructions provide no privacy guarantees, which remain a must for many applications.

We present an IOP of proximity for testing constrained interleaved linear codes that achieves (honest-verifier) zero-knowledge, while incurring a negligible overhead compared to the (non-zero-knowledge) state of the art. In line with recent constructions, our construction satisfies round-by-round knowledge soundness with a straightline extractor and negligible error.

We propose a definition of (honest-verifier) zero-knowledge for interactive oracle reductions (IORs) that we prove is compatible with composition, and then obtain our result by constructing and modularly composing several lightweight zero-knowledge IORs. Our key technical contributions are a zero-knowledge sumcheck IOR and a zero-knowledge code-switching IOR that fit the strict efficiency requirements of our setting; these contributions and other technical complications entailed overcoming several challenges with new notions and protocols. Finally, along the way, we highlight the efficiency benefits of high-distance codes obtained from dispersers, which may be of independent interest.

Keywords: interactive oracle reductions; zero-knowledge; linear codes; dispersers

Contents

1	Introduction	3
1.1	Our results	4
2	Technical overview	8
2.1	Basic notions	8
2.2	Warmup: a simple zero-knowledge IOPP for constrained codes	9
2.3	Composition for honest-verifier zero-knowledge IORs	11
2.4	A “committed” relation for zero-knowledge	12
2.5	An honest-verifier zero-knowledge sumcheck reduction	13
2.6	HVZK code switching	18
2.7	From building blocks to our main results	22
2.8	Bonus: distance-amplified codes	24
3	Preliminaries	26
3.1	Interactive oracle reductions	26
3.2	Coding theory	27
4	Zero-knowledge for IORs and composition	32
5	Succinct linear forms	35
6	Zero-knowledge sumcheck IOR	37
6.1	Zero-knowledge	38
6.2	Round-by-round security	40
7	A non-succinct zero-knowledge protocol for constrained codes	43
7.1	Zero-knowledge	44
7.2	Round-by-round knowledge soundness	45
8	A sublinear zero-knowledge IOP for constrained codes	49
9	Zero-knowledge code-switching	51
9.1	Out-of-domain samples	51
9.2	Private zero-evaders	51
9.3	Code-switching complexity	53
9.4	Code-switching IOR	54
9.5	Zero-knowledge	56
9.6	Round-by-round knowledge soundness	57
9.7	Proof of Theorem 2	60
10	Zero-knowledge IOPP for constrained codes	62
11	Zero-knowledge reduction from R1CS	66
12	High-distance codes from dispersers	72
12.1	Preliminaries	72
12.2	Amplified Reed–Solomon code	73
12.3	Distance amplification and Merkle-compiled argument size	75
	Acknowledgments	77
	References	77

1 Introduction

Succinct non-interactive arguments (SNARGs) are short cryptographic proofs that admit a fast-verification procedure. They are widely deployed across applications in decentralized systems and cloud computing. We study *hash-based SNARGs*, a class of constructions notable for their transparent setup (the choice of hash function), post-quantum security (in the quantum random oracle model), fast provers (they avoid public-key cryptography), and more. Such SNARGs are, e.g., used for scaling distributed systems (e.g. [Sta; Pol; Ris; Suc]) and for anonymous credentials [Fs24].

Hash-based SNARGs are constructed via the BCS transformation [BCS16] from an interactive oracle proof (IOP), and a beautiful line of research investigates efficient IOP constructions. In this paper we investigate efficient constructions of (honest-verifier) *zero-knowledge* IOPs with the goal of obtaining efficient constructions of (malicious-verifier) *zero-knowledge* hash-based SNARGs.

The efficiency of code-agnostic IOPs. Many constructions of efficient IOPs rely on polynomial codes [AHIV17; BBHR19; BCRSVW19; COS20; ACFY24; HK24; Dia25; MZ25]. The limitations of constructions based on polynomial codes (e.g., superlinear encoding time) have motivated the study of IOPs based only on combinatorial transformations of general linear codes (e.g., tensoring, interleaving), which has achieved “code-agnostic” IOPs [BCGGHJ17; BCG20; RR20; RR22; NA25; BMMS25a; DMR25; DL25; ARR25; BFRW25; BMMS25b]. This is reminiscent of similar efforts for PCPs (the non-interactive counterparts of IOPs) where researchers have sought to decouple PCPs from the properties of the underlying codes in order to understand the minimal structure that enables PCPs and related tools [GS00; DR04; Din07; IKW09; DM11; Mei13; Mei12].

Recent works have shown that general linear codes and their interleaving are sufficient to construct state-of-the-art IOPs in terms of efficiency [BCGGHJ17; BCFW25; ACFY25; BCFRRZ25; NA25; BMMS25a], *even outperforming* constructions tailored to an error-correcting code (such as e.g. [ACFY24]). This has provided a class of highly configurable and efficient IOP constructions.

The cost of zero-knowledge. Zero-knowledge for hash-based SNARGs is increasingly in demand [Fs24; Mid; Goo; Lig], which requires the underlying IOP to satisfy (honest-verifier) zero-knowledge. Such an IOP is usually obtained by modifying an underlying (non-private) IOP. A key goal is minimizing the “zero-knowledge overhead”, i.e., the cost incurred by the prover and verifier in using the zero-knowledge version of the protocol compared to its non-private equivalent.

The zero-knowledge overhead of concretely efficient IOPs is $\geq 2\times$ (in prover and verifier times), and they are specific to Reed–Solomon codes [RS60] (or other polynomial codes). This overhead deters developers and engineers from using the zero-knowledge version. Moreover, *none of the state-of-the-art code-agnostic IOPs in the literature provide zero-knowledge*.

Two notable works study zero-knowledge IOPs based on any “zero-knowledge” code (we define these shortly). [BCL22] obtains a tensor-code-based IOP with $\geq 2\times$ overhead; the IOP is not concretely efficient partly due to heavy PCP machinery. [RW24] obtains a tensor-code-based IOP that reduces the overhead to $1 + o(1)$ relative to witness size, however only targets constant soundness. Both works achieve malicious-verifier zero-knowledge, while honest-verifier zero-knowledge suffices for achieving (malicious-verifier) zero-knowledge via the Fiat–Shamir transformation.

Tensoring vs. interleaving. There are two fundamental operations on codes that are used to create a modicum of structure for designing “code-agnostic” protocols.

- *Code tensoring.* The tensor product of codes $\mathcal{C}_1 \subseteq \Sigma^{m_1}$ and $\mathcal{C}_2 \subseteq \Sigma^{m_2}$ is the code $\mathcal{C}_1 \otimes \mathcal{C}_2 \subseteq \Sigma^{m_1 \times m_2}$ of matrices $u \in \Sigma^{m_1 \times m_2}$ such that each column of u is in $\mathcal{C}_1$ and each row of u is in $\mathcal{C}_2$. The code $\mathcal{C}_1 \otimes \mathcal{C}_2$ is a “nice” code if $\mathcal{C}_1$ and $\mathcal{C}_2$ both have (even general forms of) linear structure.

- *Code interleaving.* The K -interleaving of a code $\mathcal{C} \subseteq \Sigma^m$ is the code $\mathcal{C}^{\equiv K} \subseteq (\Sigma^K)^m$ obtained by “stacking” K codewords from $\mathcal{C}$, that is, a word $u \in (\Sigma^K)^m$ is in $\mathcal{C}^{\equiv K}$ if and only if there exist words $v_1, \dots, v_K \in \Sigma^m$ in $\mathcal{C}$ such that $u(z) = (v_1(z), \dots, v_K(z)) \in \Sigma^K$ for every $z \in [m]$.

Both operations lead to linear-time IOP provers by way of linear-time encodable codes, but they offer different tradeoffs. Tensoring creates key structures for local testability and tensor sumchecks [Mei13; RR20; BCG20; RR22; BCL22; RW24; BFRW25; BMMS25b], enabling for example query-optimal IOPs; this comes with the cost of worse rate and distance compared to the original code as well as a less efficient encoding. Interleaving also enables key structures for alternative sumchecks and code-switching [AHIV17; BBHV22; ACFY25; NA25; BMMS25a], while preserving rate and distance at the cost of increasing the alphabet size (which precludes query-optimal IOPs); on the other hand, interleaving leads to better encoding efficiency (including with small space [BBHV22]) and also works for *every* code, enabling the most freedom in code choice for a protocol.

1.1 Our results

We present a concretely-efficient HVZK IOPP for constrained interleaved codes, as well as a corresponding HVZK IOP for R1CS. We match the efficiency of state-of-the-art (non-private) IOPs up to low-order factors, obtaining a zero-knowledge overhead of $1 + o(1)$; moreover, we achieve round-by-round straightline extraction for every code by relying on mutual correlated agreement.

We elaborate on our results below. We begin by recalling the notions of zero-knowledge encodings and constrained interleaved codes, which we need to state our results.

ZK encodings. We use *zero-knowledge encodings* for codes, also known as zero-knowledge codes [ISVW13; BCGV16; BBHR19; BCL22; RW24]. For a finite field $\mathbb{F}$, we say that a code $\mathcal{C} \subseteq \Sigma^m$ is $\mathbb{F}$ -*additive* if the alphabet Σ is an $\mathbb{F}$ -linear vector space (e.g., $\Sigma = \mathbb{F}^\iota$ for some $\iota \in \mathbb{N}$); when $\Sigma = \mathbb{F}$ this becomes an $\mathbb{F}$ -linear code in the common sense. For message length ℓ and randomness length r , we say that $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$ is a t -query zero-knowledge encoding for $\mathcal{C}$ if: (i) the image of $\text{Enc}_{\mathcal{C}}$ is in $\mathcal{C}$ (i.e., $\text{Enc}_{\mathcal{C}}(\mathbb{F}^\ell, \mathbb{F}^r) \subseteq \mathcal{C}$); and (ii) for every message $\text{msg} \in \mathbb{F}^\ell$ and index subset $S \subseteq [m]$ with $|S| \leq t$, the distribution $\text{Enc}_{\mathcal{C}}(\text{msg}, \mathbf{r} \leftarrow \mathbb{F}^r)[S]$ can be simulated efficiently, up to some error ζ , given only S (but not msg). Intuitively, a *randomized* encoding of a message yields multiple possible encodings and the view of a certain number of locations is independent of the message.

Our target: constrained interleaved codes with ZK encodings. We seek to construct concretely-efficient HVZK IOPPs for relations that involve interleaved codes whose underlying messages are subject to succinct linear constraints. This is in line with state-of-the-art protocols that consider proximity testing to codes with linear constraints [ACFY25; BMNW25; BCFW25], which enables significantly more efficient (and simpler) reductions from NP compared to merely relying on proximity tests to unconstrained codes. Informally, given a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}$ for a code $\mathcal{C}$ we consider the relation of linear constraints defined as follows:

$$\mathcal{R}_{\mathcal{C},T}^{\text{lin}} := \left\{ \left(\begin{array}{l} \mathbf{z} = (\mu, \langle \mathbf{v} \rangle) \in \mathbb{F} \times \mathbb{F}^{\log \ell}, \\ \mathbf{y} = f \in \Sigma^m, \\ \mathbf{w} = (\mathbf{f}, \mathbf{r}) \in \mathbb{F}^\ell \times \mathbb{F}^r \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r}) \\ \wedge \langle \mathbf{f}, \mathbf{v} \rangle = \mu \end{array} \right\}. \quad (1)$$

Above $\langle \mathbf{v} \rangle$ is some “succinct” description of the vector $\mathbf{v} \in \mathbb{F}^\ell$ from which $\mathbf{v}$ can be computed in time T and “ k -folds of $\mathbf{v}$ ” (which we define later) can be computed in time $T^{(k)}$. (In fact $\mathcal{R}_{\mathcal{C},T}^{\text{lin}}$ is a simplification of the relation we use; in Section 2.4 we discuss and motivate a refined variant of it.)

HVZK IOPP for constrained interleaved codes. Our first result is an honest-verifier zero-knowledge IOP of proximity for the linear constraint relation $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, T}^{\text{lin}}$ (Equation 1) for a given interleaving parameter k . This IOPP is in the “sublinear regime” and is the result of a single reduction step of a more general *HVZK code-switching* technique that we discuss further below.

Theorem 1 (informal). *Let $\mathcal{C} \subseteq \Sigma^{m_{\mathcal{C}}}$ be an $\mathbb{F}$ -additive code with a $t_{\mathcal{C}}$ -query zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^{\ell_{\mathcal{C}}} \times \mathbb{F}^{r_{\mathcal{C}}} \rightarrow \Sigma^{m_{\mathcal{C}}}$ with error $\zeta_{\mathcal{C}}$ and encoding time $\tau_{\mathcal{C}}$. Fix an interleaving parameter $k \in \mathbb{N}$, security parameter $\lambda \in \mathbb{N}$, and proximity parameter $\delta \in (0, 1)$; let $t := O(\frac{\lambda}{-\log(1-\delta)})$. We construct a $O(k)$ -round IOPP for $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, T}^{\text{lin}}$ with the following features.*

- *The prover sends $O(m_{\mathcal{C}}) + \tilde{O}(k \cdot \lambda)$ alphabet symbols, and the verifier makes t queries over the alphabet Σ^{2^k} .*
- *The prover time is $O(T + 2^k \cdot \ell_{\mathcal{C}} + \tau_{\mathcal{C}}) + \tilde{O}(k \cdot \lambda)$ and verifier time is $O(T^{(k)} + 2^k \cdot t + \tau_{\mathcal{C}}) + \tilde{O}(k \cdot \lambda)$.*
- *The protocol has round-by-round knowledge soundness error at most $2^{-\lambda}$ assuming $\mathbb{F}$ is large enough with respect to $\mathcal{C}$'s mutual correlated agreement radius at distance δ .*
- *If $t_{\mathcal{C}} \geq t$ the protocol is honest-verifier zero-knowledge with error $\zeta_{\mathcal{C}}$.*

The notation $\tilde{O}(\cdot)$ hides $\text{poly}(\log \lambda)$ factors (here and elsewhere in this introduction).

The prover sends $O(m_{\mathcal{C}}) + \tilde{O}(k \cdot \lambda)$ alphabet symbols, while the block length of $\mathcal{C}^{\equiv 2^k}$ is $N = 2^k \cdot m_{\mathcal{C}}$ alphabet symbols. For common settings of the interleaving parameter k (say $\frac{\log N}{2}$ or $\log \log N$), the prover sends $O(\frac{m_{\mathcal{C}}}{N}) \cdot N + \tilde{O}(\frac{k \cdot \lambda}{N}) \cdot N = o(1) \cdot N$ alphabet symbols. As this includes the costs arising from the addition of HVZK, the zero-knowledge overhead is $o(1)$. A notable setting of parameters for Theorem 1 is the *sublinear regime*, achieved when $2^k \approx m_{\mathcal{C}}$. In this setting, the proof length is $\approx O(\sqrt{N})$ alphabet symbols, and verifier time is sublinear.

Theorem 1 gives the first IOPP for constrained interleaved general codes with list-decoding soundness and $1 + o(1)$ zero-knowledge overhead in the *sublinear verifier* regime. Prior works either show HVZK IOPPs for interleaved Reed–Solomon codes in the unique decoding regime [AHIV17], or give IOPPs for interleaved general codes in the unique decoding regime [BCG20; GLSTW23] without providing zero-knowledge. None of these prior works consider constrained code testing relations. Theorem 1 improves upon these works, simultaneously achieving the desirable properties.

HVZK code switching. We obtain honest-verifier zero-knowledge IOPPs beyond the sublinear-verifier regime by providing an *HVZK code-switching* IOR from an interleaved code to another code (that, in particular, can itself be interleaved). Code-switching loosely refers to techniques for reducing a testing problem for a code to a testing problem for another *smaller* code, first introduced in the setting of tensor codes [RR20; RR22].

Theorem 2 (informal, code switching with HVZK). *For $b \in \{1, 2\}$, let $\mathcal{C}_b \subseteq \Sigma^{m_{\mathcal{C}_b}}$ be an $\mathbb{F}$ -additive code that has a $t_{\mathcal{C}_b}$ -query zero-knowledge encoding $\text{Enc}_{\mathcal{C}_b}: \mathbb{F}^{\ell_{\mathcal{C}_b}} \times \mathbb{F}^{r_{\mathcal{C}_b}} \rightarrow \Sigma^{m_{\mathcal{C}_b}}$ with error $\zeta_{\mathcal{C}_b}$ and encoding time $\tau_{\mathcal{C}_b}$. Fix an interleaving parameter $k \in \mathbb{N}$, security parameter $\lambda \in \mathbb{N}$, and proximity parameter $\delta \in (0, 1)$; let $t := O(\frac{\lambda}{-\log(1-\delta)})$. We construct a $O(k)$ -round IOR of proximity from $\mathcal{R}_{\mathcal{C}_1^{\equiv 2^k}, T_1}^{\text{lin}}$ to $\mathcal{R}_{\mathcal{C}_2, T_2}^{\text{lin}}$, where $T_2 = O(T_1 + T_{\mathcal{C}_2})$ and $T_{\mathcal{C}_2}$ is a quantity determined by $\mathcal{C}_2$'s generator matrix, with the following features.*

- *The prover sends $O(m_{\mathcal{C}_2}) + \tilde{O}(k \cdot \lambda)$ alphabet symbols, and the verifier makes t queries over alphabet Σ^{2^k} .*
- *The prover time is $O(2^k \cdot \ell_{\mathcal{C}_1} + \tau_{\mathcal{C}_2}) + \tilde{O}(k \cdot \lambda)$ and the verifier time is $O(2^k \cdot t) + \tilde{O}(k \cdot \lambda)$.*

- The protocol has round-by-round knowledge soundness error at most $2^{-\lambda}$ assuming $\mathbb{F}$ is large enough with respect to $\mathcal{C}_1$'s and $\mathcal{C}_2$'s mutual correlated agreement errors at distance δ .
- The protocol is honest-verifier zero-knowledge with error $\zeta_{\mathcal{C}_2}$ against distinguishers that perform at most $t_{\mathcal{C}_2}$ queries.

We consider a stronger notion of HVZK for IORs in Theorem 1, as we require an HVZK notion that is compatible with composition of IOR (accounting for later queries by algorithms we refer to as *distinguishers* on the output implicit instance). We elaborate on the “correct” HVZK notion for IORs in Section 2.3. Additionally, the fixed quantity $T_{\mathcal{C}_2}$ refers to the *code-switching friendliness* of the code, and we elaborate on this property in Section 2.6. Finally, similarly to Theorem 1, for common settings of the interleaving parameter k the zero-knowledge overhead is $o(1)$.

Polylogarithmic HVZK IOPP for constrained interleaved codes. Theorem 2 and the building blocks of Theorem 1 directly yield HVZK variants of WHIR [ACFY25] and Ligerito [NA25], as follows. Use the parameter settings in [NA25] (implicit in [ACFY25]) to apply Theorem 2 in sequence and conclude with Theorem 1. Moreover, Theorem 2 also improves upon [BCL22], who achieve HVZK IOPP for general linear codes but with $O(1)$ overhead and unique decoding soundness, while our result achieves $o(1)$ overhead and list decoding soundness.

In particular, instantiating this recipe with Reed–Solomon code yields the following corollary.

Corollary 3. *Let $\mathcal{C}^{\equiv 2^k}$ be an interleaved Reed–Solomon code with message length ℓ , evaluation domain size m , and with an $O(\lambda)$ -query zero-knowledge encoding, where $k = O(\log \log \ell)$. There exists an IOPP for $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, T}^{\text{lin}}$ with the following features.*

- The prover sends $O(m) + \tilde{O}(\lambda)$ alphabet symbols, and the verifier makes $O(\lambda)$ queries over alphabet $\mathbb{F}^{\log \ell}$.
- The prover time is $O(T + m \cdot \log m + \frac{\lambda \cdot \ell}{\log \ell}) + \tilde{O}(\lambda)$ and the verifier time is $O(T^{(\log \ell)} + \lambda \cdot \log \ell) + \tilde{O}(\lambda)$.
- The protocol has round-by-round knowledge soundness error at most $2^{-\lambda}$.
- The protocol is perfect honest-verifier zero-knowledge.

HVZK IOR from NP to constrained code testing. As an additional contribution, we provide a reduction from the NP-complete Rank-1-Constraint-Satisfaction (R1CS) problem to proximity testing of $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, T}^{\text{lin}}$. The reduction incurs $o(1)$ overhead over sending a single (interleaved) codeword, illustrating that the efficiency characteristics of the proximity test are also achievable in the reduction from an NP problem. The reduction is based on the same zero-knowledge sumcheck (see Section 2.5), which is an essential ingredient in the proofs of Theorem 1 and Theorem 2, and adapts the Σ -IOP for R1CS in [ACFY25]. In addition, we use a sublinear masking technique (similar to the one we use in our zero-knowledge sumcheck) to mask the R1CS matrix-vector product. Prior reductions from R1CS [BCL22] incur $O(1)$ overhead by sending full masking oracles.

Our reduction can, in particular, be instantiated with the interleaving of a linear-time-encodable code (with zero-knowledge encodings, as shown in [BCL22]) and, together with our proximity test, leads to an HVZK IOR from R1CS to testing proximity to a code, with a linear-time prover, list-decoding soundness, and $o(1)$ overhead. We note that our size comparison is against *non-zero-knowledge* protocols that use this style of reduction (“reduce R1CS to constraint code-testing”), e.g. [ACFY24; ACFY25; MZ25]. This improves upon the state-of-the-art for linear-time provers from [BCL22], which only achieves unique decoding soundness and constant overhead.

Composing the IOR from NP with Theorem 1 (or its polylogarithmic-verifier version that we provide in Theorem 4 in Section 2.7) achieves an HVZK IOP for NP with desirable efficiency

properties. We consider an example application of such efficiency improvements: [Fs24] develop an anonymous credential solution using the sublinear hash-based zkSNARK in [AHIV17], which is based on zero-knowledge IOPs with zero-knowledge overhead $\geq 2\times$ (and the corresponding non-private IOPs are no longer state of the art); our protocols can be used as a drop-in replacement to avoid this overhead. The effect is further amplified by the fact that we measure our overhead against *state-of-the-art* non-private IOPs.

Memory efficiency. While not an explicit focus of our work, Theorem 1 is plausibly amenable to space efficient streaming proving. The memory efficiency of the prover is concretely dominated by the space-efficiency of the encoding of the interleaved code and by the sumcheck. Crucially, an encoding of an interleaved code $\mathcal{C}^{\equiv K}$ can be performed in space equivalent to that required for the encoding of a message via the *base code* $\mathcal{C}$, which can be significantly smaller. This fact was already leveraged in [BBHV22] to achieve efficient streaming provers, for the specific case when $\mathcal{C}$ is a Reed–Solomon code. The sumcheck protocol can also be implemented in a time and space efficient manner (see e.g. [CTY11; VSBW13; BCFFMMZ25; NTZ25]), and plausibly the techniques can be adapted to achieve memory efficiency for our protocol.

On the choice of codes. Any $\mathbb{F}$ -additive code with an appropriate zero-knowledge encoding can be used in Theorem 1 and Theorem 2. These theorems use auxiliary *internal* smaller codes with zero-knowledge encoding, distinct from the *main* code being tested (exposed in the informal theorem statements). When setting the parameters for these theorems, we set the internal codes to be Reed–Solomon codes. Yet, the generality of our constructions (and full theorem statements) allows us to consider *other* codes for the codes being tested and for the internal codes.

We identify a class of codes that represents a midpoint between two popular choices of codes: (i) Reed–Solomon codes (an MDS code that is not linear-time encodable), and (ii) practical linear-time encodable codes. Specifically, we describe a class of codes obtained via *distance amplification based on dispersers*, and we argue that these codes achieve encoding time that approaches that of linear-time encodable MDS codes, but over a larger alphabet. We elaborate on this in Section 2.8.

2 Technical overview

We summarize the main ideas behind our results via an extended technical overview; the technical sections are long as we invested much effort to provide formal statements, constructions, and proofs.

- In Section 2.1 we review basic notions used in the technical overview: zero-knowledge codes, interactive oracle reductions, and round-by-round knowledge soundness.
- In Section 2.2 we describe a *non-succinct* HVZK IOPP for linear constraints, serving as a warmup to the techniques we use and as a baseline for our constructions (details are in Section 7).
- In Section 2.3 we present a new definition of HVZK for IORs that is compatible with sequential composition of IORs (details are in Section 4).
- In Section 2.4 we discuss a new *committed sumcheck relation*, which allows us to ensure zero-knowledge and efficient verifiers in our protocols.
- In Section 2.5 we present an **HVZK sumcheck reduction** from testing the interleaving of a base code to testing the base code itself (details are in Section 6), expressed via the aforementioned committed sumcheck relation. This is a key technical ingredient of our paper.
- In Section 2.6 we present an **HVZK code switching protocol** (details are in Section 10). This is the other key technical ingredient of our paper.
- In Section 2.7 we sketch how we prove our main theorems, by combining the basic HVZK IOPP, the sumcheck reduction, and code switching (via our HVZK IOR composition theorem).
- In Section 2.8 we propose the use of high-distance codes constructed via distance amplification for improved efficiency, in encoding time and in argument size. These codes are compatible with our constructions, and we deem them of independent interest for other probabilistic proofs.

2.1 Basic notions

We review basic notions used in this extended technical overview; full definitions appear in Section 3. Below $\mathbb{F}$ -*additive codes* refers to codes $\mathcal{C} \subseteq \Sigma^m$ whose alphabet Σ is an $\mathbb{F}$ -linear vector space (e.g., $\Sigma = \mathbb{F}^\iota$ for some $\iota \in \mathbb{N}$); when the alphabet $\Sigma = \mathbb{F}$ this becomes a linear code in the usual sense.

ZK codes. A code $\mathcal{C} \subseteq \Sigma^m$ has a *t-query zero-knowledge encoding* $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$ if the image of $\text{Enc}_{\mathcal{C}}$ is in $\mathcal{C}$ (i.e., $\text{Enc}_{\mathcal{C}}(\mathbb{F}^\ell, \mathbb{F}^r) \subseteq \mathcal{C}$) and, for every message $\text{msg} \in \mathbb{F}^\ell$ and index subset $S \subseteq [m]$ with $|S| \leq t$, the distribution $\text{Enc}_{\mathcal{C}}(\text{msg}, \mathbf{r} \leftarrow \mathbb{F}^r)[S]$ can be simulated efficiently, up to some error, given only S . Zero-knowledge codes are key ingredients in prior work on zero-knowledge IOPs (e.g. [ISVW13; BCL22; RW24]), and all codes in this work have zero-knowledge encodings. A concrete example one can keep in mind is the Reed–Solomon code, where encoding a message extended with randomness in $\mathbf{r} \in \mathbb{F}^t$ achieves perfect *t*-query zero-knowledge (see Proposition 3.19). Many other zero-knowledge codes are known, including some with linear-time encoding algorithms [BCL22].

IORs. Recent constructions of hash-based succinct arguments and accumulation schemes are obtained via a modular approach based on *interactive oracle reductions* (IORs) [BCGGRS19; BMNW25]. Informally, an IOR is an interactive oracle protocol in which the verifier, rather than outputting a decision bit, either rejects or outputs an instance for a target relation; in addition, instances are split into two parts, an *explicit instance* part that the verifier receives as input and an *implicit instance* part that the verifier receives as an oracle to query.

In more detail, an IOR (of proximity) from relation $\mathcal{R}$ to $\mathcal{R}'$ works as follows. The prover $\mathbf{P}$ receives as input $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$, whereas the verifier $\mathbf{V}$ receives as (explicit) input $\mathbf{x}$ and query access to $\mathbf{y}$. After interacting, $\mathbf{V}$ either rejects or outputs a new explicit instance $\mathbf{x}'$ and implicit instance

y' , and (in the honest case) $\mathbf{P}$ outputs a new witness w' such that $(x', y', w') \in \mathcal{R}'$. We say that IOR has (perfect) completeness if for every $(x, y, w) \in \mathcal{R}$ it always holds that $(x', y', w') \in \mathcal{R}'$.

RBR security. We target the strong notion of round-by-round security [CCHLRR18; CMS19; CY24; BMNW25; BCFW25] to ensure that our protocols are suitable to construct *non-interactive* succinct arguments (in the random oracle model). Informally, this amounts to establishing, for each round, a *round error* that upper bounds the probability of transitioning, in that round, from a *bad* state for the attacker (“the statement is false”) into a *good* state (“the next statement is true”). Specifically, we target the *straightline RBR knowledge soundness* notion for IORs in [BCFW25], which is suitable for code-agnostic protocols such as ours. The notion essentially states that an IOR from $\mathcal{R}$ to $\mathcal{R}'$ can be viewed as a sequence of one-round reductions each satisfying straightline knowledge soundness, which requires providing for each round a knowledge extractor and bounding its extraction error. Finally, since both $\mathcal{R}$ and $\mathcal{R}'$ involve implicit instances ($\mathbf{V}$ receives query access to an implicit instance y and outputs an implicit instance y') the extraction errors are a function of proximity parameters δ for $\mathcal{R}$ (“ y is δ -far from $\mathcal{R}$ ”) and δ' for $\mathcal{R}'$ (“ y' is δ' -close to $\mathcal{R}'$ ”).

List decoding and mutual correlated agreement. The RBR knowledge soundness errors that we achieve for our protocols depend, as in prior work, on several properties of the given code $\mathcal{C}$: distance, list-decoding size, and mutual correlated agreement error. We review the latter two.

- We denote the worst-case list size of a code $\mathcal{C}$ at distance $\delta \in (0, 1)$ as $|\Lambda(\mathcal{C}, \delta)| := \max_f |\Lambda(\mathcal{C}, f, \delta)|$ where $\Lambda(\mathcal{C}, f, \delta) := \{u \in \mathcal{C} : \Delta(u, f) \leq \delta\}$ is the set of codewords in $\mathcal{C}$ at relative distance $\leq \delta$ from f . Hence $\mathcal{C}$ ’s list decoding size captures the maximum number of codewords in the Hamming ball of a certain radius centered at any codeword in $\mathcal{C}$.
- Mutual correlated agreement (MCA) [ACFY25] is a strong notion of distance preservation. Informally, it bounds the probability that the random combination $f_1 + \gamma \cdot f_2$ is close to $\mathcal{C}$ when f_1, f_2 are not individually close to $\mathcal{C}$. We let $\epsilon_{\text{mca}}(\mathcal{C}, \delta)$ be the mutual correlated agreement error of $\mathcal{C}$ at distance $\delta \in (0, 1)$ (see Definition 3.14). In this overview we also consider the random combination $(1 - \gamma) \cdot f_1 + \gamma \cdot f_2$, and use the same notation because the error is the same.

2.2 Warmup: a simple zero-knowledge IOPP for constrained codes

We describe a simple honest-verifier zero-knowledge IOPP for linear constraints that works for any given zero-knowledge $\mathbb{F}$ -additive code. The protocol illustrates IOP-native ideas commonly used to achieve zero-knowledge, incurring a $2\times$ overhead. This serves as a simple concrete example to understand before we describe more efficient protocols; moreover, variants of this protocol will serve as a “base case” on small instances in our protocols.

An initial relation. Let $\mathcal{C} \subseteq \Sigma^m$ be an $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$. We define a linear constraint relation $\bar{\mathcal{R}}_{\mathcal{C}}$: (a) the explicit instance x contains a target $\mu \in \mathbb{F}$ and a coefficient vector $v \in \mathbb{F}^\ell$, (b) the implicit instance y is an oracle $f \in \Sigma^m$, and (c) the witness is a message $f \in \mathbb{F}^\ell$ and randomness $r \in \mathbb{F}^r$ such that $f = \text{Enc}_{\mathcal{C}}(f, r)$ and $\langle f, v \rangle = \mu$. Namely:

$$\bar{\mathcal{R}}_{\mathcal{C}} := \left\{ \left(\begin{array}{l} x = (\mu, v) \in \mathbb{F} \times \mathbb{F}^\ell, \\ y = f \in \Sigma^m, \\ w = (f, r) \in \mathbb{F}^\ell \times \mathbb{F}^r \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}}(f, r) \\ \wedge \langle f, v \rangle = \mu \end{array} \right\}. \quad (2)$$

As explained in Section 2.4, $\bar{\mathcal{R}}_{\mathcal{C}}$ is a simplification of the linear constraints relation that we use.

Recall: HVZK for IOPPs. We recall the standard definition of HVZK for interactive oracle proofs of proximity [BCFGRS17], which are for ternary relations $\mathcal{R}$ consisting of tuples (x, y, w)

where $\mathbf{x}$ is the explicit instance, $\mathbf{y}$ is the implicit instance, and $\mathbf{w}$ is the witness. The key feature of the definition is that the simulator has oracle access to the implicit instance $\mathbf{y}$ since the honest verifier also does, and we account for the query complexity of the simulator to $\mathbf{y}$.

Definition 2.1 (informal). Let $\text{IOP} = (\mathbf{P}, \mathbf{V})$ be an IOPP for a ternary relation $\mathcal{R}$. The **view** of $(\mathbf{P}, \mathbf{V})$ on $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$ is the random variable $\text{View}(\mathbf{P}, \mathbf{V}, \mathbf{x}, \mathbf{y}, \mathbf{w})$ that consists of $\mathbf{V}$'s randomness and the answers to $\mathbf{V}$'s queries in an interaction between $\mathbf{P}(\mathbf{x}, \mathbf{y}, \mathbf{w})$ and $\mathbf{V}^{\mathbf{y}}(\mathbf{x})$. IOP has **honest-verifier zero-knowledge with error ζ and query complexity $\mathbf{b}$** if there exists a polynomial-time simulator $\mathbf{S}$ such that the random variables $\text{View}(\mathbf{P}, \mathbf{V}, \mathbf{x}, \mathbf{y}, \mathbf{w})$ and $\mathbf{S}^{\mathbf{y}}(\mathbf{x})$ are ζ -close (in statistical distance) and $\mathbf{S}^{\mathbf{y}}(\mathbf{x})$ makes at most $\mathbf{b}$ queries to $\mathbf{y}$.

A simple protocol for linear constraints. We sketch a *non-succinct* HVZK IOPP for $\bar{\mathcal{R}}_{\mathcal{C}}$.

Construction 2.2. The verifier receives as explicit input $\mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F} \times \mathbb{F}^\ell$ and as implicit input $\mathbf{y} = f \in \Sigma^m$. The prover receives as input a message $\mathbf{f} \in \mathbb{F}^\ell$ and randomness $\mathbf{r} \in \mathbb{F}^r$ such that $\mathbf{f} = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r})$ and $\langle \mathbf{f}, \mathbf{v} \rangle = \mu$.

1. The prover samples $\mathbf{g} \in \mathbb{F}^\ell$ and $\mathbf{r}' \in \mathbb{F}^r$, and sends $g := \text{Enc}_{\mathcal{C}}(\mathbf{g}, \mathbf{r}') \in \Sigma^m$ and $\tilde{\mu} := \langle \mathbf{g}, \mathbf{v} \rangle \in \mathbb{F}$.
2. The verifier samples and sends $\varepsilon \leftarrow \mathbb{F}$.
3. The prover sends $\mathbf{h} := \mathbf{g} + \varepsilon \cdot \mathbf{f} \in \mathbb{F}^\ell$ and $\mathbf{r}^* := \mathbf{r}' + \varepsilon \cdot \mathbf{r} \in \mathbb{F}^r$.
4. The verifier samples $x_1, \dots, x_t \leftarrow [m]$, computes $h := \text{Enc}_{\mathcal{C}}(\mathbf{h}, \mathbf{r}^*) \in \Sigma^m$, and checks that:
 - $\langle \mathbf{h}, \mathbf{v} \rangle = \tilde{\mu} + \varepsilon \cdot \mu$.
 - for $i \in [t]$, $h(x_i) = g(x_i) + \varepsilon \cdot f(x_i)$ (by querying f and g at $x_1, \dots, x_t$). Recall that $h(x_i), g(x_i), f(x_i) \in \Sigma$ and they are compared as alphabet symbols.

Completeness of Construction 2.2 follows by linearity of $\text{Enc}_{\mathcal{C}}$, and its (straightline) round-by-round knowledge soundness follows via an analysis similar to prior work (e.g., [BCFW25]).

Lemma 2.3. For every $\delta \in (0, 1)$, Construction 7.2 has round-by-round knowledge soundness with errors $(\epsilon_{\text{mca}}(\mathcal{C}, \delta) + \frac{|\Lambda(\mathcal{C}^{\equiv 2}, \delta)|}{|\mathbb{F}|}, (1 - \delta)^t)$.

We wish to highlight the privacy properties of Construction 2.2, per the following lemma.

Lemma 2.4. If $\text{Enc}_{\mathcal{C}}$ is a t -query zero-knowledge encoding for $\mathcal{C}$ with error ζ , then Construction 2.2 is an honest-verifier zero-knowledge IOPP for $\bar{\mathcal{R}}_{\mathcal{C}}$ with error ζ and query complexity $\mathbf{b} = t$.

The intuition for the lemma is that if the encoding $\text{Enc}_{\mathcal{C}}$ is t -query zero-knowledge then the queries of the verifier can be simulated efficiently, so they do not leak any information about the underlying message. The remaining information sent by the prover can also be efficiently simulated.

Takeaway. Informally, the overhead to achieve HVZK in Construction 2.2 is $2\times$ (the prover sends g of the same size as f), and we will see how to achieve IOPPs for the same task with less overhead. Even so, Construction 2.2 will play the role of a “base case” on small instances in our protocols, so we describe and analyze (a generalization of) it in Section 7. Note also that Construction 2.2 is a “minimal example” of an IOP-native HVZK protocol, as it relies on both interaction and the verifier’s ability to query oracles; more restricted models such as IPs and PCPs typically require other techniques to achieve HVZK (or have significant limitations in their HVZK capabilities).

2.3 Composition for honest-verifier zero-knowledge IORs

We explain how prior (honest-verifier) zero-knowledge definitions of IOPs (and IOPPs), when naively adapted to IORs, do *not* preserve zero knowledge, *precluding a modular treatment of zero knowledge*. We address this via a **new definition (and composition theorem) for IORs**. This enables constructing zero-knowledge IOPs by modularly composing simple zero-knowledge IORs.

Definition 2.1 is insufficient for IORs. The notion of HVZK for IOPPs in Definition 2.1 can be straightforwardly adapted to IORs. However, this notion is insufficient for composing IORs, as we now explain via an (artificial) example. Let $\mathcal{R}$ be a binary relation of instance-witness pairs (x, w) , and let $\mathcal{R}'$ be the ternary relation $\{(x', y', w' = \perp) : (x', y') \in \mathcal{R}\}$. Consider two IORs.

- In IOR_1 , the prover sends w as an oracle to the verifier; the verifier makes no queries and then outputs the new instance $(x', y') := (x, w)$ for $\mathcal{R}'$. This IOR from $\mathcal{R}$ to $\mathcal{R}'$ has perfect completeness and soundness, and it is honest verifier zero-knowledge: the view of the verifier is empty (the verifier performs no queries), so it can be efficiently (and perfectly) simulated.
- In IOR_2 , the prover does nothing; the verifier queries the implicit instance y' entirely and then checks that $(x', y') \in \mathcal{R}$. This is an IOPP for $\mathcal{R}'$ with perfect completeness and soundness, and it is honest verifier zero-knowledge: the simulator in Definition 2.1 can query the entire implicit instance, so it can simulate the view of the honest verifier.

The composition of IOR_1 and IOR_2 yields a trivial IOP for $\mathcal{R}$. Note that IOR_1 and IOR_2 each satisfy Definition 2.1, yet the prover in their composition sends the witness to the verifier, which is not honest-verifier zero-knowledge (unless $\mathcal{R}$ is trivial). We conclude that Definition 2.1 is not strong enough to capture the notion of HVZK necessary for composing IORs (while preserving HVZK).

Variants of IOR_1 where the prover sends w encoded with a zero-knowledge code will be central to our constructions. The problem with Definition 2.1 is that it *does not account for the way in which the implicit instance y' output by IOR_1 is queried in IOR_2* .

Defining HVZK for IORs. We strengthen Definition 2.1 by requiring the simulator to additionally be responsible for simulating all future queries to the implicit instance y' output by the verifier. We model these queries via an oracle algorithm $\mathcal{D}$ that we call a *distinguisher*, because $\mathcal{D}$ captures the additional distinguishing opportunities arising from querying y' .

Definition 2.5 (informal). *Let $\text{IOR} = (\mathbf{P}, \mathbf{V})$ be an IOR from $\mathcal{R}$ to $\mathcal{R}'$. The **view** of IOR on $(x, y, w) \in \mathcal{R}$ with respect to an oracle algorithm $\mathcal{D}$ is the random variable $\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}, x, y, w)$ that consists of $\mathbf{V}$'s and $\mathcal{D}$'s randomnesses, the answers to $\mathbf{V}$'s and $\mathcal{D}$'s queries, and $\mathcal{D}$'s output out in this experiment: $\mathbf{P}(x, y, w)$ and $\mathbf{V}^y(x)$ interact yielding output (x', y', w') , and then $\mathcal{D}^{y'}(x')$ is run to produce out. IOR has **honest-verifier zero-knowledge with error ζ and query complexity b with respect to $\mathcal{D}$** if there exists a polynomial time simulator $\mathbf{S}$ such that the random variables $\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}, x, y, w)$ and $\mathbf{S}^{y, \mathcal{D}}(x)$ are ζ -close (in statistical distance) and $\mathbf{S}^{y, \mathcal{D}}(x)$ makes at most b queries to y .*

Above, the simulator $\mathbf{S}$ has black-box access to $\mathcal{D}$ so, in particular, can run $\mathcal{D}(x')$ to deduce simulated answers for its queries to y' . Note that Definition 2.5 collapses to Definition 2.1 when $\mathcal{D}$ makes no queries to y' (i.e., $\mathcal{D}$ is trivial).

IOR_1 is honest-verifier zero-knowledge with respect to the trivial distinguisher $\mathcal{D}^{y'}(x') = \perp$, but does not satisfy Definition 2.5 for any non-trivial distinguisher $\mathcal{D}^{y'}(x')$ that queries its implicit

instance y' , and in particular does not satisfy honest-verifier zero-knowledge with respect to the simulator for IOR_2 (unless the relation $\mathcal{R}$ is trivial).

The oracle algorithm $\mathcal{D}$ enables us to formulate a composition theorem, which we discuss next.

HVZK composition theorem. We informally state a composition theorem for HVZK IORs.

Lemma 2.6 (informal). *Let IOR_1 be an IOR from $\mathcal{R}_1$ to $\mathcal{R}_2$ and IOR_2 be an IOR from $\mathcal{R}_2$ to $\mathcal{R}_3$, and let $\text{IOR} := \text{IOR}_1 \circ \text{IOR}_2$ be the IOR from $\mathcal{R}_1$ to $\mathcal{R}_3$ obtained by composing these. Suppose that:*

- *IOR_2 is HVZK with error ζ_2 and query complexity b_2 with respect to $\mathcal{D}_2$, with simulator $\mathbf{S}_2$;*
- *IOR_1 is HVZK with error ζ_1 and query complexity b_1 with respect to $\mathcal{D}_1^y(x) := \mathbf{S}_2^{y, \mathcal{D}_2}(x)$.*

Then IOR is HVZK with error $\zeta_1 + \zeta_2$ and query complexity b_1 with respect to $\mathcal{D}_2$.

Proof sketch. Let $\mathbf{S}_1$ be the simulator for IOR_1 and define $\mathbf{S}^{y_1}(x_1) := \mathbf{S}_1^{y_1, \mathcal{D}_1}(x_1)$. By the honest-verifier zero-knowledge of IOR_1 with respect to $\mathcal{D}_1$, the distribution of $\mathbf{S}^{y_1}(x_1)$ has statistical distance at most ζ_1 from the experiment in which $(\mathbf{P}_1, \mathbf{V}_1)$ interact on (x_1, y_1, w_1) , outputting (x_2, y_2, w_2) , and then $\mathcal{D}_1^{y_2}(x_2)$ is run. Since $\mathcal{D}_1^{y_2}(x_2) = \mathbf{S}_2^{y_2, \mathcal{D}_2}(x_2)$, by honest-verifier zero-knowledge of IOR_2 with respect to $\mathcal{D}_2$, this latter part has statistical distance at most ζ_2 from running $(\mathbf{P}_2, \mathbf{V}_2)$ on (x_2, y_2, w_2) to obtain (x_3, y_3, w_3) and then running $\mathcal{D}_2^{y_3}(x_3)$. Thus, the distribution of $\mathbf{S}^{y_1}(x_1)$ has statistical distance at most $\zeta_1 + \zeta_2$ from an honest execution of IOR followed by $\mathcal{D}_2$. $\square$

Distinguisher classes in our constructions. Lemma 2.6 spells out the minimal condition for HVZK composition by relying on Definition 2.5 with $\mathcal{D}$ set to a specific algorithm. More generally we consider an extension of Definition 2.5 that requires the simulator to work for every algorithm $\mathcal{D}$ in a given class $\mathfrak{D}$. This strengthening enables preserving HVZK in a more flexible way: for HVZK composition, it suffices that the algorithm $\mathcal{D}$ arising from the second IOR’s simulator belongs to the class $\mathfrak{D}$. We typically construct IORs that satisfy this notion for a specific class of algorithms that is both general and restricted enough for our purposes: the class of *t-restricted distinguishers*, denoted $\mathfrak{D}^{\leq t}$, consisting of algorithms that make at most t queries to the implicit instance.

See Section 4 for the formal definition of HVZK for IORs and the associated composition theorem.

2.4 A “committed” relation for zero-knowledge

We describe and motivate the main relation that we use in this work. The relation is similar to constrained relations in prior work [ACFY25; BCFW25] but with differences that we motivate below: (1) succinct linear constraints to allow for a polylogarithmic verifier, and (2) a “committed” version of the relation to allow for zero-knowledge.

The relation in Equation 2 is insufficient. The linear constraint relation $\bar{\mathcal{R}}_{\mathcal{C}}$ in Equation 2 has two limitations from the perspective of our work. First, linear constraints are expressed via an *explicit* coefficient vector $v \in \mathbb{F}^\ell$ in the explicit instance x , which rules out verifiers that run in time $o(\ell)$; instead, we seek a *succinct* representation of linear constraints. Second, the target $\mu \in \mathbb{F}$ in the explicit instance x contains information about the secret message f (even if $\text{Enc}_{\mathcal{C}}$ is a zero-knowledge encoding), whereas we seek to avoid revealing even a single linear constraint about the message f . We modify $\bar{\mathcal{R}}_{\mathcal{C}}$ to address these two limitations.

Modification 1: succinct linear constraints. We replace v in x with a succinct state st that implicitly represents such a vector. We hardcode in the relation a *succinct linear form* sl that, given as input a “small” state st (from some space determined by sl), outputs an explicit vector $v \in \mathbb{F}^\ell$. A simple example to keep in mind is the succinct constraint sl mapping $\text{st} = \alpha \in \mathbb{F}$ to the

vector $\text{sl}(\text{st}) := (\alpha^{i-1})_{i \in [\ell]}$. These succinct linear forms are similar to sumcheck-based constraints in [ACFY25]. While the two constraint classes are formally incomparable, both would suffice for our protocols; we opt to use succinct linear constraints as they are a better notational fit in this paper.

Modification 2: masks in the constraint. The target $\mu \in \mathbb{F}$ leaks the linear constraint $\langle \mathbf{f}, \mathbf{v} \rangle$ about the secret message $\mathbf{f}$. We further modify the relation to incorporate sumcheck-friendly masks intended to eliminate the leakage in μ . We introduce an additional linear code $\mathcal{C}_{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{zk}}}$ with zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{zk}}} : \mathbb{F}^{\ell_{\text{zk}}} \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{zk}}}$ that we use to encode small masks $\xi_1, \dots, \xi_n \in \mathbb{F}^{\ell_{\text{zk}}}$ (as many masks as there are variables). We associate linear constraints $\mathbf{u}_1, \dots, \mathbf{u}_n \in \mathbb{F}^{\ell_{\text{zk}}}$ to these masks, which need not be succinct (as the masks are in fact small).

Committed sumcheck relation. The above considerations lead to the relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}$ where: (a) the explicit instance $\mathbf{x} = ((\mu, \text{st}, (\mathbf{u}_i)_{i \in [n]}))$ contains the target, succinct linear constraint, and explicit linear constraints; (b) the implicit instance $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$ contains the main encoding and the encoded masks; and (c) the witness $\mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$ contains the underlying secret messages and secret masks and their corresponding secret randomnesses. Namely:

$$\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}} := \left\{ \left(\begin{array}{l} \mathbf{x} = ((\mu, \text{st}, (\mathbf{u}_i)_{i \in [n]})), \\ \mathbf{y} = (f, (\xi_i)_{i \in [n]}), \\ \mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]}) \end{array} \right) \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r}) \\ \wedge \forall i \in [n]: \xi_i = \text{Enc}_{\mathcal{C}_{\text{zk}}}(\xi_i, \mathbf{r}_i) \\ \wedge \langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \mathbf{u}_i \rangle = \mu \end{array} \right\}. \quad (3)$$

We call this a **committed sumcheck relation**: the implicit instance $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$ can informally be viewed as a commitment to the secret message $\mathbf{f}$ and masks $(\xi_i)_{i \in [n]}$, which are jointly constrained via a sumcheck-friendly linear constraint. The encoding f is the “main” oracle, while the encodings $(\xi_i)_{i \in [n]}$ are much smaller encoded “mask” oracles: they appear in the joint constraint to “hide” the value of a linear constraint on $\mathbf{f}$. We refer to the last constraint as the *joint* constraint, which combines the masks as well as the main oracle.

Verifier succinctness in our protocols will come from the succinct state st : in our sumcheck reduction, the verifier will only need to evaluate a “fold” of st , which is in polylogarithmic time for the succinct linear forms that we consider.

2.5 An honest-verifier zero-knowledge sumcheck reduction

We present an HVZK sumcheck *reduction* from testing the (constrained) interleaving of a code to testing the (constrained) code itself plus several small testing problems for another code (with a zero-knowledge encoding); the reduction is based on an extension to zero-knowledge codes of the notion of interleaved folding in [ACFY25; NA25; BCFW25]. Later in Section 2.7 we discuss how we use this HVZK sumcheck reduction multiple times to obtain our results: we use it to establish Theorem 1 and Theorem 2, and we use it for an HVZK variant of [ACFY25]’s sumcheck-query IOP reduction from R1CS to testing interleaved codes.

Code interleaving and its folding. Recall that the K -interleaving of a code $\mathcal{C} \subseteq \Sigma^m$ is the code $\mathcal{C}^{\equiv K} \subseteq (\Sigma^K)^m$ consisting of words $u \in (\Sigma^K)^m$ such that there exist words $v_1, \dots, v_K \in \Sigma^m$ in $\mathcal{C}$ with $u(z) = (v_1(z), \dots, v_K(z)) \in \Sigma^K$ for every $z \in [m]$. In this paper we consider the interleaving operation for $K = 2^k$ and we define interleaving directly in terms of a zero-knowledge encoding $\text{Enc}_{\mathcal{C}^{\equiv 2^k}}$ that separately encodes 2^k pieces of the given message, chosen in a special way based on multilinear polynomials, via the given zero-knowledge encoding $\text{Enc}_{\mathcal{C}}$ for $\mathcal{C}$. Below eq is the

k -variable equality polynomial

$$\text{eq}(X_1, \dots, X_k, Y_1, \dots, Y_k) := \prod_{i=1}^k (X_i \cdot Y_i + (1 - X_i) \cdot (1 - Y_i)).$$

Definition 2.7. Let $\mathcal{C} \subseteq \Sigma^m$ be a code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$. For $k \in \mathbb{N}$, the 2^k -interleaving of $\mathcal{C}$ is the code $\mathcal{C}^{\equiv 2^k} \subseteq (\Sigma^{2^k})^m$ obtained via the zero-knowledge encoding $\text{Enc}_{\mathcal{C}^{\equiv 2^k}}: \mathbb{F}^{2^k \ell} \times \mathbb{F}^{2^k r} \rightarrow (\Sigma^{2^k})^m$ defined as follows. We split a message $\mathbf{f} \in \mathbb{F}^{2^k \ell}$ and randomness $\mathbf{r} \in \mathbb{F}^{2^k r}$ into 2^k pieces $(\mathbf{f}_{\mathbf{b}})_{\mathbf{b} \in \{0,1\}^k}$ and $(\mathbf{r}_{\mathbf{b}})_{\mathbf{b} \in \{0,1\}^k}$ by viewing both as multilinear polynomials $\hat{\mathbf{f}} \in \mathbb{F}^{<2}[X_1, \dots, X_{k+\log \ell}]$ and $\hat{\mathbf{r}} \in \mathbb{F}^{<2}[X_1, \dots, X_{k+\log r}]$ and writing

$$\hat{\mathbf{f}}(X_1, \dots, X_{k+\log \ell}) = \sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{b}, (X_1, \dots, X_k)) \cdot \mathbf{f}_{\mathbf{b}}(X_{k+1}, \dots, X_{k+\log \ell}), \quad (4)$$

$$\hat{\mathbf{r}}(X_1, \dots, X_{k+\log r}) = \sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{b}, (X_1, \dots, X_k)) \cdot \mathbf{r}_{\mathbf{b}}(X_{k+1}, \dots, X_{k+\log r}). \quad (5)$$

Then we define

$$\forall z \in [m], \text{Enc}_{\mathcal{C}^{\equiv 2^k}}(\mathbf{f}, \mathbf{r})(z) := (\text{Enc}_{\mathcal{C}}(\mathbf{f}_{\mathbf{b}}, \mathbf{r}_{\mathbf{b}})(z))_{\mathbf{b} \in \{0,1\}^k} \in \Sigma^{2^k}.$$

Next, we define the notion of *interleaved folding*.

Definition 2.8. The **interleaved fold** of a function $f: [m] \rightarrow \Sigma^{2^k}$ at the point $\gamma \in \mathbb{F}^i$ with $i \in [k]$ is the function $\text{Fold}(f, \gamma): [m] \rightarrow \Sigma^{2^{k-i}}$ defined as follows. We identify $\{0,1\}^k \cong \{0,1\}^i \times \{0,1\}^{k-i}$, and for every $z \in [m]$ we write $f(z) = (f_{(\mathbf{b}, \mathbf{c})}(z))_{\mathbf{b} \in \{0,1\}^i, \mathbf{c} \in \{0,1\}^{k-i}}$, meaning that $f_{(\mathbf{b}, \mathbf{c})}(z) \in \Sigma$ is the coordinate of the vector $f(z) \in \Sigma^{2^k}$ indexed by $(\mathbf{b}, \mathbf{c})$. Then, for every $z \in [m]$ we define

$$\text{Fold}(f, \gamma)(z) \in \Sigma^{2^{k-i}} \quad \text{s.t.} \quad \forall \mathbf{c} \in \{0,1\}^{k-i}, (\text{Fold}(f, \gamma)(z))_{\mathbf{c}} := \sum_{\mathbf{b} \in \{0,1\}^i} \text{eq}(\mathbf{b}, \gamma) \cdot f_{(\mathbf{b}, \mathbf{c})}(z).$$

The expressions are well-defined because Σ is an $\mathbb{F}$ -linear vector space (e.g., $\Sigma = \mathbb{F}^\iota$ for some $\iota \in \mathbb{N}$).

The key feature of the above definitions is that “encode then fold” equals “fold then encode”.

Claim 2.9 (informal). For every $\mathbf{f} \in \mathbb{F}^{2^k \ell}$, $\mathbf{r} \in \mathbb{F}^{2^k r}$, $i \in [k]$, and $\gamma \in \mathbb{F}^i$,

$$\text{Fold}(\text{Enc}_{\mathcal{C}^{\equiv 2^k}}(\mathbf{f}, \mathbf{r}), \gamma) = \text{Enc}_{\mathcal{C}^{\equiv 2^{k-i}}}(\text{Fold}(\mathbf{f}, \gamma), \text{Fold}(\mathbf{r}, \gamma)).$$

The above definitions and identity are a direct extension to zero-knowledge codes of the notions of folding for arbitrary linear codes that appear in [ACFY25; NA25; BCFW25]. These folding notions are similar to, but distinct from, other notions of folding that apply to several classes of codes in [AHIV17; BCG20; BBHR18; BGKS20; GLSTW23; ZCF24; DP24; ACFY24].

An HVZK committed-sumcheck IOR. Armed with these definitions, we can *fold* an initial interleaved codeword and then claim that the resulting codeword is an interleaved codeword with a *smaller* interleaving factor, while keeping track of the succinct linear constraints. This intuition is what underlies the HVZK committed-sumcheck IOR that we now describe. Specifically, we outline a queryless IOR from $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{\text{zk}}, \text{sl}}$ to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}'}$ where $(n, \mathcal{C}_{\text{zk}}, \text{sl})$ are as in the definition of the relation in Equation 3, $\mathcal{C}$ is an $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}$, and $k \in \mathbb{N}$.

Construction 2.10. The verifier receives as explicit input $\mathbf{x} := (\mu, \mathbf{st}, (\mathbf{u}_i)_{i \in [n]})$ and oracle access to the implicit input $\mathbf{y} := (f, (\xi_i)_{i \in [n]})$. In the honest case, the prover receives as input a witness $\mathbf{w} := (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \text{sl}}$.

1. The prover sends oracles $s_1, \dots, s_k \in \Sigma_{\text{zk}}^{m_{\text{zk}}}$. The honest prover samples masks $\mathbf{s}_1, \dots, \mathbf{s}_k \in \mathbb{F}^{\ell_{\text{zk}}}$ and randomnesses $\mathbf{r}'_1, \dots, \mathbf{r}'_k \in \mathbb{F}^{r_{\text{zk}}}$, and sets $s_j := \text{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{s}_j, \mathbf{r}'_j)$ for $j \in [k]$. The masks are interpreted as univariate polynomials $\hat{\mathbf{s}}_1, \dots, \hat{\mathbf{s}}_k \in \mathbb{F}^{<\ell_{\text{zk}}}[\mathbf{X}]$: $\forall j \in [k], \hat{\mathbf{s}}_j(\mathbf{X}) := \sum_{l=1}^{\ell_{\text{zk}}} (s_j)_l \cdot \mathbf{X}^{l-1}$.
2. The prover sends $\tilde{\mu} \in \mathbb{F}$. The honest prover sets $\tilde{\mu} := \sum_{(b_1, \dots, b_k) \in \{0,1\}^k} (\hat{\mathbf{s}}_1(b_1) + \dots + \hat{\mathbf{s}}_k(b_k))$.
3. The verifier samples and sends $\varepsilon \leftarrow \mathbb{F}$.
4. The prover and the verifier interpret $\mathbf{f}$ as a multilinear polynomial $\hat{\mathbf{f}} \in \mathbb{F}^{<2}[\mathbf{X}_1, \dots, \mathbf{X}_{k+\log \ell}]$ (see Equation 4) and they run a k -round sumcheck protocol on the claim

$$\sum_{(b_1, \dots, b_k) \in \{0,1\}^k} (\hat{\mathbf{s}}_1(b_1) + \dots + \hat{\mathbf{s}}_k(b_k) + \varepsilon \cdot \hat{G}(b_1, \dots, b_k)) = \tilde{\mu} + \varepsilon \cdot \mu$$

where

$$\hat{G}(\mathbf{X}_1, \dots, \mathbf{X}_k) := \left(\sum_{\mathbf{a} \in \{0,1\}^{\log \ell}} \hat{\mathbf{f}}(\mathbf{X}_1, \dots, \mathbf{X}_k, \mathbf{a}) \cdot \text{Fold}(\text{sl}(\mathbf{st}), \mathbf{X}_1, \dots, \mathbf{X}_k)[\mathbf{a}] \right) + \sum_{i \in [n]} \langle \xi_i, \mathbf{u}_i \rangle$$

and Fold is defined to mirror Definition 2.8:

$$\text{Fold}(\text{sl}(\mathbf{st}), \mathbf{X}_1, \dots, \mathbf{X}_k) := \left(\sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{X}_1, \dots, \mathbf{X}_k, \mathbf{b}) \cdot \text{sl}(\mathbf{st})[\mathbf{a}, \mathbf{b}] \right)_{\mathbf{a} \in \{0,1\}^{\log \ell}}.$$

In each round $j \in [k]$ the prover sends a univariate polynomial $\hat{h}_j \in \mathbb{F}^{<\ell_{\text{zk}}}[\mathbf{X}]$ and the verifier replies with a random $\gamma_j \leftarrow \mathbb{F}$. Define $\gamma_j := (\gamma_1, \dots, \gamma_j) \in \mathbb{F}^j$ for every $j \in [k]$, and set $\gamma := \gamma_k$.

5. The verifier sets $\mu' := \hat{h}_k(\gamma_k)$ and outputs a new instance for the target relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}'}$ with the joint constraint:

$$\hat{\mathbf{s}}_1(\gamma_1) + \dots + \hat{\mathbf{s}}_k(\gamma_k) + \langle \text{Fold}(\mathbf{f}, \gamma), \varepsilon \cdot \text{Fold}(\text{sl}(\mathbf{st}), \gamma) \rangle + \sum_{i \in [n]} \langle \xi_i, \varepsilon \cdot \mathbf{u}_i \rangle = \mu'.$$

That is, sl' is defined to express the succinct linear constraint $(\mathbf{st}, \gamma, \varepsilon) \mapsto \varepsilon \cdot \text{Fold}(\text{sl}(\mathbf{st}), \gamma)$ and the outputs are:

- $\mathbf{x}' := (\mu', (\mathbf{st}, \gamma, \varepsilon), (\varepsilon \cdot \mathbf{u}_i)_{i \in [n]}, ((\gamma_j^{l-1})_{l=1}^{\ell_{\text{zk}}})_{j \in [k]})$ where $(\mathbf{st}, \gamma, \varepsilon)$ is a succinct state for sl' and for each $j \in [k]$ the vector $(\gamma_j^{l-1})_{l=1}^{\ell_{\text{zk}}}$ is chosen so that $\langle \mathbf{s}_j, (\gamma_j^{l-1})_{l=1}^{\ell_{\text{zk}}} \rangle = \hat{\mathbf{s}}_j(\gamma_j)$;
- $\mathbf{y}' := (\text{Fold}(\mathbf{f}, \gamma), (\xi_i)_{i \in [n]}, (s_j)_{j \in [k]})$; and
- in the honest case $\mathbf{w}' := (\text{Fold}(\mathbf{f}, \gamma), \text{Fold}(\mathbf{r}, \gamma), (\xi_i, \mathbf{r}_i)_{i \in [n]}, (s_j, \mathbf{r}'_j)_{j \in [k]})$.

Completeness. The completeness of the construction follows by Claim 2.9 after observing that $\text{Fold}(\mathbf{f}, \gamma)[a] = \hat{\mathbf{f}}(\gamma, \text{binary}(a))$ where $a \in [\ell]$ and $\text{binary}(a) \in \{0,1\}^{\log \ell}$ is the binary representation of a (the same holds also for the folding of $\text{sl}(\mathbf{st})$). This follows by Definition 2.8:

$$\begin{aligned} \text{Fold}(\mathbf{f}, \gamma)[a] &= \sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{b}, \gamma) \cdot \mathbf{f}_{\mathbf{b}}[a] = \sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{b}, \gamma) \cdot \mathbf{f}_{\mathbf{b}}[a] \\ &= \sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{b}, \gamma) \cdot \mathbf{f}_{\mathbf{b}}(\text{binary}(a)) = \hat{\mathbf{f}}(\gamma, \text{binary}(a)). \end{aligned}$$

Honest-verifier zero-knowledge. We sketch why Construction 2.10 is HVZK.

Lemma 2.11. *Let $t \in \mathbb{N}$ and assume $\text{char}(\mathbb{F}) \neq 2$. If $\text{Enc}_{\mathcal{C}_{\text{zk}}} : \mathbb{F}_{\text{zk}}^\ell \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{zk}}}$ is a t_{zk} -query zero-knowledge encoding for $\mathcal{C}_{\text{zk}}$ with error $\zeta_{\mathcal{C}_{\text{zk}}}$ and $\ell_{\text{zk}} \geq 2$, then Construction 6.3 is honest-verifier zero-knowledge with error $k \cdot \zeta_{\mathcal{C}_{\text{zk}}}$ with respect to the family of distinguishers $\mathcal{D}$ making t queries to $\mathbf{y}'$ and t_{zk} queries to each of the other oracles in $\mathbf{y}'$. The simulator makes t queries to f and t_{zk} queries to each oracle $(\xi_i)_{i \in [n]}$ in $\mathbf{y}$.*

Proof sketch. We outline the zero-knowledge simulator. First, since the distinguisher class is restricted to make at most t_{zk} queries to the oracles $(s_j)_{j \in [k]}$ in $\mathbf{y}'$, the simulator uses the simulator for the code $\mathcal{C}_{\text{zk}}$ to sample answers for these queries. Taking a union bound gives the simulation error $k \cdot \zeta_{\mathcal{C}_{\text{zk}}}$. Second, for the sumcheck part, the simulator samples $\tilde{\mu}, \varepsilon, (\gamma_j)_{j \in [k]}$ uniformly at random, and then samples $(\hat{h}_j)_{j \in [k]}$ uniformly at random *conditioned* on satisfying all consistency checks in the sumcheck protocol. This matches the distribution of the sumcheck protocol, as we show via a linear-algebraic argument by counting the degrees of freedom provided by the random masks (this part is similar to the analysis of the zero-knowledge sumcheck in [XZZPS19]). The assumption $\text{char}(\mathbb{F}) \neq 2$ ensures that the mask contribution in the sumcheck claim does not cancel. $\square$

Round-by-round knowledge soundness. We sketch why Construction 2.10 satisfies straightline RBR knowledge soundness [BCFW25]. The construction is an IOR of proximity from $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{\text{zk}}, \text{sl}}$ to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}'}$, so the knowledge soundness error depends on proximity parameters for the first relation and for the second relation. Moreover, implicit instances involve multiple oracles of different sizes so we use multiple proximity parameters: a proximity parameter δ for the main oracle, and a proximity parameter δ_{zk} for all mask oracles. One way to express this, similarly to [BMNW25; BCFW25], is via the “relaxed” relation corresponding to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}$ (Equation 3) defined as follows:

$$\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}^{\delta, \delta_{\text{zk}}} := \left\{ \left(\begin{array}{l} \mathbb{X}, \\ \mathbf{y} = (f, (\xi_i)_{i \in [n]}), \\ (\mathbf{w}, \bar{\mathbf{y}} := (f, \xi_1, \dots, \xi_n)) \end{array} \right) \mid \begin{array}{l} \Delta(f, \bar{f}) \leq \delta \\ \wedge \forall i \in [n], \Delta(\xi_i, \bar{\xi}_i) \leq \delta_{\text{zk}} \\ \wedge (\mathbb{X}, \bar{\mathbf{y}}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}} \end{array} \right\}.$$

Finally, we target distance preservation, so we use the *same* proximity parameters $\delta, \delta_{\text{zk}}$ to relax the initial and target relations. In light of all this, the round-by-round knowledge soundness of the construction can be stated as follows.

Lemma 2.12 (informal). *For every $\delta, \delta_{\text{zk}} \in (0, 1)$, Construction 2.10 has round-by-round knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{\text{zk}}, \text{sl}}^{\delta, \delta_{\text{zk}}}, \tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}'}^{\delta, \delta_{\text{zk}}})$ with the following round errors:*

$$\left(\frac{|\Lambda(\mathcal{C}^{\equiv 2^k}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \left(\epsilon_{\text{mca}}(\mathcal{C}^{\equiv 2^{k-j}}, \delta) + \frac{\ell_{\text{zk}} \cdot |\Lambda(\mathcal{C}^{\equiv 2^{k-(j-1)}}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|} \right)_{j \in [k]} \right).$$

Proof sketch. The proof is similar in spirit to analyses in [ACFY25; BCFW25], but is more challenging due to the fact that multiple codewords participate in the constraint being checked. Typically, (knowledge) soundness analyses for IORs involve two types of errors: proximity preservation errors and list size errors. Proximity preservation errors are captured, as in prior work, via a quantity $\epsilon_{\text{mca}}(\mathcal{C}, \delta)$ that equals the MCA (mutual correlated agreement) error of a given code $\mathcal{C}$ at distance δ . List size errors require some care in our setting, as we now explain.

In each round we upper bound the probability that *any* codeword at distance δ from the sent oracles causes that round’s “claim” to flip from false to true. This leads to taking a union bound over the list size of the code at distance δ . However, “breaking” the interleaved word f into its 2^k parts and then naively applying a union bound over *each* one would lead to a union bound over $|\Lambda(\mathcal{C}, \delta)|^{2^k}$ many events. Similarly, for the $n + k$ masks allegedly in the code $\mathcal{C}_{\text{zk}}$, we may end up with a union bound over $|\Lambda(\mathcal{C}_{\text{zk}}, \delta_{\text{zk}})|^{n+k}$ many events. Such an approach would ultimately require selecting an extremely large field $\mathbb{F}$.

Instead, in our analysis we analyze codewords relative to the interleaved codes $\mathcal{C} \equiv 2^k$ and $\mathcal{C}_{\text{zk}} \equiv n+k$. As interleaving *does not* increase a code’s list-size by more than a constant factor [GGR11], this yields a significantly improved soundness error. This crucially relies on the fact that our protocol *correlates the queries* (over a larger alphabet) to these oracles (which also reduces the argument size of the resulting argument after compilation). $\square$

Comparisons. We compare our zero-knowledge sumcheck with prior variants. While all known zero-knowledge sumchecks are similar at a high level in their masking of sumcheck polynomials, our specific construction differs from other variants in crucial aspects motivated by our context.

- [CFS17; BCL22; Dia25] considers a “full mask” in the form of a random polynomial added to the original one via a random linear combination; this incurs an overhead of at least 2, which is too expensive for us. Moreover, [BCL22] does not employ out-of-domain techniques to achieve list-decoding soundness and, in particular, only targets constant soundness error.
- [XZZPS19] describes a zero-knowledge sumcheck, based on polynomial commitment schemes, that uses a small mask for each round rather than a full mask at the start. Our protocol also uses small masks but the different setting of IOPs/IORs demands different techniques: the small masks are *oracles* that we carry throughout the reduction (as opposed to them being committed and opened once). Moreover, the protocol in [XZZPS19] reveals an evaluation of the masks and thus an evaluation of the main oracle; but we cannot afford this in our setting, which is why we define the relation to accumulate claims about the masks. Finally, our sumcheck is a reduction from an interleaved code to the base code, while the sumcheck in [XZZPS19] serves as an ingredient in the GKR protocol, and hence it is used in a different context.
- [RW24] uses zero-knowledge codes for masks tailored to the tensor-code sumcheck. Their context is similar to ours: they reduce the testing of a tensor code to the testing of the base code, and they similarly use small masks to achieve zero-knowledge (in their case they achieve *malicious*-verifier zero-knowledge while we achieve honest-verifier zero-knowledge). The main difference between their approach and ours is that they use tensor codes while we use interleaved codes. Moreover, we achieve list-decoding soundness, while [RW24] targets constant soundness, and additionally, our work emphasizes the efficiency of the prover and the verifier, whereas the prover in [RW24] runs in polynomial time.

There are other variants of zero-knowledge sumchecks in the PCP setting [GOS24; GOS25], though these are less relevant to this work as they are closely related to the “full mask” approach in [CFS17] (which has an overhead of at least 2) and are adapted to the unique challenges of PCPs.

2.6 HVZK code switching

We describe our HVZK code-switching reduction: a HVZK IOR that reduces a claim in $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}$ for the code $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\ell)^m$ to $\mathcal{R}_{\mathcal{C}', \mathcal{C}_{\text{zk}}, \text{sl}'}$ for an *arbitrary code* $\mathcal{C}' \subseteq (\Sigma')^{m'} \equiv (\mathbb{F}^{\ell'})^{m'}$. The setting that we consider presents two significant challenges compared to previous codeswitches (e.g. [RR20; RW24; ACFY24; ACFY25; BCFRRZ25; BMMS25a; BCFW25; BFRW25]).

Code-switching without ZK. Known code-switching methods follow a template.

1. The verifier has access to an oracle $f \in \Sigma^m$. In the honest case, $f = \mathcal{C}(\mathbf{f}) \wedge \Phi(\mathbf{f}) = 1$, i.e., f is the encoding under the code $\mathcal{C}$ of a message $\mathbf{f}$ that satisfies a given predicate Φ .
2. The prover sends $g \in (\Sigma')^{m'}$. In the honest case, $g = \mathcal{C}'(\mathbf{f})$, i.e., g is the encoding of $\mathbf{f}$ under the code $\mathcal{C}'$.
3. The verifier samples challenges $x_1, \dots, x_t \in [m]$ and queries $f(x_1), \dots, f(x_t)$.
4. The prover and verifier engage in a proximity protocol for the claim “ $\mathbf{g} = \mathcal{C}^{-1}(g) \wedge \Phi'(\mathbf{g}) = 1$ ” where $\Phi'(\mathbf{g}) := \Phi(\mathbf{g}) \wedge \bigwedge_{i \in [t]} \mathcal{C}(\mathbf{g})[x_i] = f(x_i)$.

The checks $\bigwedge_{i \in [t]} (\mathcal{C}(\mathbf{g})[x_i] = f(x_i))$ are a “consistency check” that the new oracle g indeed encodes the same message as the original oracle f . This template has not been applied to zero-knowledge codes, with the notable exception of [RW24], whose code-switching relies on specific properties of (zero-knowledge) tensor codes.

Code-switching with ZK encodings. When using a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}$ for a code $\mathcal{C}$, the codeword $f = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r})$ depends on both the message $\mathbf{f}$ and the randomness $\mathbf{r}$. Hence, in order to be complete, a consistency check must also take into account $\mathbf{r}$. One option to address this would be for the new oracle g to encode, under $\mathcal{C}'$ both the message and the randomness. Unfortunately, since later rounds will require to invoke the sumcheck protocol, that would require introducing padding that would involve at least a $2\times$ overhead. Instead, we leverage the committed sumcheck relation introduced in Section 2.4 to introduce an *additional mask* encoding the randomness using $\mathcal{C}_{\text{zk}}$. Then, supposing that $g = \text{Enc}_{\mathcal{C}'}(\mathbf{f}, \mathbf{r}')$ and $s = \text{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{r}, \mathbf{r}'')$ (where $\mathbf{r}', \mathbf{r}''$ are freshly sampled pieces of randomness), for $x \in [m \cdot \iota]$ one can write the following claim

$$f(x) = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r})[x] = \left(\mathbf{G}_{\mathcal{C}} \cdot \begin{bmatrix} \mathbf{f} \\ \mathbf{r} \end{bmatrix} \right) [x] = \langle \mathbf{f}, \mathbf{G}_{\mathcal{C}}^{\#}[x] \rangle + \langle \mathbf{r}, \mathbf{G}_{\mathcal{C}}^{\$}[x] \rangle,$$

where $\mathbf{G}_{\mathcal{C}} = [\mathbf{G}_{\mathcal{C}}^{\#}, \mathbf{G}_{\mathcal{C}}^{\$}] \in \mathbb{F}^{(\iota \cdot m) \times (\ell + r)}$ is the linear map associated with $\text{Enc}_{\mathcal{C}}$ (recall that we interpret alphabet symbols in Σ as vectors in $\mathbb{F}^\ell$ and then again reinterpret the output of the encoding map as m symbols of $\mathbb{F}^\ell$). This claim can be written as a succinct linear form via our committed sumcheck relation.

OOD sampling. Efficient support for proximity parameters in the list-decoding regime typically relies on some form of out-of-domain (OOD) sampling [BGKS20]; without it, the above ideas would lead to inefficient protocols (regardless of zero knowledge considerations). Informally, OOD sampling refers to methods that force the prover, after sending an oracle f , to “choose” a single codeword in $\Lambda(\mathcal{C}, f, \delta)$ (if $\delta > \delta(\mathcal{C})/2$ then it can be that $|\Lambda(\mathcal{C}, f, \delta)| > 1$). Several methods for this task are used in the literature [BGKS20; ACY23; ACFY24; ACFY25; BMNW25; BCFW25; BFRW25; MZ25]; here we follow the presentation of [BCFW25]. Let $\mathcal{C} \subseteq \Sigma^m$ be a code with message space $\mathbb{F}^\ell$, and fix a function $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{\ell_{\text{ood}} \times \ell}$. OOD sampling roughly works as follows.

1. The prover sends $f \in \Sigma^m$. In the honest case, $f = \mathcal{C}(\mathbf{f})$.

2. The verifier samples $\rho \leftarrow D_{\text{ze}}$.
3. The prover sends $\mathbf{y} \in \mathbb{F}^{t_{\text{ood}}}$. In the honest case $\mathbf{y} := \text{ze}(\rho) \cdot \mathbf{f}$.
4. The prover and verifier proceed, with the additional constraint “ $\text{ze}(\rho) \cdot \mathcal{C}^{-1}(\mathbf{f}) = \mathbf{y}$ ”.

[BCFW25] shows that, for appropriate choices of ze , with high probability over ρ a *single* codeword in $\Lambda(\mathcal{C}, \mathbf{f}, \delta)$ satisfies the aforementioned additional constraint, and this property enables building efficient protocols in the list-decoding regime.

However, the above protocol is *not compatible* with zero-knowledge. The vector $\mathbf{y}$ leaks information about $\mathbf{f}$ (which encodes information about the private witness). The tension is inherent, in that the OOD protocol is supposed to uniquely identify a single codeword (and thus witness), while the zero-knowledge guarantee mandates that no information is revealed about this witness.

OOD sampling with ZK. We resolve this tension by extending the function ze to allow for additional randomization. Let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times (\ell + r)}$ be a function. The revised OOD sample is set as

$$\mathbf{y} := \text{ze}(\rho) \cdot \begin{bmatrix} \mathbf{f} \\ \mathbf{r} \end{bmatrix}, \quad (6)$$

where $\rho \leftarrow D_{\text{ze}}$ and $\mathbf{r} \leftarrow \mathbb{F}^r$ are freshly sampled randomness. We require ze to satisfy two conditions: (i) ze must be suitable as an OOD sample, and (ii) ze must guarantee that $\mathbf{y}$ leaks no information on $\mathbf{f}$. The first condition, as shown in [BCFW25], is equivalent to ze being a zero-evader (a widely studied notion): for every $\mathbf{v} \neq \mathbf{0}$ it holds that $\Pr_{\rho}[\text{ze}(\rho) \cdot \mathbf{v} = \mathbf{0}] \leq \varepsilon_{\text{zero}}$. For the second condition, we introduce the notion of a (r, ζ_{ze}) -**private zero-evader** which is a zero-evader where $(\mathbf{y}, \rho)$ in Equation 6 can be efficiently simulated (with statistical error at most ζ_{ze}). We show that zero-evaders satisfying the two conditions are plentiful (and in fact with perfect privacy error $\zeta_{\text{ze}} = 0$).

Lemma 2.13. *Let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times \ell}$ be a zero-evader with error $\varepsilon_{\text{zero}}$. For every $r \in \mathbb{N}$ there exists a zero-evader $\text{ze}': D'_{\text{ze}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times (\ell + r)}$ with error $\approx \varepsilon_{\text{zero}}$ that is $(r, 0)$ -private.*

One last challenge must be solved. The newly sampled randomness $\mathbf{r}$ must somehow be included in the code switching claim. Including $\mathbf{r}$ in the message encoded with $\mathcal{C}'$ involves the same padding complication that we encountered in the zero-knowledge code-switching. Instead, we again make use of the flexibility of the committed sumcheck relation, and perform a *joint OOD sample* of both g and s . To do so, we rely on the following lemma, which generalizes [BCFW25]. In particular, compatibly with zero-knowledge encodings, we allow *many codewords close to the original words* to satisfy the same constraint, but we enforce that the “message” parts must indeed be equal.

Lemma 2.14. *Let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times (\ell + \ell_{\text{zk}})}$ be a zero-evader with error $\varepsilon_{\text{zero}}$. Let $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^{\ell})^m$, $\mathcal{C}_{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{zk}}} \equiv (\mathbb{F}^{\ell_{\text{zk}}})^{m_{\text{zk}}}$ be $\mathbb{F}$ -additive codes with zero-knowledge encodings $\text{Enc}_{\mathcal{C}}: \mathbb{F}^{\ell} \times \mathbb{F}^r \rightarrow \mathbb{F}^{\ell \cdot m}$ and $\text{Enc}_{\mathcal{C}_{\text{zk}}}: \mathbb{F}_{\text{zk}}^{\ell} \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \mathbb{F}^{\ell' \cdot m_{\text{zk}}}$. For every $\mathbf{f}, \mathbf{s}$,*

$$\Pr_{\rho} \left[\begin{array}{l} \exists (\bar{\mathbf{f}}_1, \bar{\mathbf{f}}_2) \in \Lambda(\mathcal{C}, \mathbf{f}, \delta), (\bar{\mathbf{s}}_1, \bar{\mathbf{s}}_2) \in \Lambda(\mathcal{C}_{\text{zk}}, \mathbf{s}, \delta_{\text{zk}}) \\ (\mathbf{f}_1, \mathbf{s}_1) \neq (\mathbf{f}_2, \mathbf{s}_2) \wedge \text{ze}(\rho) \cdot (\mathbf{f}_1, \mathbf{s}_1)^{\top} = \text{ze}(\rho) \cdot (\mathbf{f}_2, \mathbf{s}_2)^{\top} \\ \text{where:} \\ (\mathbf{f}_1, \mathbf{r}_1) := \text{Enc}_{\mathcal{C}}^{-1}(\bar{\mathbf{f}}_1), (\mathbf{s}_1, \mathbf{r}'_1) := \text{Enc}_{\mathcal{C}_{\text{zk}}}^{-1}(\bar{\mathbf{s}}_1), \\ (\mathbf{f}_2, \mathbf{r}_2) := \text{Enc}_{\mathcal{C}}^{-1}(\bar{\mathbf{f}}_2), (\mathbf{s}_2, \mathbf{r}'_2) := \text{Enc}_{\mathcal{C}_{\text{zk}}}^{-1}(\bar{\mathbf{s}}_2) \end{array} \right] \leq |\Lambda(\mathcal{C}, \delta)|^2 \cdot |\Lambda(\mathcal{C}_{\text{zk}}, \delta_{\text{zk}})|^2 \cdot \varepsilon_{\text{zero}}$$

Proof sketch. Fix distinct $(\bar{\mathbf{f}}_1, \bar{\mathbf{s}}_1), (\bar{\mathbf{f}}_2, \bar{\mathbf{s}}_2) \in \Lambda(\mathcal{C}, \mathbf{f}, \delta) \times \Lambda(\mathcal{C}_{\text{zk}}, \mathbf{s}, \delta_{\text{zk}})$. Write $(\mathbf{f}_1, \mathbf{r}_1) := \text{Enc}_{\mathcal{C}}^{-1}(\bar{\mathbf{f}}_1)$, $(\mathbf{s}_1, \mathbf{r}'_1) := \text{Enc}_{\mathcal{C}_{\text{zk}}}^{-1}(\bar{\mathbf{s}}_1)$ and $(\mathbf{f}_2, \mathbf{r}_2) := \text{Enc}_{\mathcal{C}}^{-1}(\bar{\mathbf{f}}_2)$, $(\mathbf{s}_2, \mathbf{r}'_2) := \text{Enc}_{\mathcal{C}_{\text{zk}}}^{-1}(\bar{\mathbf{s}}_2)$. If $(\mathbf{f}_1, \mathbf{s}_1) = (\mathbf{f}_2, \mathbf{s}_2)$ we are

done. Else, since $\mathbf{ze}$ is a zero-evader, it must be that except with probability $\varepsilon_{\text{zero}}$,

$$\mathbf{ze}(\rho) \cdot \begin{bmatrix} \mathbf{f}_1 - \mathbf{f}_2 \\ \mathbf{s}_1 - \mathbf{s}_2 \end{bmatrix} \neq \mathbf{0}.$$

Taking a union bound over the $\binom{|\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}, \delta_{\text{zk}})|}{2}$ many such words concludes the proof. $\square$

In particular, supposing that $g = \text{Enc}_{\mathcal{C}'}(\mathbf{f}, \mathbf{r}')$ and $s = \text{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{r}, \mathbf{r}'')$ (where $\mathbf{r}', \mathbf{r}''$ are freshly sampled choices of randomness), Equation 6 can be rewritten as the claims that

$$\forall i \in [t_{\text{ood}}]: \mathbf{y}_i = \left(\mathbf{ze}(\rho) \cdot \begin{bmatrix} \mathbf{f} \\ \mathbf{r} \end{bmatrix} \right) [i] = \langle \mathbf{f}, \mathbf{ze}_i^{\#}(\rho) \rangle + \langle \mathbf{r}, \mathbf{ze}_i^{\$}(\rho) \rangle,$$

where $\mathbf{ze}_i^{\#}(\rho)$ (resp. $\mathbf{ze}_i^{\$}(\rho)$) denote the first ℓ (resp. last r) elements of the i -th row of $\mathbf{ze}(\rho)$. Again, these are claims compatible with our committed sumcheck relation.

Codeswitch construction. We incorporate the ideas so far into the following construction.

Construction 2.15. The verifier receives explicit input $\mathbf{x} = (\mu, \text{st}, \mathbf{u}_1, \dots, \mathbf{u}_n)$ and implicit input $\mathbf{y} = (f, \xi_1, \dots, \xi_n)$. The prover receives witness $\mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}$.

1. The prover sends oracles $g \in (\Sigma')^{m'}$ and $s \in \Sigma_{\text{zk}}^{m_{\text{zk}}}$. In the honest case, the prover samples $\mathbf{r}' \in \mathbb{F}^{r'}$, $\mathbf{s} \in \mathbb{F}^{\ell_{\text{zk}} - r}$, $\mathbf{r}'' \leftarrow \mathbb{F}^{r_{\text{zk}}}$, and sets $g := \text{Enc}_{\mathcal{C}'}(\mathbf{f}, \mathbf{r}')$ and $s := \text{Enc}_{\mathcal{C}_{\text{zk}}}((\mathbf{s}, \mathbf{r}), \mathbf{r}'')$.
2. The verifier samples and sends out-of-domain randomness $\rho \leftarrow D_{\text{ze}}$.
3. The prover replies with $\mathbf{y} \in \mathbb{F}^{t_{\text{ood}}}$. In the honest case the prover sets $\mathbf{y} := \mathbf{ze}(\rho) \cdot (\mathbf{f}, \mathbf{r}, \mathbf{s})^{\top}$.
4. The verifier samples and sends in-domain query randomness $x_1, \dots, x_t \leftarrow [m]$ and batching randomness $\varepsilon \leftarrow \mathbb{F}^{t_{\text{ood}} + t \cdot \iota}$.
5. The verifier queries $f(x_1), \dots, f(x_t) \in \Sigma \equiv \mathbb{F}^{\iota}$ and sets

$$\mu' := \mu + \sum_{i \in [t_{\text{ood}}]} \varepsilon_i \cdot \mathbf{y}_i + \sum_{i \in [t]} \sum_{l \in [\iota]} \varepsilon_{t_{\text{ood}} + i \cdot \iota + l} \cdot f(x_i)_l.$$

The verifier outputs a new instance for the target relation $\mathcal{R}_{\mathcal{C}', \mathcal{C}_{\text{zk}}, \text{sl}'}$ with the joint constraint

$$\begin{aligned} & \langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \mathbf{u}_i \rangle \\ & + \sum_{i \in [t_{\text{ood}}]} \varepsilon_i \cdot \langle (\mathbf{f}, \mathbf{s}, \mathbf{r}), \mathbf{ze}_i(\rho) \rangle \\ & + \sum_{i \in [t]} \sum_{l \in [\iota]} \varepsilon_{t_{\text{ood}} + i \cdot \iota + l} \cdot \langle (\mathbf{f}, \mathbf{r}), \mathbf{G}_{\mathcal{C}}[x_{i,l}] \rangle = \mu', \end{aligned} \tag{7}$$

where $\mathbf{G}_{\mathcal{C}}[x_{i,l}] \in \mathbb{F}^{\ell+r}$ denotes the row of $\text{Enc}_{\mathcal{C}}$'s generator matrix corresponding to the l -th symbol of the x_i -th position and $\mathbf{ze}_i$ denotes the restriction of $\mathbf{ze}$ to the i -th row. The output consists of:

- $\mathbf{x}' := (\mu', \text{st}', (\mathbf{u}_i)_{i \in [n]}, \mathbf{u}_{n+1})$ where $\text{st}' := (\text{st}, \varepsilon, \rho, x_1, \dots, x_t)$ denotes the state for the linear form associated with $\mathbf{f}$ and $\mathbf{u}_{n+1}$ denotes the vector associated with $(\mathbf{s}, \mathbf{r})$ (both the linear form and the vector are obtained by appropriately grouping terms in Equation 7),
- $\mathbf{y}' := (g, (\xi_i)_{i \in [n]}, s)$, and

- in the honest case $\mathbf{w}' := (\mathbf{f}, \mathbf{r}', (\xi_i, \mathbf{r}_i)_{i \in [n]}, ((\mathbf{r}, \mathbf{s}), \mathbf{r}''))$.

Analysis of Construction 2.15. We outline how we analyze the construction above.

Lemma 2.16. *Assume that $\mathbf{ze}$ is a $(\ell_{\mathbf{zk}} - r, \zeta_{\mathbf{ze}})$ -private zero-evader, $\text{Enc}_{\mathcal{C}'}$ is a t' -zero-knowledge encoding for $\mathcal{C}'$ with error $\zeta_{\mathcal{C}'}$, and that $\text{Enc}_{\mathcal{C}_{\mathbf{zk}}}$ is $t_{\mathbf{zk}}$ -zero-knowledge encoding for $\mathcal{C}_{\mathbf{zk}}$. Then, Construction 2.15 is HVZK with error $\zeta_{\mathcal{C}'} + \zeta_{\mathcal{C}_{\mathbf{zk}}} + \zeta_{\mathbf{ze}}$ with respect to the family of distinguishers $\mathfrak{D}$ making t' queries to the first oracle of $\mathbf{y}'$ and $t_{\mathbf{zk}}$ queries to the other oracles in $\mathbf{y}'$. The simulator performs t queries to the first oracle of $\mathbf{y}$ and $t_{\mathbf{zk}}$ queries to the remaining oracles.*

Proof sketch. Since $\mathbf{ze}$ is a $(\ell_{\mathbf{zk}} - r, \zeta_{\mathbf{ze}})$ -private zero-evader, there exists an efficient algorithm that simulates the distribution of $(\mathbf{y}, \rho)$. Since $\text{Enc}_{\mathcal{C}'}$ and $\text{Enc}_{\mathcal{C}_{\mathbf{zk}}}$ are zero-knowledge encodings, there are also efficient simulators for the distribution of queries to their encodings. The simulator for Construction 2.15 samples $x_1, \dots, x_t$ uniformly at random, queries f at the corresponding positions, and then runs the three simulators described above (answering queries to $\xi_1, \dots, \xi_n$ by querying the corresponding oracle). $\square$

Lemma 2.17 (informal). *For every $\delta, \delta', \delta_{\mathbf{zk}} \in (0, 1)$, Construction 2.15 has round-by-round knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\mathbf{zk}}, \mathbf{sl}}^{\delta, \delta_{\mathbf{zk}}}, \tilde{\mathcal{R}}_{\mathcal{C}', \mathcal{C}_{\mathbf{zk}}, \mathbf{sl}'}^{\delta', \delta_{\mathbf{zk}}})$ with errors*

$$\left(|\Lambda(\mathcal{C}', \delta')|^2 \cdot |\Lambda(\mathcal{C}_{\mathbf{zk}}, \delta_{\mathbf{zk}})|^2 \cdot \varepsilon_{\text{zero}}, \frac{|\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\mathbf{zk}}^{\equiv n+1}, \delta_{\mathbf{zk}})|}{|\mathbb{F}|} + (1 - \delta)^t \right).$$

Proof sketch. We only sketch soundness; the round-by-round knowledge soundness proof is given in Lemma 9.9.

Fix an arbitrary $(\bar{g}, \bar{s}) \in \Lambda(\mathcal{C}', g, \delta') \times \Lambda(\mathcal{C}_{\mathbf{zk}}, s, \delta_{\mathbf{zk}})$ and let $(\mathbf{f}, \mathbf{r}) := \text{Enc}_{\mathcal{C}'}^{-1}(\bar{g})$, $(\mathbf{s}, \mathbf{r}') := \text{Enc}_{\mathcal{C}_{\mathbf{zk}}}^{-1}(\bar{s})$. By Lemma 2.14 there is at most a single pair $(\mathbf{f}, \mathbf{s})$ such that $\mathbf{ze}(\rho) \cdot (\mathbf{f}, \mathbf{s})^\top = \mathbf{y}$. Write $\mathbf{s} = (\mathbf{r}^*, \mathbf{s}')$ and $\bar{f} = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r}^*)$. Note that $\Delta(f, \bar{f}) > \delta$, or else $\mathbf{f}, \mathbf{r}^*$ constitute (together with a valid witness for the other masks) a valid witness for the relation, a contradiction to soundness. In this case, except with probability $(1 - \delta)^t$, one of the consistency claims is not satisfied. Thus, for arbitrary $(\bar{g}, \bar{s}) \in \Lambda(\mathcal{C}', g, \delta') \times \Lambda(\mathcal{C}_{\mathbf{zk}}, s, \delta_{\mathbf{zk}})$ it must either be that: (i) the OOD condition is not satisfied; (ii) the associated joint constraint is not satisfied; or (iii) one of the consistency claims is not satisfied. Taking a final union bound over the batch yields the final bound. $\square$

Efficiency of Construction 2.15. The prover time in Construction 2.15 is dominated by the time to compute the encoding $\text{Enc}_{\mathcal{C}'}$. The verifier time is dominated by the time to compute μ' . Both of these quantities are *independent* of the starting code $\mathcal{C}$, which is counterintuitive. The resolution is that $\mathcal{C}$ affects future protocols used to test claims for $\tilde{\mathcal{R}}_{\mathcal{C}', \mathcal{C}_{\mathbf{zk}}, \mathbf{sl}'}^{\delta, \delta_{\mathbf{zk}}}$, since $\mathbf{sl}'$ *does* depend $\mathcal{C}$. This added complexity of $\mathbf{sl}'$ was informally reflected in the quantity $T_{\mathcal{C}}$ introduced in Theorem 2. More formally, in protocols for $\tilde{\mathcal{R}}_{\mathcal{C}', \mathcal{C}_{\mathbf{zk}}, \mathbf{sl}'}^{\delta, \delta_{\mathbf{zk}}}$ (such as Constructions 2.2 and 2.10) the prover time depends on $t_{\mathbf{sl}'}$ and the verifier time depends on the cost of evaluating “folds” of the vectors specified in $\mathbf{sl}'$.

- The prover time will depend on the time required to compute $t \cdot \iota$ rows of $\mathbf{G}_{\mathcal{C}}^\#$ and $t \cdot \iota$ rows of $\mathbf{G}_{\mathcal{C}}^s$. We refer to this quantity as $t_{\mathcal{C}}^p$.
- The verifier time will depend on the time required to evaluate $t \cdot \iota$ folded rows of $\mathbf{G}_{\mathcal{C}}^\#$ according to Definition 2.8), and $t \cdot \iota$ (non-folded) rows of $\mathbf{G}_{\mathcal{C}}^s$. We refer to this quantity as $t_{\mathcal{C}}^v$.

2.7 From building blocks to our main results

Theorem 1 follows by composing the zero-knowledge sumcheck IOR from Section 2.5 with the non-succinct IOPP from Section 2.2. Theorem 2 further uses the code-switching protocol from Section 2.6. This sequential composition satisfies HVZK because we establish the composable notion of HVZK for IORs described in Section 2.3: in each IOR we explicitly address a “future distinguisher” that is going to query that IOR’s output implicit instance, capturing as a special case the subsequent IOR invoked on that implicit instance. Making these dependencies explicit and instantiating the distinguishers based on the “next” IORs allows us to safely compose the sub-protocols while maintaining honest-verifier zero-knowledge throughout.

On the code for the small masks. The relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \text{sl}}$ (Equation 3) is defined with respect to the two zero-knowledge codes $\mathcal{C}$ and $\mathcal{C}_{zk}$. Extending the non-succinct “base case” protocol from Section 2.2 from the initial relation $\bar{\mathcal{R}}_{\mathcal{C}}$ (Equation 2) to the relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \text{sl}}$ requires additional care, incurring an additional term of $(1 - \delta_{zk})^{t_{zk}}$ to the soundness error, where δ_{zk} is $\mathcal{C}_{zk}$ ’s distance and t_{zk} is the number of queries to each mask oracle. In our constructions $\mathcal{C}$ and $\mathcal{C}_{zk}$ can be any $\mathbb{F}$ -additive code (with a zero-knowledge simulator supporting the appropriate number of queries).

To state Theorem 1 we *fix* $\mathcal{C}_{zk}$ to a Reed–Solomon code with message length $O(\lambda / \log \log \lambda)$ and rate $1 / \log \lambda$. Then each mask has size $O(\lambda \cdot \log \lambda / \log \log \lambda) = \tilde{O}(\lambda)$ and the distance $\delta_{zk} \approx 1 - 1 / \log \lambda$ is such that the number of queries t_{zk} required for $(1 - \delta_{zk})^{t_{zk}} \approx 2^{-\lambda}$ is $t_{zk} = O(\lambda / \log \log \lambda) = o(\lambda)$.

To state Theorem 2 (as well as Theorem 4 below), instead $\mathcal{C}_{zk}$ must encode messages with size $O(\lambda)$. Again choosing rate $1 / \log \lambda$, each mask has size $O(\lambda \cdot \log \lambda) = \tilde{O}(\lambda)$, and the number of queries t_{zk} required is $t_{zk} = O(\lambda / \log \log \lambda) = o(\lambda)$.

Other choices of $\mathcal{C}_{zk}$ are possible; the above choices achieve $o(1)$ overhead in the query complexity of the protocol over the non-private protocols (and in fact over any such IOPP, which requires at least $O(\lambda)$ queries [BMMS25b]).

Theorem 1: sublinear HVZK IOPP for constrained interleaved codes. First we run the zero-knowledge sumcheck from Section 2.5 to reduce from $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{zk}, \text{sl}}$ to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \text{sl}'}$, reducing a committed sum over the interleaved code $\mathcal{C}^{\equiv 2^k}$ to a committed sum over the base code $\mathcal{C}$. Then, we use a *variant* of the non-succinct IOPP from Section 2.2 for the relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \text{sl}'}$. The sumcheck IOR reduces the tested object size from $2^k \cdot m$ alphabet symbols to m alphabet symbols, the size of the base code. The verifier’s runtime is dominated by the queries made by the non-succinct IOPP to the *folded* oracle, with each query mapping to a single query to the original oracle over the alphabet Σ^{2^k} .

One parameter regime of interest is the following. Assume the encoded message in $\mathcal{C}^{\equiv 2^k} \subseteq (\Sigma^{2^k})^m$ has size $\ell := 2^k \cdot \ell'$ for $\ell' := \sqrt{\ell}$ and that $k := \log \ell'$.¹ This implies that $\mathcal{C} \subseteq \Sigma^m$ encodes messages of size ℓ' . After running the sumcheck IOR, the queries made by the non-succinct IOPP to the tested codeword, i.e., the virtual oracle that the sumcheck IOR reduction outputs, amount to a query of the original oracle over alphabet Σ^{2^k} , whose size is $|\Sigma|^{2^k} = |\Sigma|^{\ell'}$. Since the problem size has been reduced by a square-root factor, the full-mask strategy in the non-succinct protocol over messages of sublinear size compared to the original oracle yields an $o(1)$ overhead, as desired.

Theorem 2: HVZK code-switching. We combine components developed in prior subsections to obtain a protocol for reducing proximity testing for $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{zk}, \text{sl}}$ to proximity testing for $\mathcal{R}_{\mathcal{C}', \mathcal{C}_{zk}, \text{sl}'}$. The construction first runs the zero-knowledge sumcheck from Section 2.5 to reduce from testing

¹A more concretely efficient version can be obtained by balancing the interleaving factor with the message size to minimize the overall argument size (see e.g. [AHIV17; GLSTW23; BFRW25]).

the interleaved code $\mathcal{C}^{\equiv 2^k}$ to testing the base code $\mathcal{C}$. Then, we apply the code-switching from Section 2.6 to reduce to the relation $\mathcal{R}_{\mathcal{C}', \mathcal{C}_{2^k}, \text{sl}'}$. As before, the queries by the verifier in the code-switching sub-protocol are made to the “virtual” oracle in the output of the sumcheck IOR, and are simulated by queries to the initial interleaved codeword (this dominates the verifier’s runtime).

HVZK polylogarithmic-verifier protocol. To achieve much improved verifier complexity we fix a folding parameter $k \in \mathbb{N}$ and a sequence of codes $\mathcal{C}_1, \dots, \mathcal{C}_n$, selected so that the 2^k interleaving of $\mathcal{C}_{i+1}$ has the same message size as (the non-interleaved) $\mathcal{C}_i$. The input relation is $\mathcal{R}_{\mathcal{C}_1^{\equiv 2^k}, \mathcal{C}_{2^k}, \text{sl}'}$, which we apply Construction 2.10 and Construction 2.15 (i.e. Theorem 2) to reduce to an instance of $\mathcal{R}_{\mathcal{C}_2^{\equiv 2^k}, \mathcal{C}_{2^k}, \text{sl}'}$. The size of the problem has been reduced by a factor of 2^k , and we iterate this process multiple times, concluding with the same variant of Section 2.2 used in the proof of Theorem 1. Similar to [ACFY24; ACFY25; NA25; BFRW25], the theorem is then instantiated with codes whose distances increase progressively across the reduction, which yields improved query complexity (an example instantiation yielding Corollary 3 follows). We give a generic formulation of the theorem, for any sequence of codes $\mathcal{C}_1, \dots, \mathcal{C}_n$ satisfying the conditions previously specified.

Theorem 4 (informal, polylogarithmic HVZK). *Let $\mathcal{C}_1 \subseteq \Sigma^{m_{\mathcal{C}_1}}, \dots, \mathcal{C}_n \subseteq \Sigma^{m_{\mathcal{C}_n}}$ be $\mathbb{F}$ -additive codes, and fix a folding parameter $k \in \mathbb{N}$ such that for each $i \in [n]$, $\mathcal{C}_i$ has a $t_{\mathcal{C}_i}$ -query zero-knowledge encoding $\text{Enc}_{\mathcal{C}_i} : \mathbb{F}^{\ell_{\mathcal{C}_i}} \times \mathbb{F}^{r_{\mathcal{C}_i}} \rightarrow \Sigma^{m_{\mathcal{C}_i}}$ with error $\zeta_{\mathcal{C}_i}$ and encoding time $\tau_{\mathcal{C}_i}$. For each $i \in [n-1]$, let $\ell_{\mathcal{C}_i} := 2^k \cdot \ell_{\mathcal{C}_{i+1}}$. For a security parameter $\lambda \in \mathbb{N}$, proximity parameters $\delta_1, \dots, \delta_n \in (0, 1)$, letting $t_i = O(-\frac{\lambda}{\log(1-\delta_i)})$, we construct a $O(k \cdot n)$ -round IOPP for $\mathcal{R}_{\mathcal{C}_1^{\equiv 2^k}, T}^{\text{lin}}$, with the following features.*

- The prover sends $O\left(\sum_{i \in [2, n]} m_{\mathcal{C}_i}\right) + \tilde{O}(n \cdot k \cdot \lambda)$ alphabet symbols, and the verifier makes $\sum_{i \in [n]} t_i$ queries over alphabet Σ^{2^k} .
- The prover time is $O\left(T + 2^k \cdot \ell_{\mathcal{C}_1} + \sum_{i \in [2, n]} \tau_{\mathcal{C}_i} + \sum_{i \in [2, n]} t_{\mathcal{C}_{i-1}}^{\text{P}}\right) + \tilde{O}(n \cdot k \cdot \lambda)$ and the verifier time is $O\left(T^{(k \cdot n)} + m_{\mathcal{C}_n} + \sum_{i \in [n]} \left(2^k \cdot t_i + t_{\mathcal{C}_i}^{\text{V}}\right)\right) + \tilde{O}(n \cdot k \cdot \lambda)$.
- The protocol has round-by-round knowledge soundness error at most $2^{-\lambda}$ assuming that, for every $i \in [n]$, $\mathbb{F}$ is large enough with respect to $\mathcal{C}_i$ ’s mutual correlated agreement radius and list-size at distance δ_i .
- If, for $i \in [2, n]$, $t_{\mathcal{C}_i} \geq t_i$, the protocol is honest-verifier zero-knowledge with error $\sum_{i \in [2, n]} 2^k \cdot \zeta_{\mathcal{C}_i}$.

To achieve the parameters of Corollary 3, we consider an instantiation of Theorem 4 with Reed–Solomon codes, resembling [ACFY25]. Fix an initial message size $\ell \in \mathbb{N}$ and an initial evaluation domain of size $m \in \mathbb{N}$, set $k := O(\log \log \ell)$ and $n := O(\log \ell / \log \log \ell)$.

- For $i \in [n]$, let $\mathcal{C}_i$ be a Reed–Solomon code with evaluation domain size $m_{\mathcal{C}_i} := 2^{-i} \cdot m$ and let $\text{Enc}_{\mathcal{C}_i} : \mathbb{F}^{\ell_{\mathcal{C}_i}} \times \mathbb{F}^{r_{\mathcal{C}_i}} \rightarrow \mathbb{F}^{m_{\mathcal{C}_i}}$ be the corresponding $r_{\mathcal{C}_i}$ -query zero-knowledge encoding.
- Enforce that $\ell = 2^k \cdot \ell_{\mathcal{C}_1}$ and, for every $i \in [n-1]$, enforce that $\ell_{\mathcal{C}_i} = 2^k \cdot \ell_{\mathcal{C}_{i+1}}$.

The rest of Corollary 3 follows by accounting, and noticing that if $\mathcal{C}$ is the Reed–Solomon code encoding messages of size ℓ with randomness size r into codewords of size m , $t_{\mathcal{C}}^{\text{P}} = O(\ell + r)$ and $t_{\mathcal{C}}^{\text{V}} = O(\log m + r)$. (Both are implicit in [ACFY25; NA25] and made explicit in [BFRW25].)

A zero-knowledge reduction from R1CS to proximity-testing. We additionally provide a $(1+o(1))$ -overhead reduction from R1CS to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{2^k}, \text{sl}}$ (in particular, $\mathcal{C}$ can be an interleaved code, for use with our testing protocols). Using our masking technique as in Section 2.5, we achieve sublinear zero-knowledge overhead in the encoded codeword, whereas [BCL22] obtains overhead $O(1)$ due to

full masking. We note that since, for simplicity, we do not employ a holographic/pre-processing step (as done in [BCL22; RW24]) the verifier in this protocol is non-succinct.

On a high level, our R1CS reduction is similar to the construction in [ACFY25] (generalized to work with interleaved codes). To add zero-knowledge, instead of padding the R1CS instance with randomness (as done in [BCL22]), we use a similar masking technique to mask the matrix-vector multiplications for each of the R1CS matrices. Then, the reduction uses sumcheck as its main building block, but now with two types of sublinear masks: the first are masks “protecting” the R1CS witness, and the second are the sumcheck masks as in Section 2.4. We can propagate the constraints for both mask types (which are syntactically the same) into an instance of the relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}$ (which our protocols can test in zero knowledge).

2.8 Bonus: distance-amplified codes

The HVZK protocols in this work are *code-agnostic*: they can be instantiated with any $\mathbb{F}$ -additive zero-knowledge code. The concrete efficiency of our protocols, like that of other recent code-agnostic IOPs/IORs based on interleaving, depends on the code’s parameters; for example, the *encoding time* impacts the IOP prover time, the *distance* impacts the query complexity of the IOP verifier, and the *alphabet size* impacts how many bits each query reads. In this section, we propose **distance amplification via dispersers** (i.e., the ABNNR construction [ABNNR92]) as a way to achieve concrete improvements in the prover time and argument size for the resulting succinct arguments. This provides further opportunities for optimizations of zero-knowledge succinct arguments based on our techniques.

The code-design tension. In typical IOPs, a larger code distance enables fewer verifier queries for a fixed soundness target, which reduces the number of Merkle commitment openings in the resulting succinct argument. At the same time, the argument prover must (i) *encode* to obtain a codeword and (ii) *commit* to the codeword via a Merkle commitment; so it is desirable for the code to have a fast encoding and a small alphabet. Ideally, one would like a *linear-time encodable* code that is also *nearly MDS* (approaching the Singleton bound) over a small alphabet. Unfortunately, known families with near-MDS distance and linear-time encoding [GI05] are not known to be competitive in the parameter regimes relevant for implementations, whereas Reed–Solomon codes are MDS (meet the Singleton bound) but they have quasilinear-time encoding (via FFT-style multipoint evaluation) and their alphabet grows with block length.

ABNNR distance–amplification. High-distance linear-time encodable codes rely on distance amplification, and here we discuss one such procedure from [ABNNR92]. A bipartite graph $G = (L \cup R, E)$ with $|L| = |R| = m$ is a (ϵ, γ) -*disperser* if every left set $S \subseteq L$ with $|S| \geq \epsilon$ has neighbors $N(S) := \{j \in R : \exists i \in S \text{ with } (i, j) \in E\}$ covering at least a $(1 - \gamma)$ -fraction of the right set R , and also (within this paper) G is right-regular of some degree d . Given a base code $\mathcal{C} \subseteq \mathbb{F}^m$ with distance $\delta(\mathcal{C}) \geq 1 - \epsilon$, the ABNNR amplification map $A_G: \mathbb{F}^m \rightarrow (\mathbb{F}^d)^m$ is defined by fixing, for each right vertex $j \in R$, an arbitrary ordering of its left neighbors $N(j) = \{i_1, \dots, i_d\}$ and setting $(A_G(c))_j := (c_{i_1}, \dots, c_{i_d}) \in \mathbb{F}^d$. This leads to a new code $G(\mathcal{C}) := \{A_G(c) : c \in \mathcal{C}\} \subseteq (\mathbb{F}^d)^m$ with distance $\delta(G(\mathcal{C})) \geq 1 - \gamma$. In the parameters achieved by probabilistic constructions, with all but negligible probability, a random right-regular G of degree $d = O(1/\gamma)$ is such a disperser; the transformation then increases any constant base code distance $1 - \epsilon$ to distance $1 - \gamma$ while keeping blocklength $|R| = m$ and incurring an alphabet blow-up from $\mathbb{F}$ to $\mathbb{F}^{O(1/\gamma)}$. Explicit constructions of dispersers with these parameters achieve only a larger degree $d = \text{poly}(1/\gamma) \cdot \text{poly}(\log m)$ [GUV09; TUZ07],

and reducing the gap between explicit and probabilistic constructions can further accentuate the efficiency benefits that we discuss below.

Efficiency benefits. We illustrate the efficiency benefits of distance amplification via the features of recent IOP-based succinct arguments [ACFY24; ACFY25; BCFW25], where the dominant error term in the soundness error is $\approx (1 - \delta)^t$ where δ is the distance of the code and t is the number of spot checks by the verifier. In such protocols, the prover encodes a message via the given code and then commits to the resulting codeword via a Merkle commitment, and then (in a later round) sends the requested t random locations of the codeword along with the corresponding Merkle openings. We consider two efficiency metrics in this setting: encoding time and argument size.

- **Improved encoding time via amplified Reed–Solomon.** We instantiate the distance amplification transformation with a Reed–Solomon code as the base code, yielding an *amplified Reed–Solomon code* (ARS code). Fix a constant $c > 1$ and encode a message of length ℓ as a Reed–Solomon codeword on an evaluation domain of size $m = \lceil c\ell \rceil$, achieving a relative distance $\approx 1 - \frac{1}{c}$; then amplify to a target distance $1 - \gamma$ using a disperser. (See Definition 12.5.)

The encoding procedure in an ARS code separates the *expensive* FFT-based evaluation step performed at size $\Theta(\ell)$ and the *fast* distance-boosting step performed by simply gathering neighbors; this yields an improved encoding time precisely in the regime where plain RS would require evaluation at $\Theta(\ell/\gamma)$ points. Overall, encoding for plain RS takes time $O(\frac{\ell}{\gamma} \cdot \log \frac{\ell}{\gamma})$ while encoding for ARS takes the (improved) time $O(\frac{\ell}{\gamma} + \ell \cdot \log \ell)$.

While improving encoding time, using ARS *increases* argument size. The number of spot checks by the verifier depends on the code’s distance alone and each is answered via the requested symbol plus the corresponding Merkle opening. Plain RS and ARS with the same distance yield the same number of spot checks but have different argument sizes: for plain RS the requested symbol is one field element and for ARS the requested symbol is $d = O(1/\gamma)$ field elements.

- **Reducing argument size (with an increase in prover time).** Increasing distance by amplification reduces the number of spot checks for a fixed soundness target while increasing alphabet size, as each opened leaf now contains d field symbols. This leads to a parameter regime in which one benefits from the reduction in the number of queries, while the cost of the increase in the alphabet size is not too big. Under reasonable parameters, which we give in Section 12, one can obtain an improvement of $\approx 30\%$ in argument size.

This comes at a cost: the prover has to perform an additional distance amplification, making $O(1/\gamma)$ copies of each symbol in the original codeword, increasing its runtime.

Compatibility with our protocols. While the amplified symbols have large alphabet $\mathbb{F}^d$, the resulting code remains $\mathbb{F}$ -additive (by viewing each alphabet symbol as a d -tuple over $\mathbb{F}$). Moreover, if a base code admits a t -query zero-knowledge encoding, then placing d base symbols per amplified symbol yields a corresponding encoding for the amplified code that is (t/d) -query zero-knowledge. Thus, amplified codes can be used in all of our main theorems.

3 Preliminaries

We define objects and state results that we use in this paper. We use the following notation.

- For two functions $f, g: [m] \rightarrow \Sigma$ and a set $S \subseteq [m]$, we write $f(S) = g(S)$ to mean $f(x) = g(x)$ for every $x \in S$. Conversely, $f(S) \neq g(S)$ means there exists $x \in S$ with $f(x) \neq g(x)$.
- For two functions $f, g: [m] \rightarrow \Sigma$, $\Delta(f, g)$ is the fractional Hamming distance between f and g (the fraction of points in which they disagree). For a set $S \subseteq \Sigma^{[m]}$, $\Delta(f, S) := \min_{h \in S} \Delta(f, h)$ (or if S is empty, $\Delta(f, S) := 1$).
- The equality polynomial eq for n variables is $\text{eq}(\mathbf{X}, \mathbf{Y}) := \prod_{i=1}^n (\mathbf{X}_i \cdot \mathbf{Y}_i + (1 - \mathbf{X}_i) \cdot (1 - \mathbf{Y}_i))$.
- $\mathbb{F}^{<d}[\mathbf{X}_1, \dots, \mathbf{X}_n]$ is the ring of polynomials over $\mathbb{F}$ with variables $\mathbf{X}_1, \dots, \mathbf{X}_n$ of total degree $< d$.

Claim 3.1. For every $\hat{f} \in \mathbb{F}^{<2}[\mathbf{X}_1, \dots, \mathbf{X}_n]$ and $\boldsymbol{\tau} \in \mathbb{F}^n$, $\hat{f}(\boldsymbol{\tau}) = \sum_{\mathbf{b} \in \{0,1\}^n} \hat{f}(\mathbf{b}) \cdot \text{eq}(\boldsymbol{\tau}, \mathbf{b})$.

Lemma 3.2. For every non-zero polynomial $\hat{p} \in \mathbb{F}^{<d}[\mathbf{X}_1, \dots, \mathbf{X}_n]$, $\Pr_{\boldsymbol{\tau} \leftarrow \mathbb{F}^n} [\hat{p}(\boldsymbol{\tau}) = 0] \leq \frac{n \cdot (d-1)}{|\mathbb{F}|}$.

Definition 3.3. A **zero-evader** over $\mathbb{F}$ with error $\varepsilon_{\text{zero}}$ is a function $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t \times \ell}$ such that

$$\forall \mathbf{v} \in \mathbb{F}^\ell \setminus \{0^\ell\}, \quad \Pr_{\rho \leftarrow D_{\text{ze}}} [\text{ze}(\rho) \cdot \mathbf{v} = 0^t] \leq \varepsilon_{\text{zero}}.$$

Relations. A relation $\mathcal{R}$ consists of triples $(\mathbf{x}, \mathbf{y}, \mathbf{w})$ where $\mathbf{x}$ is the explicit instance, $\mathbf{y}$ the implicit instance, and $\mathbf{w}$ the witness. The corresponding language is $\mathcal{L}(\mathcal{R}) := \{(\mathbf{x}, \mathbf{y}) : \exists \mathbf{w}, (\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}\}$.

3.1 Interactive oracle reductions

Interactive Oracle Reductions (IORs) [BCGGRS19; BMNW25] are a generalization of Interactive Oracle Proofs (IOPs) [BCS16; RRR16]. Below we describe the flavor of IOR relevant to this paper.

A public-coin IOR of proximity from relation $\mathcal{R}$ to relation $\mathcal{R}'$ is a tuple $\text{IOR} = (\mathbf{P}, \mathbf{V})$ that works as follows. The prover $\mathbf{P}$ is an interactive algorithm that receives as input $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$, and the verifier $\mathbf{V}$ is an interactive oracle algorithm that receives input $\mathbf{x}$ and query access to $\mathbf{y}$ (consisting of l_0 symbols over some alphabet Σ_0). They interact over k rounds: in each round $i \in [k]$, $\mathbf{P}$ sends a proof string Π_i consisting of l_i symbols over some alphabet Σ_i , and $\mathbf{V}$ sends a uniformly random message $\rho_i \in \{0,1\}^{r_i}$. After k rounds of interaction, $\mathbf{V}$ makes queries to $\mathbf{y}, \Pi_1, \dots, \Pi_k$ (each query to a symbol). After that: (i) $\mathbf{V}$ either rejects or outputs a new explicit instance $\mathbf{x}'$ and implicit instance $\mathbf{y}'$ (which is implicitly specified in terms of the oracles $\mathbf{y}$ and $\Pi_1, \dots, \Pi_k$), and (ii) $\mathbf{P}$ outputs a new witness $\mathbf{w}'$ such that $(\mathbf{x}', \mathbf{y}', \mathbf{w}') \in \mathcal{R}'$. We write $\langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})$ for the interaction of $\mathbf{P}, \mathbf{V}$ above.

Definition 3.4. $\text{IOR} = (\mathbf{P}, \mathbf{V})$ has **(perfect) completeness** if, for every $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$,

$$\Pr \left[(\mathbf{x}', \mathbf{y}', \mathbf{w}') \in \mathcal{R}' \mid (\mathbf{x}', \mathbf{y}', \mathbf{w}') \leftarrow \langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w}) \right] = 1.$$

Efficiency measures. Efficiency measures are implicitly functions of $\mathbf{V}$'s inputs.

- *Rounds:* there are k rounds of interaction (a round is a prover message and verifier message).
- *Prover communication:* for each $i \in [k]$, Π_i consists of l_i symbols over alphabet Σ_i ; in this paper the alphabets $\Sigma_1, \dots, \Sigma_k$ are $\mathbb{F}$ -additive spaces, so we simply count the total number of elements of $\mathbb{F}$ in $\Pi_1, \dots, \Pi_k$, and we refer to this quantity as *prover communication*.

- *Input queries:* q_y is the number of alphabet symbols read by the verifier from y .
- *Proof queries:* q_Π is the number of alphabet symbols read by the verifier from $\Pi_1, \dots, \Pi_k$.
- *Input query blowup:* $m_{y' \rightarrow y}$ is the number of queries that a query to y' induces to y .
- *Proof query blowup:* $m_{y' \rightarrow \Pi}$ is the number of queries that a query to y' induces to $\Pi_1, \dots, \Pi_k$.
- *Randomness:* $r := \sum_{i \in [k]} r_i$ is the total number of random bits sent by the verifier.
- *Prover time:* pt is $\mathbf{P}'$ time measured in algebraic field operations.
- *Verifier time:* vt is $\mathbf{V}$'s time measured in algebraic field operations.

Remark 3.5. An IOP for a relation $\mathcal{R}$ is an IOR from the relation $\mathcal{R}$ to the trivial relation $\mathcal{R}_{\text{triv}} := \{(x = \perp, y = \perp, w = \perp)\}$.

The IORs that we consider in this paper satisfy a strong notion of knowledge soundness, namely round-by-round knowledge soundness [BCFW25], extended to handle relaxed relations.

Definition 3.6. Let $\text{IOR} = (\mathbf{P}, \mathbf{V})$ be an IOR from a relation $\mathcal{R}$ to $\mathcal{R}'$. A **knowledge state function with relaxation** $(\tilde{\mathcal{R}}, \tilde{\mathcal{R}}')$ for IOR is a (possibly inefficient) function KState that, on input a statement (x, y) , interaction transcript tr , and **knowledge state witness** w , outputs a bit and has the following properties.

- *Empty transcript:* if $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}(x, y, \text{tr}, w) = 1$ if and only if $(x, y, w) \in \tilde{\mathcal{R}}$.
- *Prover moves:* if tr is a transcript where the prover is about to move and $\text{KState}(x, y, \text{tr}, w) = 0$, then $\text{KState}(x, y, \text{tr} \parallel \Pi, w) = 0$ for every prover message Π .
- *Full transcript:* if $\text{tr} = (\Pi_1, \rho_1, \dots, \Pi_k, \rho_k)$ is a full transcript, then $\text{KState}(x, y, \text{tr}, w) = 1$ if and only if $\mathbf{V}^{y, \Pi_1, \dots, \Pi_k}(x, \rho_1, \dots, \rho_k)$ outputs (x', y') such that $(x', y', w) \in \tilde{\mathcal{R}}'$.

Definition 3.7. $\text{IOR} = (\mathbf{P}, \mathbf{V})$ from a relation $\mathcal{R}$ to $\mathcal{R}'$ has **round-by-round knowledge soundness with relaxation** $(\tilde{\mathcal{R}}, \tilde{\mathcal{R}}')$, **errors** $(\varepsilon_1, \dots, \varepsilon_k)$, and **extraction times** $(\text{et}_1, \dots, \text{et}_k)$ if there exist a knowledge state function KState with relaxation $\tilde{\mathcal{R}}$ to $\tilde{\mathcal{R}}'$ for IOR and a deterministic extractor $\mathbf{E}_{\text{rbr}}$ with the following property: for every statement (x, y) , round index $i \in [k]$, and interaction transcript $\text{tr} = (\Pi_1, \rho_1, \dots, \Pi_{i-1}, \rho_{i-1}, \Pi_i)$ where the verifier is about to move, $\mathbf{E}_{\text{rbr}}$ runs in time at most et_i and

$$\Pr_{\rho_i} \left[\exists w : \begin{array}{l} \text{KState}(x, y, \text{tr}, \mathbf{E}_{\text{rbr}}(x, y, \text{tr} \parallel \rho_i, w)) = 0 \\ \wedge \text{KState}(x, y, \text{tr} \parallel \rho_i, w) = 1 \end{array} \right] \leq \varepsilon_i(x, y).$$

The **total extraction time** is $\sum_{i \in [k]} \text{et}_i$.

3.2 Coding theory

We provide definitions for (error-correcting) codes and associated notions.

Definition 3.8. Let $\mathbb{F}$ be a finite field. A **code** $\mathcal{C}$ over an alphabet Σ with message size ℓ and **block length** m is an injective map $\mathcal{C}: \mathbb{F}^\ell \rightarrow \Sigma^m$. The **minimum distance** of $\mathcal{C}$ is

$$\delta(\mathcal{C}) := \min_{\substack{f, g \in \mathcal{C} \\ f \neq g}} \Delta(f, g).$$

We say that $\mathcal{C}$ is **$\mathbb{F}$ -additive** if $\Sigma = \mathbb{F}^\iota$, where $\mathbb{F}$ is a finite field, and $\mathcal{C}$ is a linear subspace of $\mathbb{F}^{\iota \cdot m}$. When $\iota = 1$ we say that $\mathcal{C}$ is **$\mathbb{F}$ -linear** (or just linear when $\mathbb{F}$ is clear from context).

We sometimes highlight the alphabet structure $\Sigma = \mathbb{F}^\ell$ when we explicitly think of an alphabet symbol as a vector of field elements. We often identify a code with its image $\mathcal{C}(\mathbb{F}^\ell) \subseteq \Sigma^m$. We consider several algorithmic measures associated to a code $\mathcal{C}$:

- $\text{tenc}(\mathcal{C})$ is the number of field operations required to encode a message $x \in \mathbb{F}^\ell$ into its corresponding codeword $u := \mathcal{C}(x)$;
- $\text{tcor}(\mathcal{C})$ is the number of field operations required to *erasure correct* a codeword (Definition 3.9); every linear code has efficient erasure correction (Lemma 3.10, see [GRS12]).

Definition 3.9. A code $\mathcal{C} \subseteq \Sigma^m$ has **erasure correction with time** $\text{tcor}(\mathcal{C})$ if there exists a deterministic algorithm $\mathbf{E}_{\mathcal{C}}$ that on input $f \in \Sigma^m$ and $S \subseteq [m]$: (i) if $|S| \geq (1 - \delta(\mathcal{C})) \cdot m$ and there exists a codeword $u \in \mathcal{C}$ such that $f(i) = u(i)$ for all $i \in S$, then $\mathbf{E}_{\mathcal{C}}(f, S) = u$ (the codeword u is unique); and (ii) otherwise, $\mathbf{E}_{\mathcal{C}}(f, S) = \perp$. Moreover, $\mathbf{E}_{\mathcal{C}}$ performs at most $\text{tcor}(\mathcal{C})$ field operations.

Lemma 3.10. Every additive code $\mathcal{C} \subseteq \Sigma^m$ has erasure correction with time $\text{tcor}(\mathcal{C}) = O((\ell \cdot m)^3)$

Interleaving. An interleaved code is obtained by “stacking” a code’s codewords.

Definition 3.11. Let $\mathcal{C} \subseteq \Sigma^m$ be a code and let $K \in \mathbb{N}$. The **K -interleaved code** $\mathcal{C}^{\equiv K}$ is the code over alphabet Σ^K with block length m defined as

$$\mathcal{C}^{\equiv K} := \{(u_1, \dots, u_K) \mid \forall i \in [K] : u_i \in \mathcal{C}\} \subseteq (\Sigma^K)^m.$$

A codeword $u \in \mathcal{C}^{\equiv K}$ consists of $K \cdot m$ symbols in Σ viewed as m symbols in Σ^K .

If $\mathcal{C}$ is $\mathbb{F}$ -additive then $\mathcal{C}^{\equiv K}$ is $\mathbb{F}$ -additive.

Lists. We recall the combinatorial list-decoding of a code.

Definition 3.12. Let $\mathcal{C} \subseteq \Sigma^m$ be a code. The **list around** $f \in \Sigma^m$ at distance $\delta \in [0, 1]$ is

$$\Lambda(\mathcal{C}, f, \delta) := \{g \in \mathcal{C} \mid \Delta(f, g) \leq \delta\}.$$

Moreover we define $|\Lambda(\mathcal{C}, \delta)| := \max_{f \in \Sigma^m} |\Lambda(\mathcal{C}, f, \delta)|$.

For every $K \in \mathbb{N}$ and $\delta \in (0, 1)$ it holds that $|\Lambda(\mathcal{C}, \delta)| \leq |\Lambda(\mathcal{C}^{\equiv K}, \delta)| \leq |\Lambda(\mathcal{C}, \delta)|^K$. In fact the list size of an interleaved code at some distance is larger than the list size of the base code by a factor *independent of the interleaving factor K* (but which depends on the distance δ considered).

Lemma 3.13 ([GGR11]). Let $\mathcal{C} \subseteq \Sigma^m$ be a code and let $\delta \in [0, \delta(\mathcal{C}))$. Let $b := \left\lceil \frac{\delta}{\delta(\mathcal{C}) - \delta} \right\rceil$ and $r := \left\lceil \log \frac{\delta(\mathcal{C})}{\delta(\mathcal{C}) - \delta} \right\rceil$. For every $K \geq 1$, $|\Lambda(\mathcal{C}^{\equiv K}, \delta)| \leq \binom{b+r}{r} \cdot |\Lambda(\mathcal{C}, \delta)|^r$.

MCA. We recall the notion of mutual correlated agreement [ACFY25]. When Σ is an $\mathbb{F}$ -additive alphabet, scalar multiplication and sums in the definition below are interpreted componentwise in the fixed $\mathbb{F}$ -vector-space structure of Σ .

Definition 3.14. Let $\mathcal{C} \subseteq \Sigma^m$ be an $\mathbb{F}$ -additive code. The **mutual correlated agreement (MCA) error** of a function $\text{PG}: D \rightarrow \mathbb{F}^n$ for $\mathcal{C}$ at distance $\delta \in [0, 1]$ is

$$\epsilon_{\text{mca}}(\mathcal{C}, \delta) := \max_{f_1, \dots, f_n \in \Sigma^m} \Pr_{\substack{\alpha \leftarrow D \\ \gamma := \text{PG}(\alpha)}} \left[\begin{array}{l} |S| \geq (1 - \delta) \cdot m \\ \exists u \in \mathcal{C} : \sum_i \gamma_i \cdot f_i(S) = u(S) \\ \exists i \in [n], \forall u \in \mathcal{C} : f_i(S) \neq u(S) \end{array} \right]$$

Upper bounds on $\epsilon_{\text{mca}}(\mathcal{C}, \delta)$ are known for every $\mathbb{F}$ -additive code (see e.g. [AHIV17; BCIKS20; ACFY25; Zei24; GKL24; BCGM25]). In this work we only consider functions $\text{PG}: \mathbb{F} \rightarrow \mathbb{F}^2$ where $\gamma \mapsto (1 - \gamma, \gamma)$ or $\gamma \mapsto (1, \gamma)$; these two have the same error, as can be seen by considering the identity $(1 - \gamma) \cdot f_1 + \gamma \cdot f_2 = f_1 + \gamma \cdot (f_2 - f_1)$.

Lemma 3.15. *Let $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\iota)^m$ be an $\mathbb{F}$ -additive code and let $\text{PG}: D \rightarrow \mathbb{F}^n$ have MCA error ϵ_{mca} . For every $K \in \mathbb{N}$, $\epsilon_{\text{mca}}(\mathcal{C}^{\equiv K}, \delta) \leq K \cdot \epsilon_{\text{mca}}(\mathcal{C}, \delta)$.*

Proof. For $f \in (\Sigma^K)^m$ and $j \in [K]$, we denote by $f[j] \in \Sigma^m$ the j -th row of f . We write

$$\begin{aligned} & \epsilon_{\text{mca}}(\mathcal{C}^{\equiv K}, \delta) \\ &= \max_{f_1, \dots, f_n \in (\Sigma^K)^m} \Pr_{\substack{\alpha \leftarrow D \\ \gamma := \text{PG}(n; \alpha)}} \left[\begin{array}{l} |S| \geq (1 - \delta) \cdot m \\ \exists S \subseteq [m] : \exists u \in \mathcal{C}^{\equiv K} : \sum_i \gamma_i \cdot f_i(S) = u(S) \\ \exists i \in [n], \forall u \in \mathcal{C}^{\equiv K} : f_i(S) \neq u(S) \end{array} \right] \\ &= \max_{f_1, \dots, f_n \in (\Sigma^K)^m} \Pr_{\substack{\alpha \leftarrow D \\ \gamma := \text{PG}(n; \alpha)}} \left[\begin{array}{l} |S| \geq (1 - \delta) \cdot m \\ \exists S \subseteq [m] : \exists (u_1, \dots, u_K) \in \mathcal{C}, \forall j \in [K] : \sum_i \gamma_i \cdot f_i[j](S) = u_j(S) \\ \exists i \in [n], \forall (u_1, \dots, u_K) \in \mathcal{C}, \exists j \in [K] : f_i[j](S) \neq u_j(S) \end{array} \right] \\ &\leq K \cdot \epsilon_{\text{mca}}(\mathcal{C}, \delta). \end{aligned}$$

where the last inequality follows from a union bound over the K events. $\square$

3.2.1 Zero-knowledge codes

Definition 3.16. *Let $\mathcal{C} \subseteq \Sigma^m$ be a code. For message length ℓ and randomness length r , a **t -query zero-knowledge encoding for $\mathcal{C}$ with error ζ_c** is a function $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$ such that there exists a polynomial-time probabilistic simulator $\text{Sim}_{\mathcal{C}}$ such that, for every $\text{msg} \in \mathbb{F}^\ell$ and $S \subseteq [m]$ with $|S| \leq t$, the following two random variables have statistical distance at most ζ_c :*

- The random variable $\text{Enc}_{\mathcal{C}}(\text{msg}, \mathbf{r})[S]$ consisting of $\text{Enc}_{\mathcal{C}}(\text{msg}, \mathbf{r})$ restricted to locations in S , where the randomness is over $\mathbf{r} \leftarrow \mathbb{F}^r$.
- The random variable $\text{Sim}_{\mathcal{C}}(S)$, where the randomness is over $\text{Sim}_{\mathcal{C}}$'s randomness.

Definition 3.17. *Let $\mathcal{C} \subseteq (\mathbb{F}^\iota)^m$ be an $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow (\mathbb{F}^\iota)^m$. A **generating matrix for $\text{Enc}_{\mathcal{C}}$** is $\mathbf{G}_{\mathcal{C}} = \begin{bmatrix} \mathbf{G}_{\mathcal{C}}^\# & \mathbf{G}_{\mathcal{C}}^\$ \end{bmatrix} \in \mathbb{F}^{(\iota \cdot m) \times \ell} \times \mathbb{F}^{(\iota \cdot m) \times r}$ such that*

$$\forall \mathbf{f} \in \mathbb{F}^\ell, \forall \mathbf{r} \in \mathbb{F}^r, \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r}) = \mathbf{G}_{\mathcal{C}} \cdot \begin{bmatrix} \mathbf{f} \\ \mathbf{r} \end{bmatrix} = \mathbf{G}_{\mathcal{C}}^\# \cdot \mathbf{f} + \mathbf{G}_{\mathcal{C}}^\$ \cdot \mathbf{r},$$

viewing a vector $\mathbb{F}^{\iota \cdot m}$ as m symbols of $(\mathbb{F}^\iota)$. For $i \in [\iota \cdot m]$, $\mathbf{G}_{\mathcal{C}}[i, \cdot]$ denotes the i -th row of $\mathbf{G}_{\mathcal{C}}$.

We recall the definition of the Reed–Solomon code and its zero-knowledge encodings, and a linear-time encodable code with a zero-knowledge encoding.

Definition 3.18. *Let $\mathbb{F}$ be a finite field, $\mathcal{L} \subseteq \mathbb{F}$, and $\ell \in \mathbb{N}$. We define*

$$\text{RS}[\mathbb{F}, \mathcal{L}, \ell] := \left\{ f: \mathcal{L} \rightarrow \mathbb{F} : \exists \hat{f} \in \mathbb{F}^{<\ell}[\mathbf{X}] \text{ s.t. } \forall x \in \mathcal{L} \ \hat{f}(x) = f(x) \right\}.$$

Proposition 3.19. *For every $t \in \mathbb{N}$, $\mathcal{C} := \text{RS}[\mathbb{F}, \mathcal{L}, \ell]$ has a t -query zero-knowledge encoding with message length $\ell - t$, randomness length t , and error $\zeta_{\mathcal{C}} = 0$.*

Theorem 3.20 ([BCL22, Theorem 6], informal). *For every $\varepsilon \in (0, 1)$, $\ell, t \in \mathbb{N}$, there exists a randomized linear time encoding algorithm that maps messages in $\mathbb{F}^\ell$ to a code $\mathcal{C} \subseteq \mathbb{F}^{O_\varepsilon(\ell+t)}$ such that this encoding is t -query zero-knowledge with error ε .*

3.2.2 Interleaving and folding for zero-knowledge encodings

We define interleaving and folding for zero-knowledge codes. These operations are designed to be compatible with the *multilinear polynomials* we use later in the paper.

Definition 3.21. *For a function $f: \mathbb{F}^{n+m} \rightarrow \mathbb{F}$ and $\mathbf{b} \in \mathbb{F}^n$ we write $f_{\mathbf{b}}: \mathbb{F}^m \rightarrow \mathbb{F}$ as $f_{\mathbf{b}}(x) := f(\mathbf{b}, x)$.*

Definition 3.22. *Let $\mathcal{C} \subseteq \Sigma^m$ be a code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$. For a folding parameter $k \in \mathbb{N}$, the 2^k -interleaved zero-knowledge encoding $\text{Enc}_{\mathcal{C}^{\equiv 2^k}}: \mathbb{F}^{2^k \ell} \times \mathbb{F}^{2^k r} \rightarrow (\Sigma^{2^k})^m$ is defined as follows. We split a message $\mathbf{f} \in \mathbb{F}^{2^k \ell}$ and randomness $\mathbf{r} \in \mathbb{F}^{2^k r}$ into 2^k pieces $(\mathbf{f}_{\mathbf{b}})_{\mathbf{b} \in \{0,1\}^k}$ and $(\mathbf{r}_{\mathbf{b}})_{\mathbf{b} \in \{0,1\}^k}$ by viewing both as multilinear polynomials $\hat{\mathbf{f}} \in \mathbb{F}^{<2}[\mathbf{X}_1, \dots, \mathbf{X}_{k+\log \ell}]$ and $\hat{\mathbf{r}} \in \mathbb{F}^{<2}[\mathbf{X}_1, \dots, \mathbf{X}_{k+\log r}]$ and writing*

$$\begin{aligned} \hat{\mathbf{f}}(\mathbf{X}_1, \dots, \mathbf{X}_{k+\log \ell}) &= \sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{b}, (\mathbf{X}_1, \dots, \mathbf{X}_k)) \cdot \mathbf{f}_{\mathbf{b}}(\mathbf{X}_{k+1}, \dots, \mathbf{X}_{k+\log \ell}), \\ \hat{\mathbf{r}}(\mathbf{X}_1, \dots, \mathbf{X}_{k+\log r}) &= \sum_{\mathbf{b} \in \{0,1\}^k} \text{eq}(\mathbf{b}, (\mathbf{X}_1, \dots, \mathbf{X}_k)) \cdot \mathbf{r}_{\mathbf{b}}(\mathbf{X}_{k+1}, \dots, \mathbf{X}_{k+\log r}). \end{aligned}$$

Then we define

$$\text{Enc}_{\mathcal{C}^{\equiv 2^k}}(\mathbf{f}, \mathbf{r}) := (\text{Enc}_{\mathcal{C}}(\mathbf{f}_{\mathbf{b}}, \mathbf{r}_{\mathbf{b}}))_{\mathbf{b} \in \{0,1\}^k} \in (\Sigma^{2^k})^m.$$

Claim 3.23. *Let $\mathcal{C} \subseteq \Sigma^m$ be a code and let $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$ be a t -query zero-knowledge encoding for $\mathcal{C}$. For every $k \in \mathbb{N}$, $\text{Enc}_{\mathcal{C}^{\equiv 2^k}}: \mathbb{F}^{2^k \ell} \times \mathbb{F}^{2^k r} \rightarrow (\Sigma^{2^k})^m$ from Definition 3.22 is a t -query zero-knowledge encoding for $\mathcal{C}^{\equiv 2^k} \subseteq (\Sigma^{2^k})^m$ with error $2^k \cdot \zeta$.*

Proof. Let $\text{Sim}_{\mathcal{C}}$ be the simulator for $\text{Enc}_{\mathcal{C}}$. We define a simulator $\text{Sim}_{\mathcal{C}^{\equiv 2^k}}$ for $\text{Enc}_{\mathcal{C}^{\equiv 2^k}}$. For a query set S , let $S_{\mathbf{b}}$ be the projection of queries in S of the block $\mathbf{b} \in \{0,1\}^k$. We define $\text{Sim}_{\mathcal{C}^{\equiv 2^k}}(S) := \{\text{Sim}_{\mathcal{C}}(S_{\mathbf{b}})\}_{\mathbf{b} \in \{0,1\}^k}$. Let S be any query set where $|S| \leq t$, which implies $|S_{\mathbf{b}}| \leq t$ for every $\mathbf{b}$. For every message $\mathbf{f}$ and randomness $\mathbf{r}$, $\text{Enc}_{\mathcal{C}^{\equiv 2^k}}(\mathbf{f}, \mathbf{r})[S] = \{\text{Enc}_{\mathcal{C}}(\mathbf{f}_{\mathbf{b}}, \mathbf{r}_{\mathbf{b}})[S_{\mathbf{b}}]\}_{\mathbf{b} \in \{0,1\}^k}$. For each $\mathbf{b} \in \{0,1\}^k$, by the zero-knowledge property of $\text{Enc}_{\mathcal{C}}$, $\delta(\text{Enc}_{\mathcal{C}}(\mathbf{f}_{\mathbf{b}}, \mathbf{r}_{\mathbf{b}})[S_{\mathbf{b}}], \text{Sim}_{\mathcal{C}}(S_{\mathbf{b}})) \leq \zeta$. By union bound over all 2^k blocks, $\Delta(\text{Enc}_{\mathcal{C}^{\equiv 2^k}}(\mathbf{f}, \mathbf{r})[S], \text{Sim}_{\mathcal{C}^{\equiv 2^k}}(S)) \leq \sum_{\mathbf{b} \in \{0,1\}^k} \zeta = 2^k \cdot \zeta$. $\square$

Definition 3.24. *Let Σ be $\mathbb{F}$ -additive, and let $f \in (\Sigma^{2^k})^m$. For $i \in [k]$, the interleaved fold of f at $\gamma \in \mathbb{F}^i$ is the function $\text{Fold}(f, \gamma) \in (\Sigma^{2^{k-i}})^m$ defined as follows. We identify $\{0,1\}^k \cong \{0,1\}^i \times \{0,1\}^{k-i}$, and for every $z \in [m]$ we write $f(z) = (f_{(\mathbf{b}, \mathbf{c})}(z))_{\mathbf{b} \in \{0,1\}^i, \mathbf{c} \in \{0,1\}^{k-i}}$ to mean that $f_{(\mathbf{b}, \mathbf{c})}(z) \in \Sigma$ is the $(\mathbf{b}, \mathbf{c})$ -th coordinate of $f(z) \in \Sigma^{2^k}$. Then we define*

$$\forall z \in [m], \text{Fold}(f, \gamma)(z) := \left(\sum_{\mathbf{b} \in \{0,1\}^i} \text{eq}(\mathbf{b}, \gamma) \cdot f_{(\mathbf{b}, \mathbf{c})}(z) \right)_{\mathbf{c} \in \{0,1\}^{k-i}}$$

where the product $\text{eq}(\mathbf{b}, \gamma) \cdot f_{(\mathbf{b}, \mathbf{c})}(z)$ and the sum are computed after identifying Σ with $\mathbb{F}^t$.

Remark 3.25. Let Σ , f , i be as in Definition 3.24. The interleaved fold of f is well-defined when $\gamma \in \mathbb{F}^i$ is replaced with indeterminates $(\mathbf{x}_1, \dots, \mathbf{x}_i)$. Indeed, for every $z \in [m]$,

$$\text{Fold}(f, (\mathbf{x}_1, \dots, \mathbf{x}_i))(z) := \left(\sum_{\mathbf{b} \in \{0,1\}^i} \text{eq}(\mathbf{b}, \mathbf{x}_1, \dots, \mathbf{x}_i) \cdot f_{(\mathbf{b}, \mathbf{c})}(z) \right)_{\mathbf{c} \in \{0,1\}^{k-i}} \in (\Sigma^{<2}[\mathbf{x}_1, \dots, \mathbf{x}_i])^{2^{k-i}}.$$

Lemma 3.26. Let $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\iota)^m$ be an $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$ and generating matrix $\mathbf{G}_{\mathcal{C}}$ (as in Definition 3.17). For every $\mathbf{f} \in \mathbb{F}^{2^k \ell}$, $\mathbf{r} \in \mathbb{F}^{2^k r}$, $j \in [k]$, and $\gamma \in \mathbb{F}^j$,

$$\text{Fold}(\text{Enc}_{\mathcal{C} \equiv 2^k}(\mathbf{f}, \mathbf{r}), \gamma) = \text{Enc}_{\mathcal{C} \equiv 2^{k-j}}(\text{Fold}(\mathbf{f}, \gamma), \text{Fold}(\mathbf{r}, \gamma)).$$

Proof. Identify $\{0,1\}^k \cong \{0,1\}^j \times \{0,1\}^{k-j}$. By the definition of $\text{Enc}_{\mathcal{C} \equiv 2^k}$ (Definition 3.22),

$$\text{Enc}_{\mathcal{C} \equiv 2^k}(\mathbf{f}, \mathbf{r}) = (\text{Enc}_{\mathcal{C}}(\mathbf{f}_{(\mathbf{b}, \mathbf{c})}, \mathbf{r}_{(\mathbf{b}, \mathbf{c})}))_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^j \times \{0,1\}^{k-j}}.$$

By the definition of Fold over Σ (Definition 3.24) and the above equality,

$$\text{Fold}(\text{Enc}_{\mathcal{C} \equiv 2^k}(\mathbf{f}, \mathbf{r}), \gamma) = \left(\sum_{\mathbf{b} \in \{0,1\}^j} \text{eq}(\mathbf{b}, \gamma) \cdot \text{Enc}_{\mathcal{C}}(\mathbf{f}_{(\mathbf{b}, \mathbf{c})}, \mathbf{r}_{(\mathbf{b}, \mathbf{c})}) \right)_{\mathbf{c} \in \{0,1\}^{k-j}}.$$

Using Definition 3.17 for each summand, for every $\mathbf{b} \in \{0,1\}^j$ and $\mathbf{c} \in \{0,1\}^{k-j}$,

$$\text{Enc}_{\mathcal{C}}(\mathbf{f}_{(\mathbf{b}, \mathbf{c})}, \mathbf{r}_{(\mathbf{b}, \mathbf{c})}) = \mathbf{G}_{\mathcal{C}} \cdot \begin{bmatrix} \mathbf{f}_{(\mathbf{b}, \mathbf{c})} \\ \mathbf{r}_{(\mathbf{b}, \mathbf{c})} \end{bmatrix} = \mathbf{G}_{\mathcal{C}}^{\#} \cdot \mathbf{f}_{(\mathbf{b}, \mathbf{c})} + \mathbf{G}_{\mathcal{C}}^{\$} \cdot \mathbf{r}_{(\mathbf{b}, \mathbf{c})}.$$

Therefore,

$$\begin{aligned} & \left(\sum_{\mathbf{b} \in \{0,1\}^j} \text{eq}(\mathbf{b}, \gamma) \cdot \text{Enc}_{\mathcal{C}}(\mathbf{f}_{(\mathbf{b}, \mathbf{c})}, \mathbf{r}_{(\mathbf{b}, \mathbf{c})}) \right)_{\mathbf{c} \in \{0,1\}^{k-j}} \\ &= \mathbf{G}_{\mathcal{C}}^{\#} \cdot \left(\sum_{\mathbf{b} \in \{0,1\}^j} \text{eq}(\mathbf{b}, \gamma) \cdot \mathbf{f}_{(\mathbf{b}, \mathbf{c})} \right)_{\mathbf{c} \in \{0,1\}^{k-j}} + \mathbf{G}_{\mathcal{C}}^{\$} \cdot \left(\sum_{\mathbf{b} \in \{0,1\}^j} \text{eq}(\mathbf{b}, \gamma) \cdot \mathbf{r}_{(\mathbf{b}, \mathbf{c})} \right)_{\mathbf{c} \in \{0,1\}^{k-j}} \\ & \quad \text{(by linearity)} \\ &= \mathbf{G}_{\mathcal{C}}^{\#} \cdot \text{Fold}(\mathbf{f}, \gamma) + \mathbf{G}_{\mathcal{C}}^{\$} \cdot \text{Fold}(\mathbf{r}, \gamma) \quad \text{(by definition of Fold over } \mathbb{F}) \\ &= \text{Enc}_{\mathcal{C} \equiv 2^{k-j}}(\text{Fold}(\mathbf{f}, \gamma), \text{Fold}(\mathbf{r}, \gamma)). \end{aligned}$$

□

4 Zero-knowledge for IORs and composition

Definition 4.1. Let $\text{IOR} = (\mathbf{P}, \mathbf{V})$ be an IOR from $\mathcal{R}$ to $\mathcal{R}'$ and let $\mathcal{D}$ be an algorithm. The $\mathcal{D}$ -extended view for IOR on input $(x, y, w) \in \mathcal{R}$ is the random variable

$$\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}, x, y, w) = ((\rho_1, \dots, \rho_k), (Q_y, Q_1, \dots, Q_k), (\mathbf{a}_y, \mathbf{a}_1, \dots, \mathbf{a}_k), \text{out})$$

obtained by running the experiment

$$\left[\begin{array}{l} (x', y', w') \leftarrow \langle \mathbf{P}, \mathbf{V} \rangle(x, y, w) \\ \text{out} \leftarrow \mathcal{D}^{y'}(x') \end{array} \right]$$

where out may be any output string of $\mathcal{D}$, and making the following definitions for every $i \in [k]$:

- $\rho_i \in \{0, 1\}^{r_i}$ is the verifier randomness in round i ;
- $Q_y \subseteq [l_y]$ and $\mathbf{a}_y \in \Sigma^{Q_y}$ are the queries of $\mathbf{V}$ and $\mathcal{D}$ to y along with their answers; and
- $Q_i \subseteq [l_i]$ and $\mathbf{a}_i \in \Sigma^{Q_i}$ are the queries of $\mathbf{V}$ and $\mathcal{D}$ to the proof oracle Π_i sent in round i along with their answers.

Definition 4.2. Let IOR be an IOR from $\mathcal{R}$ to $\mathcal{R}'$ and let $\mathcal{D}$ be a set of algorithms. IOR is **honest-verifier zero-knowledge for $\mathcal{D}$ with error ζ and query complexity b** if and only if there exists a polynomial-time probabilistic simulator $\mathbf{S}$ such that for every $\mathcal{D} \in \mathcal{D}$ and $(x, y, w) \in \mathcal{R}$ the two distributions $\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}, x, y, w)$ and $\mathbf{S}^{y, \mathcal{D}}(x)$ have statistical distance at most $\zeta(\mathcal{D})$, and $\mathbf{S}^{y, \mathcal{D}}(x)$ makes at most $b(\mathcal{D})$ queries to y . We let $\zeta := \max_{\mathcal{D} \in \mathcal{D}} \zeta(\mathcal{D})$ and $b := \max_{\mathcal{D} \in \mathcal{D}} b(\mathcal{D})$.

Remark 4.3. When the IOR is from $\mathcal{R}$ to the trivial relation $\mathcal{R}_{\text{triv}}$ (i.e., the IOR is an IOP) one can verify that Definition 4.2 coincides with honest-verifier zero-knowledge for the IOP.

Definition 4.4. Given algorithms $\mathbf{S}$ and $\mathcal{D}$, we let $\mathcal{D}_{\mathbf{S}}$ be the algorithm such that $\mathcal{D}_{\mathbf{S}}^y(x) := \mathbf{S}^{y, \mathcal{D}}(x)$. For a set of algorithms $\mathcal{D}$ we define the **S-wise distinguisher class** $\mathcal{D}_{\mathbf{S}} := \{\mathcal{D}_{\mathbf{S}} : \mathcal{D} \in \mathcal{D}\}$.

Theorem 4.5. Let IOR_1 be an IOR from $\mathcal{R}_{\text{start}}$ to $\mathcal{R}_{\text{int}}$ and let IOR_2 be an IOR from $\mathcal{R}_{\text{int}}$ to $\mathcal{R}_{\text{fin}}$ that have the following parameters (respectively).

- Round complexities k_1 and k_2 .
- Prover communication l_1 and l_2 .
- Input query complexities $q_{y,1}$ and $q_{y,2}$.
- Proof query complexities q_1 and q_2 .
- Prover times pt_1 and pt_2 .
- Verifier times vt_1 and vt_2 .
- IOR_1 has input query blow up $m_{y' \rightarrow y}$ and proof query blowup $m_{y' \rightarrow \Pi}$.

Construction 4.6 is an IOR IOR from $\mathcal{R}_{\text{start}}$ to $\mathcal{R}_{\text{fin}}$ with the following parameters.

- Round complexity $k_1 + k_2$.
- Prover communication $l_1 + l_2$.
- Input query complexity $q_{y,1} + m_{y' \rightarrow y} \cdot q_{y,2}$.
- Proof query complexity $q_1 + q_2 + m_{y' \rightarrow \Pi} \cdot q_{y,2}$.
- Prover time $\text{pt}_1 + \text{pt}_2$.
- Verifier time $\text{vt}_1 + \text{vt}_2$.

Moreover:

- **RBR security.** If IOR_1 has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\text{start}}, \tilde{\mathcal{R}}_{\text{int}})$ and errors $(\varepsilon_1^{(1)}, \dots, \varepsilon_{k_1}^{(1)})$ and IOR_2 has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\text{int}}, \tilde{\mathcal{R}}_{\text{fin}})$ and errors $(\varepsilon_1^{(2)}, \dots, \varepsilon_{k_2}^{(2)})$, then IOR has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\text{start}}, \tilde{\mathcal{R}}_{\text{fin}})$ and errors $(\varepsilon_1^{(1)}, \dots, \varepsilon_{k_1}^{(1)}, \varepsilon_1^{(2)}, \dots, \varepsilon_{k_2}^{(2)})$.
- **Zero-knowledge.** If IOR_2 is HVZK for $\mathfrak{D}$ with error ζ_2 via the simulator $\mathbf{S}_2$, and IOR_1 is HVZK for $\mathfrak{D}_{\mathbf{S}_2}$ with error ζ_1 and query complexity $\mathbf{b}$, then IOR is HVZK for $\mathfrak{D}$ with error $\zeta_1 + \zeta_2$ and query complexity $\mathbf{b}$.

Construction 4.6. The IOR consists of the concatenation of IOR_1 and IOR_2 .

- **Inputs.** The verifier receives as explicit input $\mathbf{x}_1$ and query access to $\mathbf{y}_1$. The honest prover receives as input $\mathbf{w}_1$ such that $(\mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1) \in \mathcal{R}_{\text{start}}$.
- **Protocol.**
 1. **First IOR.** The prover and verifier run IOR_1 for $(\mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1)$. After the interaction, the verifier outputs an explicit instance $\mathbf{x}_2$ and implicit instance $\mathbf{y}_2$ (defined via $\mathbf{y}_1$ and proof oracles sent in the execution of IOR_1). The honest prover outputs a witness $\mathbf{w}_2$ such that $(\mathbf{x}_2, \mathbf{y}_2, \mathbf{w}_2) \in \mathcal{R}_{\text{int}}$.
 2. **Second IOR.** The prover and verifier run IOR_2 for $(\mathbf{x}_2, \mathbf{y}_2, \mathbf{w}_2)$. After the interaction, the verifier outputs an explicit instance $\mathbf{x}_3$ and implicit instance $\mathbf{y}_3$. The honest prover outputs a witness $\mathbf{w}_3$ such that $(\mathbf{x}_3, \mathbf{y}_3, \mathbf{w}_3) \in \mathcal{R}_{\text{fin}}$.
- **Output.** The verifier outputs explicit instance $\mathbf{x}_3$ and implicit instance $\mathbf{y}_3$. The honest prover outputs a witness $\mathbf{w}_3$.

Proof. The efficiency parameters follow via inspection and the RBR knowledge soundness errors follow similarly to prior work. Here we focus on establishing HVZK of the composed IOR , demonstrating how to use Definition 4.2. Consider the following simulator.

$\mathbf{S}^{\mathbf{y}_1, \mathcal{D}_2}(\mathbf{x}_1)$:

1. Define the algorithm $\mathcal{D}_1^{\mathbf{y}_2}(\mathbf{x}_2) := \mathbf{S}_2^{\mathbf{y}_2, \mathcal{D}_2}(\mathbf{x}_2)$.
2. Run and output $\mathbf{S}_1^{\mathbf{y}_1, \mathcal{D}_1}(\mathbf{x}_1)$.

We argue that, for every $(\mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1) \in \mathcal{R}_{\text{start}}$ and $\mathcal{D}_2 \in \mathfrak{D}$, the following two distributions are $(\zeta_1 + \zeta_2)$ -close in statistical distance:

$$\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}_2, \mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1) \quad \text{and} \quad \mathbf{S}^{\mathbf{y}_1, \mathcal{D}_2}(\mathbf{x}_1),$$

Note that $\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}_2, \mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1)$ is equivalent to the following experiment.

$\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}_2, \mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1)$:

1. Compute $(\mathbf{x}_2, \mathbf{y}_2, \mathbf{w}_2) \xleftarrow{\text{view}_1} \langle \mathbf{P}_1, \mathbf{V}_1 \rangle(\mathbf{x}_1, \mathbf{y}_1, \mathbf{w}_1)$ where view_1 is $\mathbf{V}_1$'s view in the interaction.
2. Compute $(\mathbf{x}_3, \mathbf{y}_3, \mathbf{w}_3) \xleftarrow{\text{view}_2} \langle \mathbf{P}_2, \mathbf{V}_2 \rangle(\mathbf{x}_2, \mathbf{y}_2, \mathbf{w}_2)$ where view_2 is $\mathbf{V}_2$'s view in the interaction.
3. Compute $\text{out}_2 \leftarrow \mathcal{D}_2^{\mathbf{y}_3}(\mathbf{x}_3)$.
4. Let view be $\mathbf{V}$'s view obtained by combining view_1 and view_2 .
5. Output $(\text{view}, \text{out}_2)$.

We rely on a two-step hybrid argument. For that, we define a hybrid simulator $\tilde{\mathbf{S}}$.

$\tilde{\mathbf{S}}^{y_1, \mathcal{D}_2}(x_1, w_1)$:

1. Define the algorithm $\mathcal{D}_1^{y_2}(x_2) := \mathbf{S}_2^{y_2, \mathcal{D}_2}(x_2)$.
2. Compute and output $\text{View}(\mathbf{P}_1, \mathbf{V}_1, \mathcal{D}_1, x_1, y_1, w_1)$.

We consider the two steps of the hybrid argument.

- $\Delta(\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}_2, x_1, y_1, w_1), \tilde{\mathbf{S}}^{y_1, \mathcal{D}_2}(x_1, w_1)) \leq \zeta_2$. By IOR_2 's HVZK property, since $\mathcal{D}_2 \in \mathfrak{D}$, the statistical distance of $\text{View}(\mathbf{P}_2, \mathbf{V}_2, \mathcal{D}_2, x_2, y_2, w_2)$ and $\mathbf{S}_2^{y_2, \mathcal{D}_2}(x_2)$ is at most ζ_2 . Moreover, first running $\langle \mathbf{P}_1, \mathbf{V}_1 \rangle(x_1, y_1, w_1)$ and then $\mathbf{S}_2^{y_2, \mathcal{D}_2}(x_2)$ is equivalent to $\text{View}(\mathbf{P}_1, \mathbf{V}_1, \mathcal{D}_1, x_1, y_1, w_1)$, which is the experiment in $\tilde{\mathbf{S}}^{y_1, \mathcal{D}_2}(x_1, w_1)$.
- $\Delta(\tilde{\mathbf{S}}^{y_1, \mathcal{D}_2}(x_1, w_1), \mathbf{S}^{y_1, \mathcal{D}_2}(x_1)) \leq \zeta_1$. Since $\mathcal{D}_2 \in \mathfrak{D}$ we know that $\mathcal{D}_1 \in \mathfrak{D}_{\mathbf{S}_2}$ (by definition of $\mathcal{D}_1$). By IOR_1 's HVZK property, the statistical distance of $\text{View}(\mathbf{P}_1, \mathbf{V}_1, \mathcal{D}_1, x_1, y_1, w_1)$ and $\mathbf{S}_1^{y_1, \mathcal{D}_1}(x_1)$ is at most ζ_1 .

We conclude that

$$\begin{aligned} & \Delta(\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}_2, x_1, y_1, w_1), \mathbf{S}^{y_1, \mathcal{D}_2}(x_1)) \\ & \leq \Delta(\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}_2, x_1, y_1, w_1), \tilde{\mathbf{S}}^{y_1, \mathcal{D}_2}(x_1, w_1)) + \Delta(\tilde{\mathbf{S}}^{y_1, \mathcal{D}_2}(x_1, w_1), \mathbf{S}^{y_1, \mathcal{D}_2}(x_1)) \\ & \leq \zeta_1 + \zeta_2. \end{aligned}$$

The simulator is polynomial-time as a composition of polynomial-time algorithms, and its number of queries to y_1 is exactly that of $\mathbf{S}_1$, i.e., at most b . $\square$

We introduce the class of query-bounded distinguishers, which are distinguishers that make a bounded number of queries to the implicit instance (and are otherwise unbounded). We only consider distinguishers $\mathcal{D}$ that are *non-adaptive*, i.e., $\mathcal{D}$ is specified by a pair $(\mathcal{D}_0, \mathcal{D}_1)$ such that

$$\forall x, y : \mathcal{D}^y(x) = \left[\begin{array}{l} (\text{st}, S) \leftarrow \mathcal{D}_0(x) \\ \mathcal{D}_1(\text{st}, y[S]) \end{array} \right].$$

Definition 4.7. Let $t \in \mathbb{N}$. We define the class of **query-bounded distinguishers** $\mathfrak{D}^{\leq t}$ as the set of algorithms that perform at most t (non-adaptive) queries to their implicit input.

Multiple implicit instances. Many IORs that we consider in this work have as target relations where the implicit instance can be decomposed in a number of oracles. When considering honest-verifier zero-knowledge against query-bounded adversaries, we write $\mathfrak{D}^{\leq t_1} \times \dots \times \mathfrak{D}^{\leq t_n}$ for the class of query-bounded distinguishers that query the i -th oracle in the implicit instance at most t_i times. Alternatively, we let $\mathbf{t} = (t_1, \dots, t_n)$ and write $\mathfrak{D}^{\leq \mathbf{t}}$ for conciseness. For the notions of honest-verifier zero-knowledge that we consider this suffices and we abuse notation by simulating the view of each distinguisher individually.

5 Succinct linear forms

We define a notion of succinct linear forms. These capture the fact that some linear claims can be succinctly described and efficiently computed from such succinct descriptions.

Definition 5.1. A **succinct linear form** sl is a map that takes as input a state st and outputs a matrix $\text{sl}(\text{st}) \in \mathbb{F}^{t \times n}$; we use the notation $\text{sl} \in \langle \mathbb{F}^{t \times n} \rangle$ to indicate the dimensions of the matrix. We write t_{sl} for the maximum number (over the choice of st) of field operations for computing $\text{sl}(\text{st})$.

We define a few succinct linear form that we use.

Definition 5.2. The **identity linear form** is the linear form $\text{sl}_{\text{id}} \in \langle \mathbb{F}^{t \times n} \rangle$ that takes as input a state $\mathbf{V} \in \mathbb{F}^{t \times n}$ and outputs $\mathbf{V}$ (i.e., sl_{id} is the identity map). Note that $t_{\text{sl}_{\text{id}}} = 0$.

Definition 5.3. Let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t \times n}$ be a zero-evader. The **ze-linear form** is the linear form $\text{sl}_{\text{ze}} \in \langle \mathbb{F}^{t \times n} \rangle$ that takes as input a state $\rho \in D_{\text{ze}}$ and outputs $\text{ze}(\rho)$. Note that $t_{\text{sl}_{\text{ze}}} = \text{teval}(\text{ze})$.

Succinct linear forms can be combined and operated on.

Definition 5.4. The **scalar multiplied linear form** for $\text{sl} \in \langle \mathbb{F}^{t \times n} \rangle$ is the linear form $\times(\text{sl}) \in \langle \mathbb{F}^{t \times n} \rangle$ mapping $(\text{st}, \varepsilon \in \mathbb{F})$ to $\varepsilon \cdot \text{sl}(\text{st})$. Note that $t_{\times(\text{sl})} \leq t_{\text{sl}} + O(t \cdot n)$.

Definition 5.5. For $k \in \mathbb{N}$, the **k -zero-padded linear form** for $\text{sl} \in \langle \mathbb{F}^{t \times n} \rangle$ is the linear form $0^k(\text{sl}) \in \langle \mathbb{F}^{t \times (n+k)} \rangle$ mapping st to $[\text{sl}(\text{st}), 0^{t \times k}]$. Note that $t_{0^k(\text{sl})} \leq t_{\text{sl}}$.

Definition 5.6. Let $\text{sl}_1, \dots, \text{sl}_k \in \langle \mathbb{F}^{t \times n} \rangle$ and let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^k$ be a zero-evader. The **ze-batched linear form** $\text{sl}[\text{ze}, (\text{sl}_1, \dots, \text{sl}_k)] \in \langle \mathbb{F}^{t \times n} \rangle$ maps $(\text{st}_1, \dots, \text{st}_k, \rho)$ to $\sum_{i \in [k]} \text{ze}(\rho)_i \cdot \text{sl}_i(\text{st}_i)$. Note that $t_{\text{sl}[\text{ze}, (\text{sl}_1, \dots, \text{sl}_k)]} \leq \text{teval}(\text{ze}) + \sum_{i \in [k]} t_{\text{sl}_i} + O(k \cdot t \cdot n)$.

The most important operation that we apply on succinct linear form is that of “fixing variables”. This operation takes a succinct linear form (say of size 2^n) and a vector of field elements $\alpha \in \mathbb{F}^i$ (where $i \leq n$), and outputs a succinct linear form of size 2^{n-i} , in a way that is compatible with the folding operation in Definition 3.24.

Definition 5.7. Let $i \in \mathbb{N}$ and $\text{sl} \in \langle \mathbb{F}^{t \times 2^n} \rangle$. The **i -fixing linear form associated to sl** is the linear form $\odot_i \text{sl} \in \langle \mathbb{F}^{t \times 2^{n-i}} \rangle$ that maps $(\text{st}, \gamma \in \mathbb{F}^i)$ to $\mathbf{V}' \in \mathbb{F}^{t \times 2^{n-i}}$ defined as follows:

1. Let $\mathbf{V} = \text{sl}(\text{st}) \in \mathbb{F}^{t \times 2^n}$.
2. For each row $\mathbf{v}_j \in \mathbb{F}^{2^n}$ of $\mathbf{V}$, set $\mathbf{v}'_j := \text{Fold}(\mathbf{v}_j, \gamma) \in \mathbb{F}^{2^{n-i}}$ as in Definition 3.24 i.e.

$$\forall \mathbf{c} \in \{0, 1\}^{n-i}, \mathbf{v}'_j[\mathbf{c}] := \sum_{\mathbf{b} \in \{0, 1\}^i} \text{eq}(\mathbf{b}, \gamma) \cdot \mathbf{v}_j[\mathbf{b}, \mathbf{c}].$$

3. let $\mathbf{V}' \in \mathbb{F}^{t \times 2^{n-i}}$ to be the matrix whose rows are $\mathbf{v}'_j$.
- Note that $t_{\odot_i \text{sl}} \leq t_{\text{sl}} + O(t \cdot n)$.

The fixing operation is used to reduce the instance size, and we illustrate this via an example. Consider the linear form sl_{zeeq} associated to the zero evader ze_{eq} mapping $\alpha \mapsto (\text{eq}(\alpha, \mathbf{b}))_{\mathbf{b} \in \{0, 1\}^n}$. By [VSBW13], $t_{\text{sl}_{\text{zeeq}}} = O(2^n)$. Consider now $\odot_n \text{sl}_{\text{zeeq}}$, which (by definition) is the map that maps $(\alpha, \gamma) \mapsto \text{eq}(\alpha, \gamma)$. Note that $t_{\odot_n \text{sl}_{\text{zeeq}}} \leq n$, which is succinct in the original linear form size.

Following Remark 3.25, the fixing linear form is well-defined when part of the input state consists of indeterminates. For $\mathbf{sl} \in \langle \mathbb{F}^{2^n} \rangle$,

$$(\odot_i \mathbf{sl})(\mathbf{st}, (\mathbf{x}_1, \dots, \mathbf{x}_i)) = \left(\sum_{\mathbf{b} \in \{0,1\}^i} \text{eq}(\mathbf{b}, \mathbf{x}_1, \dots, \mathbf{x}_i) \cdot \mathbf{sl}(\mathbf{st})[\mathbf{b}, \mathbf{c}] \right)_{\mathbf{c} \in \{0,1\}^{n-i}} ; \quad (8)$$

and this extends naturally to $\mathbf{sl} \in \langle \mathbb{F}^{t \times 2^n} \rangle$.

We introduce the main relation that we use in this work.

Definition 5.8. *Let:*

- $n \in \mathbb{N}$;
- $\mathcal{C} \subseteq \Sigma^m$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$;
- $\mathcal{C}_{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{zk}}}$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{zk}}}: \mathbb{F}^{\ell_{\text{zk}}} \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{zk}}}$;
- $\mathbf{sl} = (\mathbf{sl}, (\mathbf{sl}_i)_{i \in [n]}, (\mathbf{sl}_{\circ, i})_{i \in [n]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i \in \langle \mathbb{F}^{\ell_{\text{zk}}} \rangle$, $\mathbf{sl}_{\circ, i} \in \langle \mathbb{F}^{t_i \times \ell_{\text{zk}}} \rangle$.

Define:

$$\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}} := \left\{ \left(\begin{array}{l} \mathbf{x} = ((\mu, \mathbf{st}, (\mathbf{st}_i)_{i \in [n]}), ((\mu_i, \mathbf{st}_{\circ, i})_{i \in [n]})), \\ \mathbf{y} = (f, (\xi_i)_{i \in [n]}), \\ \mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]}) \end{array} \right) \mid \left(\begin{array}{l} f = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r}) \\ \forall i \in [n]: \xi_i = \text{Enc}_{\mathcal{C}_{\text{zk}}}(\xi_i, \mathbf{r}_i) \\ \forall i \in [n]: \mathbf{sl}_{\circ, i}(\mathbf{st}_{\circ, i}) \cdot \xi_i = \mu_i \\ \langle \mathbf{f}, \mathbf{sl}(\mathbf{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \mathbf{sl}_i(\mathbf{st}_i) \rangle = \mu \end{array} \right) \right\}.$$

For every $\delta, \delta_{\text{zk}} \in (0, 1)$ we define a corresponding relaxed relation for proximity testing:

$$\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}} := \left\{ (\mathbf{x}, \mathbf{y}, (\mathbf{w}, \bar{\mathbf{y}})) \mid \Delta(\mathbf{y}, \bar{\mathbf{y}}) \leq (\delta, \delta_{\text{zk}}, \dots, \delta_{\text{zk}}) \wedge (\mathbf{x}, \bar{\mathbf{y}}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}} \right\}.$$

6 Zero-knowledge sumcheck IOR

We formally describe and analyze our zero-knowledge sumcheck IOR. First, we introduce some notation for the univariate evaluation zero-evader.

Definition 6.1. For $n \in \mathbb{N}$, $\mathbf{ze}_n^*: \mathbb{F} \rightarrow \mathbb{F}^n$ is the zero-evader mapping $\rho \mapsto (1, \rho, \dots, \rho^{n-1})$, which has error $\frac{n-1}{|\mathbb{F}|}$.

Theorem 6.2. Consider the following ingredients:

- $n \in \mathbb{N}$ be a number of functions;
- $k \in \mathbb{N}$ be a folding parameter;
- $\mathcal{C} \subseteq \Sigma^m$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$;
- $\mathcal{C}_{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{zk}}} \equiv (\mathbb{F}^{\ell_{\text{zk}}})^{m_{\text{zk}}}$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{zk}}}: \mathbb{F}^{\ell_{\text{zk}}} \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{zk}}}$;
- $\mathbf{sl} = (\mathbf{sl}, (\mathbf{sl}_i)_{i \in [n]}, (\mathbf{sl}_{o,i})_{i \in [n]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i \in \langle \mathbb{F}^{\ell_{\text{zk}}} \rangle$, $\mathbf{sl}_{o,i} \in \langle \mathbb{F}^{t_i \times \ell_{\text{zk}}} \rangle$.

Define

$$\mathbf{sl}' := (\times(\odot_k \mathbf{sl}), (\times(\mathbf{sl}_i))_{i \in [n]}, (\mathbf{sl}_{\mathbf{ze}_{\ell_{\text{zk}}}^*})_{j \in [k]}, (\mathbf{sl}_{o,i})_{i \in [n]}, (\mathbf{sl}_{\text{id}})_{j \in [k]}).$$

Construction 6.3 is a IOR from $\mathcal{R}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$ to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}'}$ with the following properties.

- Round complexity: $k + 1$.
- Prover communication (in field elements): $k \cdot (\ell_{\text{zk}} + \iota_{\text{zk}} \cdot m_{\text{zk}}) + 1$. Of these, $k \cdot \iota_{\text{zk}} \cdot m_{\text{zk}}$ are sent as a single oracle message over alphabet Σ_{zk}^k and $k \cdot \ell_{\text{zk}} + 1$ are sent as non-oracle messages.
- Verifier queries: none (a queryless IOR).
- Prover time (in field operations): $O(2^k \cdot \ell + t_{\text{sl}} + \sum_{i \in [n]} t_{\text{sl}_i} + k \cdot \text{tenc}(\mathcal{C}_{\text{zk}}))$.
- Verifier time (in field operations): $O(k \cdot \ell_{\text{zk}})$.
- **Round-by-round security:** For every $\delta, \delta_{\text{zk}} \in (0, 1)$, Construction 6.3 has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}}, \tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}'}^{\delta, \delta_{\text{zk}}})$ and errors

$$\left(\frac{|\Lambda(\mathcal{C} \equiv 2^k, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \left(\epsilon_{\text{mca}}(\mathcal{C} \equiv 2^{k-j}, \delta) + \frac{\ell_{\text{zk}} \cdot |\Lambda(\mathcal{C} \equiv 2^{k-(j-1)}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|} \right)_{j \in [k]} \right).$$

The total extraction time is $O(\sum_{j \in [k]} \text{tcor}(\mathcal{C} \equiv 2^{k-j}))$.

- **Zero-knowledge:** If $\text{char}(\mathbb{F}) \neq 2$, $\ell_{\text{zk}} \geq 2$, and $\text{Enc}_{\mathcal{C}_{\text{zk}}}$ is a t_{zk} -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_{\text{zk}}}$ then, for every $\mathbf{t} \in \mathbb{N}^{1+n}$, Construction 6.3 is HVZK for $\mathfrak{D}^{\leq (t, t_{\text{zk}}, \dots, t_{\text{zk}})}$ with error $k \cdot \zeta_{\mathcal{C}_{\text{zk}}}$ and query complexity $\mathbf{t}$.

Construction 6.3.

- **Inputs and notation.** The verifier receives explicit input $\mathbf{x} = ((\mu, \mathbf{st}, (\mathbf{st}_i)_{i \in [n]}), ((\mu_i)_{i \in [n]}, (\mathbf{st}_{o,i})_{i \in [n]}))$ and oracle access to implicit input $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$. In the honest case, the prover receives as input $\mathbf{x}$ and $\mathbf{y}$ as well as witness $\mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}'}$.
- **Interaction phase.**
 1. **Sending masks.** The prover sends $s_1, \dots, s_k \in \mathbb{F}^{m_{\text{zk}}}$. In the honest case the prover samples $s_1, \dots, s_k \in \mathbb{F}^{<\ell_{\text{zk}}}[\mathbf{X}]$, $\mathbf{r}'_1, \dots, \mathbf{r}'_k \leftarrow \mathbb{F}^{r_{\text{zk}}}$ and sets $s_j := \text{Enc}_{\mathcal{C}_{\text{zk}}}(s_j, \mathbf{r}'_j)$ for each $j \in [k]$.
 2. **New target.** The prover sends $\tilde{\mu} \in \mathbb{F}$. In the honest case the prover sets

$$\tilde{\mu} := \sum_{b \in \{0,1\}^k} s_1(b_1) + \dots + s_k(b_k).$$

3. **Combination randomness.** The verifier samples and sends $\varepsilon \leftarrow \mathbb{F}$.

4. **Sumcheck.** Set $\gamma := \emptyset$. For $j \in [k]$:

(a) The prover sends $\hat{h}_j \in \mathbb{F}^{<\max\{2, \ell_{zk}\}}[\mathbf{X}]$. In the honest case

$$\hat{h}_j(\mathbf{X}) := \sum_{(b_{j+1}, \dots, b_k) \in \{0,1\}^{k-j}} \mathbf{s}_1(\gamma_1) + \dots + \mathbf{s}_j(\mathbf{X}) + \dots + \mathbf{s}_k(b_k) \\ + \varepsilon \cdot \left(\langle \mathbf{f}(\gamma, \mathbf{X}, \{0,1\}^{k-j+\log \ell}), \odot_j \mathbf{sl}(\mathbf{st}, (\gamma, \mathbf{X})) \rangle + \sum_{i \in [n]} \langle \xi_i, \mathbf{sl}_i(\mathbf{st}_i) \rangle \right)$$

where $\mathbf{f}(\gamma, \mathbf{X}, \{0,1\}^{k-j+\log \ell})$ denotes the linear polynomial (in $\mathbf{X}$) obtained by performing a partial evaluation of $\mathbf{f}$ on $(\gamma, \mathbf{X})$ and $(\odot_j \mathbf{sl})(\mathbf{st}, (\gamma, \mathbf{X}))$ is well-defined by Equation 8.

(b) The verifier samples and sends $\gamma_j \leftarrow \mathbb{F}$. Set $\gamma := \gamma \parallel \gamma_j \in \mathbb{F}^j$.

• **Decision phase.**

1. The verifier checks that $\hat{h}_1(0) + \hat{h}_1(1) = \varepsilon \cdot \mu + \tilde{\mu}$.
2. For every $j \in \{2, \dots, k\}$ the verifier checks that $\hat{h}_j(0) + \hat{h}_j(1) = \hat{h}_{j-1}(\gamma_{j-1})$.

• **Output claims.** The verifier sets new targets and states:

- $\mu' := \hat{h}_k(\gamma_k)$;
- $\mathbf{st}' := ((\mathbf{st}, \gamma), \varepsilon)$;
- $\forall i \in [n], \mathbf{st}'_i := (\mathbf{st}_i, \varepsilon)$ and $\forall j \in [k], \mathbf{st}'_{n+j} := \gamma_j$;
- $\forall i \in [n], \mathbf{st}'_{o,i} := \mathbf{st}_{o,i}, \mu'_i := \mu_i$;
- $\forall j \in [k], \mathbf{st}'_{o,n+j} := \mathbf{0}, \mu'_{n+j} := \mathbf{0}$.

The verifier outputs the following new explicit instance and implicit instance and in the honest case the prover outputs the following new witness:

$$\mathbf{x}' := ((\mu', \mathbf{st}', (\mathbf{st}'_i)_{i \in [n+k]}), (\mu'_i, \mathbf{st}'_{o,i})_{i \in [n+k]}), \\ \mathbf{y}' := (\text{Fold}(\mathbf{f}, \gamma), (\xi_i)_{i \in [n]}, (\mathbf{s}_j)_{j \in [k]}), \\ \mathbf{w}' := (\text{Fold}(\mathbf{f}, \gamma), \text{Fold}(\mathbf{r}, \gamma), (\xi_i, \mathbf{r}_i)_{i \in [n]}, (\mathbf{s}_j, \mathbf{r}'_j)_{j \in [k]}).$$

Completeness of Construction 6.3 follows by completeness of the sumcheck protocol and by Lemma 3.26.

6.1 Zero-knowledge

Lemma 6.4. *If $\text{char}(\mathbb{F}) \neq 2$, $\ell_{zk} \geq 2$, and $\text{Enc}_{C_{zk}}$ is a t_{zk} -query zero-knowledge encoding with error $\zeta_{C_{zk}}$ then, for every $\mathbf{t} \in \mathbb{N}^{1+n}$, Construction 6.3 is HVZK for $\mathcal{D}^{\leq(\mathbf{t}, t_{zk}, \dots, t_{zk})}$ with error $k \cdot \zeta_{C_{zk}}$ and query complexity $\mathbf{t}$.*

Proof. Let $\text{Sim}_{C_{zk}}$ denote the simulator for $\text{Enc}_{C_{zk}}$. We define the simulator $\mathbf{S}$ for the protocol.

$\mathbf{S}^{\mathbf{y}, \mathcal{D}}(\mathbf{x})$:

1. Sample $\varepsilon, \tilde{\mu} \leftarrow \mathbb{F}$ and $(\gamma_j)_{j \in [k]} \leftarrow \mathbb{F}^k$.
2. Sample $(\hat{h}_j)_{j \in [k]}$ uniformly from $\mathbb{F}^{<\max\{2, \ell_{zk}\}}[\mathbf{X}]$ conditioned on $\hat{h}_1(0) + \hat{h}_1(1) = \varepsilon \cdot \mu + \tilde{\mu}$ and $\forall j \in \{2, \dots, k\}, \hat{h}_j(0) + \hat{h}_j(1) = \hat{h}_{j-1}(\gamma_{j-1})$.
3. Define $\mathbf{x}'$ and $\mathbf{y}'$ from $(\mathbf{x}, \varepsilon, \gamma, \tilde{\mu}, (\hat{h}_j)_{j \in [k]})$ as in the output-claims phase of Construction 6.3.

4. Since $\mathcal{D}$ is non-adaptive (see Definition 4.7) there exists $(\mathcal{D}_0, \mathcal{D}_1)$ such that

$$\mathcal{D}^{\mathbf{y}'}(\mathbf{x}') = \left[\begin{array}{l} (\text{st}, Q_{\mathbf{y}}, (Q_{s_j})_{j \in [k]}) \leftarrow \mathcal{D}_0(\mathbf{x}') \\ \mathcal{D}_1(\text{st}, \mathbf{a}_{\mathbf{y}}, (\mathbf{a}_{s_j})_{j \in [k]}) \end{array} \right].$$

5. Answer all queries in $Q_{\mathbf{y}}$ by forwarding to $\mathbf{y}$ (for queries to $\text{Fold}(f, \gamma)$ use the folding map), and denote the resulting answers by $\mathbf{a}_{\mathbf{y}}$. For every $j \in [k]$, set $\mathbf{a}_{s_j} \leftarrow \text{Sim}_{C_{z_k}}(Q_{s_j})$.
6. Set $\text{out} := \mathcal{D}_1(\text{st}, \mathbf{a}_{\mathbf{y}}, (\mathbf{a}_{s_j})_{j \in [k]})$ and output

$$\left((\varepsilon, \gamma), (Q_{\mathbf{y}}, (Q_{s_j})_{j \in [k]}), ((\tilde{\mu}, \hat{h}_1, \dots, \hat{h}_k), \mathbf{a}_{\mathbf{y}}, (\mathbf{a}_{s_j})_{j \in [k]}), \text{out} \right).$$

The simulator queries $\mathbf{y}$ exactly on $Q_{\mathbf{y}}$, and by the distinguisher class bound this is at most t . We must show that for every $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{C \equiv 2^k, C_{z_k}, \mathbf{sl}}$ and every distinguisher $\mathcal{D}$ in the stated class,

$$\Delta \left(\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}, \mathbf{x}, \mathbf{y}, \mathbf{w}), \mathbf{S}^{\mathbf{y}, \mathcal{D}}(\mathbf{x}) \right) \leq k \cdot \zeta_{C_{z_k}}.$$

We prove this in two parts: first we show that the sumcheck-transcript component has the correct distribution, then we bound the oracle-answer simulation error.

- *Sumcheck transcript.* Fix verifier randomness (ε, γ) . Let $\mathcal{T} \subseteq \mathbb{F}^{1+k \cdot \ell_{z_k}}$ be the affine subspace of tuples $(\tilde{\mu}, \hat{h}_1, \dots, \hat{h}_k)$ satisfying the k verifier checks in Construction 6.3 (where we think of each $\hat{h}_j$ as a list of ℓ_{z_k} evaluations). Since each check imposes one independent linear constraint on the polynomial $\hat{h}_j$, the subspace $\mathcal{T}$ has dimension $1 + k \cdot (\ell_{z_k} - 1)$. The simulator samples uniformly from $\mathcal{T}$ by construction.

In the honest execution, the masks $(\mathbf{s}_j)_{j \in [k]}$ are uniform in $(\mathbb{F}^{<\ell_{z_k}}[\mathbf{X}])^k$. The honest-prover formulas in Construction 6.3 define an affine function $\mathbf{A}: (\mathbb{F}^{<\ell_{z_k}}[\mathbf{X}])^k \rightarrow \mathbb{F}^{1+k \cdot \ell_{z_k}}$ that maps $(\mathbf{s}_1, \dots, \mathbf{s}_k) \mapsto (\tilde{\mu}, \hat{h}_1, \dots, \hat{h}_k)$. For $\mathbf{z} \in (\mathbb{F}^{<\ell_{z_k}}[\mathbf{X}])^k$, let $\mathbf{L}(\mathbf{z}) := \mathbf{A}(\mathbf{z}) - \mathbf{A}(0)$ be its linear part, so that $\mathbf{A}(\mathbf{z}) - \mathbf{A}(\mathbf{z}') = \mathbf{L}(\mathbf{z} - \mathbf{z}')$ for all $\mathbf{z}, \mathbf{z}' \in (\mathbb{F}^{<\ell_{z_k}}[\mathbf{X}])^k$. Expanding the sumcheck formula, $\mathbf{s}_j$ is the only mask contributing non-constant (in $\mathbf{X}$) terms to $\hat{h}_j$. Suppose $\mathbf{z} := (\mathbf{s}_1, \dots, \mathbf{s}_k)$, $\mathbf{z}' := (\mathbf{s}'_1, \dots, \mathbf{s}'_k)$ satisfy $\mathbf{A}(\mathbf{z}) = \mathbf{A}(\mathbf{z}')$, i.e., $\mathbf{z} - \mathbf{z}' \in \ker(\mathbf{L})$. Since the non-constant coefficients of $\hat{h}_j$ are 2^{k-j} times those of $\mathbf{s}_j$ and $\text{char}(\mathbb{F}) \neq 2$, each difference $\mathbf{s}_j - \mathbf{s}'_j$ is some constant we denote by $c_j \in \mathbb{F}$. Such constants shift each $\hat{h}_j$ by $2^{k-j} \sum_{l=1}^k c_l$ and $\tilde{\mu}$ by $2^k \sum_{l=1}^k c_l$. Therefore $\ker(\mathbf{L}) = \{(c_1, \dots, c_k) \in \mathbb{F}^k : \sum_j c_j = 0\}$ (formally, c_j is identified with the constant polynomial $\mathbf{X} \mapsto c_j$). Since $\dim((\mathbb{F}^{<\ell_{z_k}}[\mathbf{X}])^k) = k \cdot \ell_{z_k}$ and $\dim \ker(\mathbf{L}) = k - 1$, rank-nullity gives $\dim \text{im}(\mathbf{L}) = 1 + k \cdot (\ell_{z_k} - 1) = \dim(\mathcal{T})$. Also, $\text{im}(\mathbf{A}) = \mathbf{A}(0) + \text{im}(\mathbf{L})$, so $\text{im}(\mathbf{A})$ is an affine subspace with the same dimension as $\text{im}(\mathbf{L})$. Because $\text{im}(\mathbf{A}) \subseteq \mathcal{T}$ and both affine subspaces have dimension $1 + k \cdot (\ell_{z_k} - 1)$, we get $\text{im}(\mathbf{A}) = \mathcal{T}$. Fix $t \in \mathcal{T}$ and choose any $\mathbf{z}_0 \in \mathbf{A}^{-1}(t)$. For any $\mathbf{z} \in (\mathbb{F}^{<\ell_{z_k}}[\mathbf{X}])^k$,

$$\mathbf{A}(\mathbf{z}) = t \iff \mathbf{A}(\mathbf{z}) - \mathbf{A}(\mathbf{z}_0) = 0 \iff \mathbf{L}(\mathbf{z} - \mathbf{z}_0) = 0 \iff \mathbf{z} - \mathbf{z}_0 \in \ker(\mathbf{L}).$$

Hence $\mathbf{A}^{-1}(t) = \mathbf{z}_0 + \ker(\mathbf{L})$, so $|\mathbf{A}^{-1}(t)| = |\ker(\mathbf{L})| = |\mathbb{F}|^{k-1}$. Therefore uniform masks induce the uniform distribution over $\mathcal{T}$.

- *Oracle answers.* Let $Q_{\mathbf{y}}$ and $(Q_{s_j})_{j \in [k]}$ be the query sets output by $\mathcal{D}_0$ in the simulator above. For $j \in \{0, \dots, k\}$, define a hybrid $H^{(j)}$ that answers $Q_{\mathbf{y}}$ from $\mathbf{y}$, uses $\text{Sim}_{\mathcal{C}_{\text{zk}}}$ for Q_{s_l} with $l \leq j$, and uses honest values for $l > j$. Then $H^{(0)}$ is the real experiment and $H^{(k)}$ is the simulated one.

For each $j \in [k]$, hybrids $H^{(j-1)}$ and $H^{(j)}$ differ only in the oracle s_j . Since $|Q_{s_j}| \leq t_{\text{zk}}$, by Definition 3.16 replacing honest answers $s_j[Q_{s_j}]$ with $\text{Sim}_{\mathcal{C}_{\text{zk}}}(Q_{s_j})$ changes statistical distance by at most $\zeta_{\mathcal{C}_{\text{zk}}}$. Summing over j gives a total simulation error of at most $k \cdot \zeta_{\mathcal{C}_{\text{zk}}}$.

By the first part, the simulator samples the sumcheck transcript with exactly the same distribution as in the real execution, hence the induced distribution of $\mathbf{x}'$ and of the query sets output by $\mathcal{D}_0$ is also the same. Therefore the differences in the simulation stem from the oracle-answer simulation which is bounded above (and applying $\mathcal{D}_1$ to these answers to produce **out**). Thus $\Delta(\text{View}(\mathbf{P}, \mathbf{V}, \mathcal{D}, \mathbf{x}, \mathbf{y}, \mathbf{w}), \mathbf{S}^{\mathbf{y}, \mathcal{D}}(\mathbf{x})) \leq k \cdot \zeta_{\mathcal{C}_{\text{zk}}}$. $\square$

6.2 Round-by-round security

Lemma 6.5. *For every $\delta, \delta_{\text{zk}} \in (0, 1)$, Construction 6.3 has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}}, \tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}'}^{\delta, \delta_{\text{zk}}})$ and errors*

$$\left(\frac{|\Lambda(\mathcal{C}^{\equiv 2^k}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \left(\epsilon_{\text{mca}}(\mathcal{C}^{\equiv 2^{k-j}}, \delta) + \frac{\ell_{\text{zk}} \cdot |\Lambda(\mathcal{C}^{\equiv 2^{k-(j-1)}}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|} \right)_{j \in [k]} \right).$$

The total extraction time is $O(\sum_{j \in [k]} \text{tcor}(\mathcal{C}^{\equiv 2^{k-j}}))$.

Proof. We give notation, describe the extractors, describe a state function, and finally we establish the RBR knowledge soundness errors (based on the extractor and state function).

Notation. Let $\mathbf{x} = ((\mu, \mathbf{st}, (\mathbf{st}_i)_{i \in [n]}), (\boldsymbol{\mu}_i, \mathbf{st}_{o,i})_{i \in [n]})$ and $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$. A complete transcript of the protocol has the form $\mathbf{tr} = ((s_j)_{j \in [k]}, \tilde{\mu}, \varepsilon, ((\hat{h}_j, \gamma_j))_{j \in [k]})$. Given a partial transcript, we define the following symbols (these are defined whenever they can be derived from the partial transcript):

- $\boldsymbol{\gamma}^{(j)} = (\gamma_1, \dots, \gamma_j)$;
- $f^{(j)} = \text{Fold}(f, \boldsymbol{\gamma}^{(j)})$; and
- $\mu^{(j)} = \hat{h}_j(\gamma_j)$ for $j > 0$ and $\mu^{(0)} = \varepsilon \cdot \mu + \tilde{\mu}$.

Extractors. We describe the round-by-round extractors from the last round to the first.

1. **Final round.** $\mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \mathbf{tr} \parallel \gamma_k, \mathbf{w})$:

- (a) Parse $\mathbf{w}$ as $((\mathbf{f}, \mathbf{r}), (\boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]}, (\mathbf{s}_j, \mathbf{r}'_j)_{j \in [k]})$.
- (b) Compute $\bar{f} = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r})$ and for every $i \in [n]$ compute $\bar{\xi}_i := \text{Enc}_{\mathcal{C}_{\text{zk}}}(\boldsymbol{\xi}_i, \mathbf{r}_i)$ and for every $j \in [k]$ compute $\bar{s}_j := \text{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{s}_j, \mathbf{r}'_j)$.
- (c) Output the round witness $\mathbf{w}' := ((\bar{f}, \mathbf{f}, \mathbf{r}), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}_j, \mathbf{s}_j, \mathbf{r}'_j)_{j \in [k]})$.

2. **Sumcheck round** $j \in [k]$. $\mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \mathbf{tr} \parallel \gamma_j, \mathbf{w})$:

- (a) Parse $\mathbf{w}$ as $((\bar{f}^{(j)}, \mathbf{f}^{(j)}, \mathbf{r}^{(j)}), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}_{j'}, \mathbf{s}_{j'}, \mathbf{r}'_{j'})_{j' \in [k]})$.
- (b) Parse $f^{(j)}$ and $f^{(j-1)}$ from $\mathbf{tr}$.

- (c) Computes the largest set $S \subseteq [m]$ such that $\bar{f}^{(j)}$ agrees with $f^{(j)}$ and computes the erasure correction $\bar{f}^{(j-1)} := \mathbf{Enc}_{\mathcal{C}^{\equiv 2^{k-j}}}(f^{(j-1)}, S)$ (see Definition 3.9).
- (d) Compute $(\mathbf{f}^{(j-1)}, \mathbf{r}^{(j-1)}) := \mathbf{Enc}_{\mathcal{C}^{\equiv 2^{k-j}}}^{-1}(\bar{f}^{(j-1)})$.
- (e) Output the round witness $\mathbf{w}' := \left((\bar{f}^{(j-1)}, \mathbf{f}^{(j-1)}, \mathbf{r}^{(j-1)}), (\bar{\xi}_i, \xi_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}_{j'}, \mathbf{s}_{j'}, \mathbf{r}'_{j'})_{j' \in [k]} \right)$.

3. **Combination randomness.** $\mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \varepsilon, \mathbf{w})$ outputs $\mathbf{w}$.

The state function. We define a state function KState .

0. **Initial transcript:** We set $\text{KState}(\mathbb{X}, \mathbb{Y}, \emptyset, \mathbf{w}) = 1$ if and only if $(\mathbb{X}, \mathbb{Y}, \mathbf{w}) \in \tilde{\mathcal{R}}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{\text{zk}}, \text{sl}}^{\delta, \delta_{\text{zk}}}$.

1. **Combination randomness:** The transcript has the form $\text{tr} = ((s_j)_{j \in [k]}, \tilde{\mu})$ and the verifier samples $\varepsilon \leftarrow \mathbb{F}$. We set $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \varepsilon, \mathbf{w}) = 1$ if and only if

$$\mathbf{w} = \left((\bar{f}, \mathbf{f}, \mathbf{r}), (\bar{\xi}_i, \xi_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}_j, \mathbf{s}_j, \mathbf{r}'_j)_{j \in [k]} \right)$$

is such that

- $\Delta((f, \xi_1, \dots, \xi_n, s_1, \dots, s_k), (\bar{f}, \bar{\xi}_1, \dots, \bar{\xi}_n, \bar{s}_1, \dots, \bar{s}_k)) \leq (\delta, \delta_{\text{zk}}, \dots, \delta_{\text{zk}})$;
 - $\bar{f} = \mathbf{Enc}_{\mathcal{C}^{\equiv 2^k}}(\mathbf{f}, \mathbf{r}), \forall i \in [n] \bar{\xi}_i = \mathbf{Enc}_{\mathcal{C}_{\text{zk}}}(\xi_i, \mathbf{r}_i), \forall j \in [k] \bar{s}_j = \mathbf{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{s}_j, \mathbf{r}'_j)$;
 - $\forall i \in [n], \text{sl}_{\circ, i}(\text{st}_{\circ, i}) \cdot \xi_i = \mu_i$;
 - $\sum_{b \in \{0,1\}^k} (s_1(b_1) + \dots + s_k(b_k)) + \varepsilon \cdot (\langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle) = \tilde{\mu} + \varepsilon \cdot \mu$.
2. **Sumcheck, round $j \in [k]$:** The transcript has the form $\text{tr} = ((s_{j'})_{j' \in [k]}, \tilde{\mu}, \varepsilon, (\hat{h}_{j'}, \gamma_{j'})_{j' < j}, \hat{h}_j)$. The verifier samples $\gamma_j \leftarrow \mathbb{F}$. We set $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \gamma_j, \mathbf{w}) = 1$ if and only if

$$\mathbf{w} = \left((\bar{f}^{(j)}, \mathbf{f}^{(j)}, \mathbf{r}^{(j)}), (\bar{\xi}_i, \xi_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}_{j'}, \mathbf{s}_{j'}, \mathbf{r}'_{j'})_{j' \in [k]} \right)$$

is such that

- $\Delta((f^{(j)}, \xi_1, \dots, \xi_n, s_1, \dots, s_k), (\bar{f}^{(j)}, \bar{\xi}_1, \dots, \bar{\xi}_n, \bar{s}_1, \dots, \bar{s}_k)) \leq (\delta, \delta_{\text{zk}}, \dots, \delta_{\text{zk}})$;
- $\bar{f}^{(j)} = \mathbf{Enc}_{\mathcal{C}^{\equiv 2^{k-j}}}(\mathbf{f}^{(j)}, \mathbf{r}^{(j)}), \forall i \in [n] \bar{\xi}_i = \mathbf{Enc}_{\mathcal{C}_{\text{zk}}}(\xi_i, \mathbf{r}_i), \forall j' \in [k] \bar{s}_{j'} = \mathbf{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{s}_{j'}, \mathbf{r}'_{j'})$;
- $\hat{h}_1(0) + \hat{h}_1(1) = \tilde{\mu} + \varepsilon \cdot \mu$ and $\forall j' \in [j-1] \hat{h}_{j'+1}(0) + \hat{h}_{j'+1}(1) = \hat{h}_{j'}(\gamma_{j'})$;
- $\forall i \in [n], \text{sl}_{\circ, i}(\text{st}_{\circ, i}) \cdot \xi_i = \mu_i$;
- $\sum_{(b_j, \dots, b_k) \in \{0,1\}^{k-j}} (s_1(\gamma_1) + \dots + s_k(b_k)) + \varepsilon \cdot (\langle \mathbf{f}^{(j)}, \odot_j \text{sl}(\text{st}, \gamma) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle) = \hat{h}_j(\gamma_j)$.

Bounding the errors. We bound the RBR errors for the protocol.

1. **Combination randomness.** Let $\text{tr} = ((s_j)_{j \in [k]}, \tilde{\mu})$. We show that:

$$\Pr \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \varepsilon, \mathbf{w}) = 1 \end{array} \right] \leq \frac{|\Lambda(\mathcal{C}^{\equiv 2^k}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|}.$$

Fix $\mathbf{w}$ such that $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \varepsilon, \mathbf{w}) = 1$. There are at most $|\Lambda(\mathcal{C}^{\equiv 2^k}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|$ choices of $\mathbf{w}$ such that the conditions in Item 1 hold. Since $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$ and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \varepsilon, \mathbf{w}) = 1$ (and most of the items in state function are independent of ε), it must be that either

$$\sum_{\mathbf{b} \in \{0,1\}^k} (\mathbf{s}_1(b_1) + \dots + \mathbf{s}_k(b_k)) \neq \tilde{\mu} \text{ or}$$

$$\langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle \neq \mu.$$

By the polynomial identity lemma, Item 1 holds with probability $\frac{1}{|\mathbb{F}|}$. Taking a union bound concludes this part.

2. **Sumcheck randomness.** Let $\text{tr} = ((s_{j'})_{j' \in [k]}, \tilde{\mu}, \varepsilon, (\hat{h}_{j'}, \gamma_{j'})_{j' < j}, \hat{h}_j)$. We show that:

$$\Pr \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \gamma_j, \mathbf{w}) = 1 \end{array} \right]$$

$$\leq \epsilon_{\text{mca}}(\mathcal{C}^{\equiv 2^{k-j}}, \delta) + \frac{\ell_{\text{zk}} \cdot |\Lambda(\mathcal{C}^{\equiv 2^{k-j+1}}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|}.$$

Consider the event E_1 that for every $S \subseteq [m]$ with $|S| \geq (1 - \delta) \cdot m$ if there exists $u \in \mathcal{C}^{\equiv 2^{k-j}}$ such $u(S) = f^{(j)}(S)$ then there exist $v_0, v_1 \in \mathcal{C}^{\equiv 2^{k-j}}$ such that $(v_0, v_1)(S) = f^{(j-1)}(S)$. Note that by Definition 3.14 we have that $\Pr[\neg E_1] \leq \epsilon_{\text{mca}}(\mathcal{C}^{\equiv 2^{k-j}}, \delta)$. The rest of the analysis is assuming that E_1 holds.

Fix $\mathbf{w}$ such that $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \gamma_j, \mathbf{w}) = 1$. Since $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w})) = 0$, it must be that the conditions in Item 2 hold and thus $\Delta(f^{(j)}, \bar{f}^{(j)}) \leq \delta$. Therefore (since E_1 holds) there are $v_0, v_1 \in \mathcal{C}^{\equiv 2^{k-j}}$ such that $(v_0, v_1)(S) = f^{(j-1)}(S)$. Since $\Delta((1 - \gamma_j) \cdot v_0 + \gamma_j \cdot v_1, \bar{f}) \leq \delta$ it must be that $\bar{f}^{(j)} = (1 - \gamma_j) \cdot v_0 + \gamma_j \cdot v_1$ and that v_0, v_1 are the words that $\mathbf{E}_{\text{rbr}}$ outputs. Let $\mathbf{f}_0, \mathbf{f}_1$ be the message (ignoring the randomness) underlying v_0, v_1 . There are at most $|\Lambda(\mathcal{C}^{\equiv 2^{k-j+1}}, \delta)|$ such choices of $\mathbf{f}_0, \mathbf{f}_1$, and at most $|\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|$ choices for the mask-related messages in $\mathbf{w}$.

It must be that $\hat{h}_j(0) + \hat{h}_j(1) = \hat{h}_{j-1}(\gamma_{j-1})$, and thus it must be that

$$\hat{h}_j(\mathbf{X}) \neq \sum_{(b_{j+1}, \dots, b_k) \in \{0,1\}^{k-j-1}} (\mathbf{s}_1(\gamma_1) + \dots + \mathbf{s}_j(\mathbf{X}) + \dots + \mathbf{s}_k(b_k)) +$$

$$\varepsilon \cdot \left(\langle \mathbf{X} \cdot \mathbf{f}_0 + (1 - \mathbf{X}) \cdot \mathbf{f}_1, \odot_j \text{sl}(\text{st}, (\gamma, \mathbf{X})) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle \right)$$

as the right hand side sums to $\hat{h}_{j-1}(\gamma_{j-1})$, and if the left hand side did then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, a contradiction. By the polynomial identity lemma, the probability this happens is bounded by $\frac{\ell_{\text{zk}}}{|\mathbb{F}|}$. A union bound over the possible choices of $\mathbf{w}$ concludes the lemma. $\square$

7 A non-succinct zero-knowledge protocol for constrained codes

We describe a zero-knowledge IOP for the relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$ (Definition 5.8). The IOP is *non-succinct*, and is used on small instances as a base case.

Theorem 7.1. *Fix the following:*

- $n \in \mathbb{N}$;
- $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\ell)^m$ has a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$;
- $\mathcal{C}_{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{zk}}} \equiv (\mathbb{F}^{\ell_{\text{zk}}})^{m_{\text{zk}}}$ has a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{zk}}}: \mathbb{F}^{\ell_{\text{zk}}} \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{zk}}}$;
- $\mathbf{sl} = (\mathbf{sl}, (\mathbf{sl}_i)_{i \in [n]}, (\mathbf{sl}_{o,i})_{i \in [n]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i \in \langle \mathbb{F}^{\ell_{\text{zk}}} \rangle$, $\mathbf{sl}_{o,i} \in \langle \mathbb{F}^{t_i \times \ell_{\text{zk}}} \rangle$;
- $t, t_{\text{zk}} \in \mathbb{N}$ are repetition parameters.

Construction 7.2 is a 2-round IOPP for $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$ with the following properties:

- **Prover communication (in field elements):** $m \cdot \iota + n \cdot m_{\text{zk}} \cdot \iota_{\text{zk}} + \ell + r + n \cdot (\ell_{\text{zk}} + r_{\text{zk}})$. Of these,
 - $m \cdot \iota$ are sent as an oracle over Σ ,
 - $n \cdot m_{\text{zk}} \cdot \iota_{\text{zk}}$ are sent as an oracle over Σ_{zk}^n , and
 - $\ell + r + n \cdot (\ell_{\text{zk}} + r_{\text{zk}})$ are sent as non-oracle messages.
- **Query complexity:** t queries to the first input oracle, t_{zk} queries to the remaining oracles.
- **Prover time (in field operations):** $O(\text{tenc}(\mathcal{C}) + n \cdot \text{tenc}(\mathcal{C}_{\text{zk}}) + t_{\text{sl}} + \sum_{i \in [n]} t_{\text{sl}_i} + \sum_{i \in [n]} t_{\text{sl}_{o,i}})$.
- **Verifier time (in field operations):** $O(\text{tenc}(\mathcal{C}) + n \cdot \text{tenc}(\mathcal{C}_{\text{zk}}) + t_{\text{sl}} + \sum_{i \in [n]} t_{\text{sl}_i} + \sum_{i \in [n]} t_{\text{sl}_{o,i}})$.
- **Round-by-round security:** For every $\delta, \delta_{\text{zk}} \in (0, 1)$, Construction 7.2 has RBR knowledge soundness with relaxation $\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}}$ and errors

$$\left(\epsilon_{\text{mca}}(\mathcal{C}, \delta) + \epsilon_{\text{mca}}(\mathcal{C}_{\text{zk}}^{\equiv n}, \delta_{\text{zk}}) + \frac{|\Lambda(\mathcal{C}^{\equiv 2}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv 2 \cdot n}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \max \left\{ (1 - \delta)^t, (1 - \delta_{\text{zk}})^{t_{\text{zk}}} \right\} \right).$$

The total extraction time is $O(\text{tcor}(\mathcal{C}) + n \cdot \text{tcor}(\mathcal{C}_{\text{zk}}))$.

- **Zero-knowledge:** If $\text{Enc}_{\mathcal{C}}$ is a t -query zero-knowledge encoding with error $\zeta_{\mathcal{C}}$ and $\text{Enc}_{\mathcal{C}_{\text{zk}}}$ is a t_{zk} -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_{\text{zk}}}$, then Construction 7.2 is HVZK with error $\zeta_{\mathcal{C}} + n \cdot \zeta_{\mathcal{C}_{\text{zk}}}$ and query complexity $(t, (t_{\text{zk}})_{i \in [n]})$.

Construction 7.2.

- **Inputs and notation.** The verifier receives explicit input $\mathbf{x} = ((\mu, \mathbf{st}, (\mathbf{st}_i)_{i \in [n]}), (\boldsymbol{\mu}_i, \mathbf{st}_{o,i})_{i \in [n]})$ and oracle access to implicit input $\mathbf{y} = (\mathbf{f}, (\xi_i)_{i \in [n]})$. Here $\mu \in \mathbb{F}$ and, for each $i \in [n]$, $\boldsymbol{\mu}_i \in \mathbb{F}^{t_i}$. In the honest case, the prover receives $\mathbf{x}$ and $\mathbf{y}$ as well as witness $\mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$.
- **Interaction phase.**
 1. **New masks.** The prover sends $g \in \Sigma^m$, $s_1, \dots, s_n \in \Sigma_{\text{zk}}^{m_{\text{zk}}}$, $\mu' \in \mathbb{F}$, and for each $i \in [n]$ sends $\mu'_i \in \mathbb{F}^{t_i}$. In the honest case, the prover samples $\mathbf{g} \in \mathbb{F}^\ell$, $\mathbf{r}' \in \mathbb{F}^r$ and for $i \in [n]$ the prover samples $\mathbf{s}_i \in \mathbb{F}^{\ell_{\text{zk}}}$ and $\mathbf{r}'_i \in \mathbb{F}^{r_{\text{zk}}}$ and sets $g := \text{Enc}_{\mathcal{C}}(\mathbf{g}, \mathbf{r}')$, $\mu' := \langle \mathbf{g}, \mathbf{sl}(\mathbf{st}) \rangle + \sum_{i \in [n]} \langle \mathbf{s}_i, \mathbf{sl}_i(\mathbf{st}_i) \rangle$ and for $i \in [n]$ sets $s_i := \text{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{s}_i, \mathbf{r}'_i)$ and $\mu'_i := \mathbf{sl}_{o,i}(\mathbf{st}_{o,i}) \cdot \mathbf{s}_i$.
 2. **Combination randomness.** The verifier samples and sends $\gamma \leftarrow \mathbb{F}$.
 3. **Answer.** The prover sends $\mathbf{f}^* \in \mathbb{F}^\ell$, $\mathbf{r}^* \in \mathbb{F}^r$ and for $i \in [n]$ sends $\xi_i^* \in \mathbb{F}^{\ell_{\text{zk}}}$ and $\mathbf{r}_i^* \in \mathbb{F}^{r_{\text{zk}}}$. In the honest case, $\mathbf{f}^* := \mathbf{g} + \gamma \cdot \mathbf{f}$ and $\mathbf{r}^* := \mathbf{r}' + \gamma \cdot \mathbf{r}$ and, for $i \in [n]$, $\xi_i^* := \mathbf{s}_i + \gamma \cdot \xi_i$ and $\mathbf{r}_i^* := \mathbf{r}'_i + \gamma \cdot \mathbf{r}_i$.

4. **Spotcheck randomness.** The verifier samples and sends $x_1, \dots, x_t \leftarrow [m]$ and $y_1, \dots, y_{t_{zk}} \leftarrow [m_{zk}]$.

• **Decision phase.** The verifier computes $f^* := \text{Enc}_{\mathcal{C}}(\mathbf{f}^*, \mathbf{r}^*)$ and, for each $i \in [n]$, computes $\xi_i^* := \text{Enc}_{\mathcal{C}_{zk}}(\xi_i^*, \mathbf{r}_i^*)$; then it performs the following checks.

- Target checks: $\bullet \forall i \in [n], \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \xi_i^* = \mu'_i + \gamma \cdot \mu_i$
 $\bullet \langle \mathbf{f}^*, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i^*, \text{sl}_i(\text{st}_i) \rangle = \mu' + \gamma \cdot \mu$.
- $\mathcal{C}$ spot checks: $\forall j \in [t], f^*(x_j) = g(x_j) + \gamma \cdot f(x_j)$.
- $\mathcal{C}_{zk}$ spot checks: $\forall i \in [n], \forall j \in [t_{zk}], \xi_i^*(y_j) = s_i(y_j) + \gamma \cdot \xi_i(y_j)$.

7.1 Zero-knowledge

Lemma 7.3. *If $\text{Enc}_{\mathcal{C}}$ is a t -query zero-knowledge encoding with error $\zeta_{\mathcal{C}}$ and $\text{Enc}_{\mathcal{C}_{zk}}$ is a t_{zk} -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_{zk}}$, then Construction 7.2 is HVZK with error $\zeta_{\mathcal{C}} + n \cdot \zeta_{\mathcal{C}_{zk}}$ and query complexity $(t, (t_{zk})_{i \in [n]})$.*

Proof. We define the simulator.

$\mathbf{S}^{\mathbb{Y}}(\mathbf{x})$:

1. Parse $\mathbf{x}$ as $((\mu, \text{st}, (\text{st}_i)_{i \in [n]}), (\mu_i, \text{st}_{o,i})_{i \in [n]})$, and recall that $\mathbf{y}$ has the form $(f, (\xi_i)_{i \in [n]})$.
2. Sample $\gamma \leftarrow \mathbb{F}$, $x_1, \dots, x_t \leftarrow [m]$, and $y_1, \dots, y_{t_{zk}} \leftarrow [m_{zk}]$.
3. Set $S := (x_1, \dots, x_t)$ and $S' := (y_1, \dots, y_{t_{zk}})$.
4. Query $f[S]$ and, for every $i \in [n]$, query $\xi_i[S']$.
5. Sample $g[S] \leftarrow \text{Sim}_{\mathcal{C}}(S)$ and, for every $i \in [n]$, sample $s_i[S'] \leftarrow \text{Sim}_{\mathcal{C}_{zk}}(S')$.
6. Sample $\mathbf{f}^* \leftarrow \mathbb{F}^{\ell}$ and $\mathbf{r}^* \leftarrow \mathbb{F}^r$ such that $\text{Enc}_{\mathcal{C}}(\mathbf{f}^*, \mathbf{r}^*)$ satisfies $\text{Enc}_{\mathcal{C}}(\mathbf{f}^*, \mathbf{r}^*)(S) = g[S] + \gamma \cdot f[S]$.
(If no solution exists, abort.)
7. For every $i \in [n]$, sample $\xi_i^* \leftarrow \mathbb{F}^{\ell_{zk}}$ and $\mathbf{r}_i^* \leftarrow \mathbb{F}^{r_{zk}}$ such that $\text{Enc}_{\mathcal{C}_{zk}}(\xi_i^*, \mathbf{r}_i^*)(S') = s_i[S'] + \gamma \cdot \xi_i[S']$. (If no solution exists, abort.)
8. Set $\mu' := \langle \mathbf{f}^*, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i^*, \text{sl}_i(\text{st}_i) \rangle - \gamma \cdot \mu$.
9. For every $i \in [n]$, set $\mu'_i := \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \xi_i^* - \gamma \cdot \mu_i$.
10. Output

$$\left(\begin{array}{l} \gamma, (x_1, \dots, x_t), (y_1, \dots, y_{t_{zk}}), \\ \mu', (\mu'_1, \dots, \mu'_n), g[S], (s_1[S'], \dots, s_n[S']), \\ \mathbf{f}^*, \mathbf{r}^*, (\xi_1^*, \dots, \xi_n^*), (\mathbf{r}_1^*, \dots, \mathbf{r}_n^*), \\ f[S], (\xi_1[S'], \dots, \xi_n[S']) \end{array} \right).$$

The simulator runs in polynomial time. This is straightforward except for Items 6 and 7. Since $\text{Enc}_{\mathcal{C}}$ and $\text{Enc}_{\mathcal{C}_{zk}}$ are injective linear maps, there exist matrices $\mathbf{G} \in \mathbb{F}^{m \times (\ell+r)}$ and $\mathbf{G}_{\mathcal{C}_{zk}} \in \mathbb{F}^{m_{zk} \times (\ell_{zk}+r_{zk})}$ such that $\text{Enc}_{\mathcal{C}}(\mathbf{u}) = \mathbf{G} \cdot \mathbf{u}$ and $\text{Enc}_{\mathcal{C}_{zk}}(\mathbf{u}) = \mathbf{G}_{\mathcal{C}_{zk}} \cdot \mathbf{u}$. Sampling $\mathbf{f}^*$ and $\mathbf{r}^*$ such that $\text{Enc}_{\mathcal{C}}(\mathbf{f}^*, \mathbf{r}^*)(S) = g[S] + \gamma \cdot f[S]$ is equivalent to sampling a solution to $(\mathbf{G} \cdot \mathbf{u})[S] = \mathbf{t}$, and similarly for each $\xi_i^*, \mathbf{r}_i^*$ with $\mathbf{G}_{\mathcal{C}_{zk}}$; both can be done in polynomial time.

Next we analyze the simulation error. We introduce an intermediate simulator $\mathbf{S}_0^{\mathbb{Y}}(\mathbf{x}, \mathbf{w})$ that has access to the witness to the relation.

$\mathbf{S}_0^{\mathbb{Y}}(\mathbf{x}, \mathbf{w}) = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$.

1. Sample $\gamma \leftarrow \mathbb{F}$, $x_1, \dots, x_t \leftarrow [m]$, and $y_1, \dots, y_{t_{zk}} \leftarrow [m_{zk}]$.
2. Set $S := (x_1, \dots, x_t)$ and $S' := (y_1, \dots, y_{t_{zk}})$.

3. Sample $\mathbf{g} \leftarrow \mathbb{F}^\ell$, $\mathbf{r}' \leftarrow \mathbb{F}^r$, and for every $i \in [n]$ sample $\mathbf{s}_i \leftarrow \mathbb{F}^{\ell_{\text{zk}}}$ and $\mathbf{r}'_i \leftarrow \mathbb{F}^{r_{\text{zk}}}$.
4. Set $g := \text{Enc}_{\mathcal{C}}(\mathbf{g}, \mathbf{r}')$ and, for every $i \in [n]$, $s_i := \text{Enc}_{\mathcal{C}_{\text{zk}}}(\mathbf{s}_i, \mathbf{r}'_i)$.
5. Set

$$\begin{aligned} \mathbf{f}^* &:= \mathbf{g} + \gamma \cdot \mathbf{f}, & \mathbf{r}^* &:= \mathbf{r}' + \gamma \cdot \mathbf{r}, \\ \forall i \in [n]: \mathbf{\xi}_i^* &:= \mathbf{s}_i + \gamma \cdot \mathbf{\xi}_i, & \mathbf{r}_i^* &:= \mathbf{r}'_i + \gamma \cdot \mathbf{r}_i. \end{aligned}$$

6. Set

$$\begin{aligned} \mu' &:= \langle \mathbf{g}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \mathbf{s}_i, \text{sl}_i(\text{st}_i) \rangle, \\ \forall i \in [n]: \mu'_i &:= \text{sl}_{\text{o},i}(\text{st}_{\text{o},i}) \cdot \mathbf{s}_i. \end{aligned}$$

7. Output

$$\left(\begin{array}{l} \gamma, (x_1, \dots, x_t), (y_1, \dots, y_{t_{\text{zk}}}), \\ \mu', (\mu'_1, \dots, \mu'_n), g[S], (s_1[S'], \dots, s_n[S']), \\ \mathbf{f}^*, \mathbf{r}^*, (\mathbf{\xi}_1^*, \dots, \mathbf{\xi}_n^*), (\mathbf{r}_1^*, \dots, \mathbf{r}_n^*), \\ f[S], (\xi_1[S'], \dots, \xi_n[S']) \end{array} \right).$$

Conditioned on $g[S]$ and $(s_i[S'])_{i \in [n]}$, the sampling of $(\mathbf{f}^*, \mathbf{r}^*)$ and $(\mathbf{\xi}_i^*, \mathbf{r}_i^*)$ in $\mathbf{S}_0$ is uniform over the solution sets of the linear systems in Item 6. Therefore the only statistical gap between $\mathbf{S}$ and $\mathbf{S}_0$ comes from simulating $g[S]$ and $(s_i[S'])_{i \in [n]}$: by Definition 3.16 and a union bound over these $n+1$ codewords, this gap is at most $\zeta_{\mathcal{C}} + n \cdot \zeta_{\mathcal{C}_{\text{zk}}}$. The distribution of $\mathbf{S}_0$ is identical to the real view, so the simulation error is at most $\zeta_{\mathcal{C}} + n \cdot \zeta_{\mathcal{C}_{\text{zk}}}$. $\square$

7.2 Round-by-round knowledge soundness

Lemma 7.4. *For every $\delta, \delta_{\text{zk}} \in (0, 1)$, Construction 7.2 has RBR knowledge soundness with relaxation $\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}^{\delta, \delta_{\text{zk}}}$ and errors*

$$\left(\epsilon_{\text{mca}}(\mathcal{C}, \delta) + \epsilon_{\text{mca}}(\mathcal{C}_{\text{zk}}^{\equiv n}, \delta_{\text{zk}}) + \frac{|\Lambda(\mathcal{C}^{\equiv 2}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv 2 \cdot n}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \max \left\{ (1 - \delta)^t, (1 - \delta_{\text{zk}})^{t_{\text{zk}}} \right\} \right).$$

The total extraction time is $O(\text{tcor}(\mathcal{C}) + n \cdot \text{tcor}(\mathcal{C}_{\text{zk}}))$.

Proof. We give notation, describe the extractors, describe a state function, and finally we establish the RBR knowledge soundness errors (based on the extractor and state function).

Notation. Let $\mathbf{z} = ((\mu, \text{st}, \text{st}_1, \dots, \text{st}_n), (\mu_i, \text{st}_{\text{o},i})_{i \in [n]})$ and $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$. A complete transcript of the protocol has the form

$$\text{tr} = \left(g, s_1, \dots, s_n, \mu', \mu'_1, \dots, \mu'_n, \gamma, \mathbf{f}^*, \mathbf{r}^*, (\mathbf{\xi}_i^*, \mathbf{r}_i^*)_{i \in [n]}, x_1, \dots, x_t, y_1, \dots, y_{t_{\text{zk}}} \right).$$

Extractor. We describe the round-by-round extractors from the last round to the first.

1. Spotcheck randomness.

- (a) Parse the transcript to obtain $\mathbf{f}^*$, $\mathbf{r}^*$, and $(\mathbf{\xi}_i^*, \mathbf{r}_i^*)_{i \in [n]}$.
- (b) Compute $\bar{f}^* := \text{Enc}_{\mathcal{C}}(\mathbf{f}^*, \mathbf{r}^*)$ and $(\bar{\xi}_1^*, \dots, \bar{\xi}_n^*) := \text{Enc}_{\mathcal{C}_{\text{zk}}^{\equiv n}}((\mathbf{\xi}_1^*, \dots, \mathbf{\xi}_n^*), (\mathbf{r}_1^*, \dots, \mathbf{r}_n^*))$.

(c) Outputs the round witness $\mathbf{w} := (\bar{f}^*, \mathbf{f}^*, \mathbf{r}^*, (\bar{\xi}_i^*, \boldsymbol{\xi}_i^*, \mathbf{r}_i^*)_{i \in [n]})$.

2. Combination randomness.

- (a) Parse $\mathbf{w}$ as $(\bar{f}^*, \mathbf{f}^*, \mathbf{r}^*, (\bar{\xi}_i^*, \boldsymbol{\xi}_i^*, \mathbf{r}_i^*)_{i \in [n]})$.
- (b) Let $S \subseteq [m]$ be the largest set on which $\bar{f}^*$ agrees with $g + \gamma \cdot f$.
- (c) Use the erasure corrector $\mathbf{E}_{\mathcal{C}}$ for $\mathcal{C}$ to obtain codewords $\bar{f}, \bar{g}$ from $f(S), g(S)$ and decode them to $(\mathbf{f}, \mathbf{r}), (\mathbf{g}, \mathbf{r}')$.
- (d) Let $S' \subseteq [m_{\text{zk}}]$ be the largest set on which $(\bar{\xi}_1^*, \dots, \bar{\xi}_n^*)$ agrees with $(s_1 + \gamma \cdot \xi_1, \dots, s_n + \gamma \cdot \xi_n)$.
- (e) For every $i \in [n]$, use the erasure corrector $\mathbf{E}_{\mathcal{C}_{\text{zk}}}$ for $\mathcal{C}_{\text{zk}}$ to obtain codewords $\bar{\xi}_i, \bar{s}_i$ from $\xi_i(S'), s_i(S')$ and decode them to $(\boldsymbol{\xi}_i, \mathbf{r}_i), (\mathbf{s}_i, \mathbf{r}'_i)$.
- (f) Output the round witness $\mathbf{w} := (\mathbf{f}, \mathbf{r}, \mathbf{g}, \mathbf{r}', (\boldsymbol{\xi}_i, \mathbf{r}_i, \mathbf{s}_i, \mathbf{r}'_i)_{i \in [n]})$.

3. Final extraction.

- (a) Parse $\mathbf{w}$ as $(\mathbf{f}, \mathbf{r}, \mathbf{g}, \mathbf{r}', (\boldsymbol{\xi}_i, \mathbf{r}_i, \mathbf{s}_i, \mathbf{r}'_i)_{i \in [n]})$.
- (b) Compute $\bar{f} := \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r})$ and for every $i \in [n]$ compute $\bar{\xi}_i := \text{Enc}_{\mathcal{C}_{\text{zk}}}(\boldsymbol{\xi}_i, \mathbf{r}_i)$.
- (c) Output $(\mathbf{w}, \mathbf{y}^*)$ where $\mathbf{w} := (\mathbf{f}, \mathbf{r}, (\boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]})$ and $\mathbf{y}^* := (\bar{f}, (\bar{\xi}_i)_{i \in [n]})$.

The state function. We define a state function KState .

0. **Initial transcript:** We set $\text{KState}(\mathbf{x}, \mathbf{y}, \emptyset, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \hat{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \text{sl}}^{\delta, \delta_{\text{zk}}}$.

1. **Combination randomness:** The transcript is $\text{tr} = (g, s_1, \dots, s_n, \mu', \boldsymbol{\mu}'_1, \dots, \boldsymbol{\mu}'_n)$ and the verifier samples $\gamma \leftarrow \mathbb{F}$. We set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \gamma, \mathbf{w}) = 1$ if and only if $\mathbf{w} = (\bar{f}^*, \mathbf{f}^*, \mathbf{r}^*, (\bar{\xi}_i^*, \boldsymbol{\xi}_i^*, \mathbf{r}_i^*))$ such that all of following hold:

- (a) $\bar{f}^* = \text{Enc}_{\mathcal{C}}(\mathbf{f}^*, \mathbf{r}^*)$ and $(\bar{\xi}_1^*, \dots, \bar{\xi}_n^*) = \text{Enc}_{\mathcal{C}_{\text{zk}}}((\boldsymbol{\xi}_1^*, \dots, \boldsymbol{\xi}_n^*), (\mathbf{r}_1^*, \dots, \mathbf{r}_n^*))$;
- (b) $\Delta(g + \gamma \cdot f, \bar{f}^*) \leq \delta$ and $\Delta((s_1 + \gamma \cdot \xi_1, \dots, s_n + \gamma \cdot \xi_n), (\bar{\xi}_1^*, \dots, \bar{\xi}_n^*)) \leq \delta_{\text{zk}}$;
- (c) the following both hold

$$\begin{aligned} \langle \mathbf{f}^*, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \boldsymbol{\xi}_i^*, \text{sl}_i(\text{st}_i) \rangle &= \mu' + \gamma \cdot \mu, \\ \forall i \in [n] : \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \boldsymbol{\xi}_i^* &= \boldsymbol{\mu}'_i + \gamma \cdot \boldsymbol{\mu}_i. \end{aligned}$$

2. **Spotcheck randomness:** The transcript is $\text{tr} = (g, (s_i)_{i \in [n]}, \mu', (\boldsymbol{\mu}'_i)_{i \in [n]}, \gamma, \mathbf{f}^*, \mathbf{r}^*, (\boldsymbol{\xi}_i^*, \mathbf{r}_i^*)_{i \in [n]})$ and the verifier samples $x_1, \dots, x_t$ and $y_1, \dots, y_{t_{\text{zk}}}$. We set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel (x_1, \dots, x_t, y_1, \dots, y_{t_{\text{zk}}}), \mathbf{w}) = 1$ if and only if $\mathbf{w} = \emptyset$ and the verifier accepts.

Bounding the errors. We bound the RBR errors for the protocol.

1. **Combination randomness.** We show that

$$\begin{aligned} \Pr_{\gamma} \left[\exists \mathbf{w} : \begin{aligned} &\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w})) = 0 \\ &\wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \gamma, \mathbf{w}) = 1 \end{aligned} \right] \\ \leq \epsilon_{\text{mca}}(\mathcal{C}, \delta) + \epsilon_{\text{mca}}(\mathcal{C}_{\text{zk}}^{\equiv n}, \delta_{\text{zk}}) + \frac{|\Lambda(\mathcal{C}^{\equiv 2}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv 2 \cdot n}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \end{aligned}$$

Let E_1 be the event that for every $S \subseteq [m]$ with $|S| \geq (1 - \delta) \cdot m$ if there exists a codeword $u \in \mathcal{C}$ such that $u(S) = (g + \gamma \cdot f)(S)$ then there exist $u_g, u_f \in \mathcal{C}$ such that $u_g(S) = g(S)$ and

$u_f(S) = f(S)$. Similarly, let E_2 be the event that for every $S' \subseteq [m_{z_k}]$ with $|S'| \geq (1 - \delta_{z_k}) \cdot m_{z_k}$ if there exists a codeword $u' \in \mathcal{C}_{z_k}^{\equiv n}$ such that $u'(S') = (s_1 + \gamma \cdot \xi_1, \dots, s_n + \gamma \cdot \xi_n)(S')$ then there exist codewords $(u_{\xi,1}, \dots, u_{\xi,n})$ in $\mathcal{C}_{z_k}^{\equiv n}$ such that $u_{\xi,i}(S') = \xi_i(S')$ for all $i \in [n]$. By Definition 3.14, $\Pr[\neg E_1] \leq \epsilon_{\text{mca}}(\mathcal{C}, \delta)$ and $\Pr[\neg E_2] \leq \epsilon_{\text{mca}}(\mathcal{C}_{z_k}^{\equiv n}, \delta_{z_k})$. We condition on $E_1 \wedge E_2$ in the rest of the analysis.

Fix $\mathbf{w} = (\bar{f}^*, \mathbf{f}^*, \mathbf{r}^*, (\bar{\xi}_i^*, \xi_i^*, \mathbf{r}_i^*)_{i \in [n]})$ such that $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \gamma, \mathbf{w}) = 1$ and let

$$(\mathbf{f}, \mathbf{r}, \mathbf{g}, \mathbf{r}', (\xi_i, \mathbf{r}_i, \mathbf{s}_i, \mathbf{r}'_i)_{i \in [n]}) = \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})$$

denote the output of the extractor on $\mathbf{w}$.

By E_1 and E_2 and the definition of the extractor $\mathbf{E}_{\text{rbr}}$ there are at most $|\Lambda(\mathcal{C}^{\equiv 2}, \delta)| \cdot |\Lambda(\mathcal{C}_{z_k}^{\equiv 2 \cdot n}, \delta_{z_k})|$ possible choices for $\mathbf{w}$. This is because:

- Since $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \gamma, \mathbf{w}) = 1$ it must be that $\Delta(g + \gamma \cdot f, \bar{f}^*) \leq \delta$ and similarly for $i \in [n]$ it must be that $\Delta(s_i + \gamma \cdot \xi_i, \bar{\xi}_i^*) \leq \delta_{z_k}$. Thus, by E_1, E_2 there must exist sets S and S' with $|S| \geq (1 - \delta) \cdot m$ and $|S'| \geq (1 - \delta_{z_k}) \cdot m_{z_k}$ such that (f, g) agrees with the interleaved code $\mathcal{C}^{\equiv 2}$ on S and $(\xi_1, \dots, \xi_n, s_1, \dots, s_n)$ agrees with the interleaved code $\mathcal{C}_{z_k}^{\equiv 2 \cdot n}$.
- The extractor by definition outputs the messages underlying the interleaved code, and thus there can be at most $|\Lambda(\mathcal{C}^{\equiv 2}, \delta)| \cdot |\Lambda(\mathcal{C}_{z_k}^{\equiv 2 \cdot n}, \delta_{z_k})|$ such choices.
- Further, it must be that $\mathbf{f}^* = \mathbf{g} + \gamma \cdot \mathbf{f}$. For any $x \in S$ it holds that $\text{Enc}_{\mathcal{C}}(\mathbf{f}^*, \mathbf{r}^*)(x) = \bar{f}^*(x) = g(x) + \gamma \cdot f(x) = \text{Enc}_{\mathcal{C}}(\mathbf{g}, \mathbf{r}')(x) + \gamma \cdot \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r})(x) = \text{Enc}_{\mathcal{C}}(\mathbf{g} + \gamma \cdot \mathbf{f}, \mathbf{r}' + \gamma \cdot \mathbf{r})(x)$. Since the two codewords agree on a set of size $(1 - \delta) \cdot m \geq (1 - \delta(\mathcal{C})) \cdot m$ they must be the same codeword, and since the encoding is injective it must in fact be that $\mathbf{f}^* = \mathbf{g} + \gamma \cdot \mathbf{f}$ and $\mathbf{r}^* = \mathbf{r}' + \gamma \cdot \mathbf{r}$. A similar fact also holds for the $\xi_i, \mathbf{s}_i$.

Suppose that $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0$, it must then be that either

$$\begin{aligned} \langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle &\neq \mu \text{ or} \\ \exists i \in [n] : \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \xi_i &\neq \mu_i. \end{aligned}$$

In either case, the probability over a choice of γ that

$$\begin{aligned} \langle g + \gamma \cdot \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \mathbf{s}_i + \gamma \cdot \xi_i, \text{sl}_i(\text{st}_i) \rangle &= \mu' + \gamma \cdot \mu \text{ and} \\ \forall i \in [n] : \text{sl}_{o,i}(\text{st}_{o,i}) \cdot (\mathbf{s}_i + \gamma \cdot \xi_i) &= \mu'_i + \gamma \cdot \mu_i. \end{aligned}$$

is at most $\frac{1}{|\mathbb{F}|}$. Since, as noted before, $\mathbf{f}^* = \mathbf{g} + \gamma \cdot \mathbf{f}$ and for $i \in [n]$ it holds that $\xi_i^* = \mathbf{s}_i + \gamma \cdot \xi_i$, this implies that $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w}) = 0$. Taking a union bound over all possible witnesses yields the bound above.

2. Spotcheck randomness. We show that

$$\Pr_{(x_1, \dots, x_t, y_1, \dots, y_{t_{z_k}})} \left[\begin{aligned} &\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \emptyset)) = 0 \\ &\wedge \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel (x_1, \dots, x_t, y_1, \dots, y_{t_{z_k}}), \emptyset) = 1 \end{aligned} \right] \leq \max \left\{ (1 - \delta)^t, (1 - \delta_{z_k})^{t_{z_k}} \right\}.$$

Write $(\bar{f}^*, \mathbf{f}^*, \mathbf{r}^*, (\bar{\xi}_i^*, \xi_i^*, \mathbf{r}_i^*)_{i \in [n]})$ for $\mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \emptyset)$. If the linear constraints in the decision phase fail, the verifier rejects regardless of the sampled points. Otherwise, either $\Delta(g + \gamma \cdot f, \bar{f}^*) > \delta$

or $\Delta((s_1 + \gamma \cdot \xi_1, \dots, s_n + \gamma \cdot \xi_n), (\bar{\xi}_1^*, \dots, \bar{\xi}_n^*)) > \delta_{zk}$. In the first case, $\Delta(g + \gamma \cdot f, \bar{f}^*) > \delta$, then with probability at most $(1 - \delta)^t$ all x_j fall in the agreement set, and the $\mathcal{C}$ -local checks pass. Similarly, if $\Delta((s_1 + \gamma \cdot \xi_1, \dots, s_n + \gamma \cdot \xi_n), (\bar{\xi}_1^*, \dots, \bar{\xi}_n^*)) > \delta_{zk}$, then with probability at most $(1 - \delta_{zk})^{t_{zk}}$ all y_j avoid disagreement positions and the $\mathcal{C}_{zk}$ -local checks pass.

□

8 A sublinear zero-knowledge IOP for constrained codes

We prove Theorem 1 by composing the IOR of Theorem 6.2 and the 2-round IOPP of Theorem 7.1.

Theorem 8.1. *Consider the following ingredients:*

- $n, k \in \mathbb{N}$;
- $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\iota)^m$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$;
- $\mathcal{C}_{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{zk}}} \equiv (\mathbb{F}^{\iota_{\text{zk}}})^{m_{\text{zk}}}$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{zk}}}: \mathbb{F}^{\ell_{\text{zk}}} \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{zk}}}$;
- $\mathbf{sl} = (\mathbf{sl}, (\mathbf{sl}_i)_{i \in [n]}, (\mathbf{sl}_{\circ, i})_{i \in [n]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i \in \langle \mathbb{F}^{\ell_{\text{zk}}} \rangle$, $\mathbf{sl}_{\circ, i} \in \langle \mathbb{F}^{t_i \times \ell_{\text{zk}}} \rangle$;
- $t, t_{\text{zk}} \in \mathbb{N}$ are repetition parameters.

Construction 8.2 is an IOPP for $\mathcal{R}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$ with the following parameters.

- **Round complexity:** $k + 3$.
- **Prover communication (in field elements):** $O(k \cdot (\ell_{\text{zk}} + m_{\text{zk}} \cdot \iota_{\text{zk}}) + m \cdot \iota + (n + k) \cdot m_{\text{zk}} \cdot \iota_{\text{zk}} + \ell + r + (n + k) \cdot (\ell_{\text{zk}} + r_{\text{zk}}))$. Of these $m \cdot \iota$ are sent as an oracle with alphabet Σ , $(n + 2 \cdot k) \cdot m_{\text{zk}} \cdot \iota_{\text{zk}}$ are sent as an oracle with alphabet $\Sigma_{\text{zk}}^{n+2 \cdot k}$, and $k \cdot \ell_{\text{zk}} + \ell + r + (n + k) \cdot (\ell_{\text{zk}} + r_{\text{zk}})$ are sent as non-oracle messages.
- **Verifier queries:** the verifier makes t oracle queries to f over alphabet Σ^{2^k} and, for each mask oracle in the protocol (namely the n masks from the instance together with the k masks introduced by the sumcheck IOR), it makes at most t_{zk} oracle queries over alphabet Σ_{zk} .
- **Prover time (in field operations):** $O(2^k \cdot \ell + t_{\text{sl}} + k \cdot \text{tenc}(\mathcal{C}_{\text{zk}}) + k \cdot \ell_{\text{zk}} + \text{tenc}(\mathcal{C}) + (n + k) \cdot \text{tenc}(\mathcal{C}_{\text{zk}}) + \sum_{i \in [n]} t_{\text{sl}_i} + \sum_{i \in [n]} t_{\text{sl}_{\circ, i}} + k \cdot t_{\text{sl}_{\text{ze}_{\ell_{\text{zk}}}}} + k \cdot t_{\text{sl}_{\text{id}}})$.
- **Verifier time (in field operations):** $O(2^k \cdot t + k \cdot \ell_{\text{zk}} + \text{tenc}(\mathcal{C}) + (n + k) \cdot \text{tenc}(\mathcal{C}_{\text{zk}}) + t_{\circ_k \text{sl}} + \sum_{i \in [n]} t_{\text{sl}_i} + \sum_{i \in [n]} t_{\text{sl}_{\circ, i}} + k \cdot t_{\text{sl}_{\text{ze}_{\ell_{\text{zk}}}}} + k \cdot t_{\text{sl}_{\text{id}}})$.
- **Round-by-round security:** For every $\delta, \delta_{\text{zk}} \in (0, 1)$, Construction 8.2 has RBR knowledge soundness with relaxation $\tilde{\mathcal{R}}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}}$ and errors:

$$\left(\frac{|\Lambda(\mathcal{C}^{\equiv 2^k}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \left(\epsilon_{\text{mca}}(\mathcal{C}^{\equiv 2^{k-j}}, \delta) + \frac{\ell_{\text{zk}} \cdot |\Lambda(\mathcal{C}^{\equiv 2^{k-(j-1)}}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}})|}{|\mathbb{F}|} \right)_{j \in [k]}, \right. \\ \left. \epsilon_{\text{mca}}(\mathcal{C}, \delta) + \epsilon_{\text{mca}}(\mathcal{C}_{\text{zk}}^{\equiv n+k}, \delta_{\text{zk}}) + \frac{|\Lambda(\mathcal{C}^{\equiv 2}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv 2(n+k)}, \delta_{\text{zk}})|}{|\mathbb{F}|}, \max \left\{ (1 - \delta)^t, (1 - \delta_{\text{zk}})^{t_{\text{zk}}} \right\} \right).$$

The total extraction time is $O\left(\sum_{j \in [k]} \text{tcor}(\mathcal{C}^{\equiv 2^{k-j}}) + (n + k) \cdot \text{tcor}(\mathcal{C}_{\text{zk}})\right)$.

- **Zero-knowledge:** If $\text{char}(\mathbb{F}) \neq 2$, $\ell_{\text{zk}} \geq 2$, $\text{Enc}_{\mathcal{C}}$ is a t -query zero-knowledge encoding with error $\zeta_{\mathcal{C}}$, and $\text{Enc}_{\mathcal{C}_{\text{zk}}}$ is a t_{zk} -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_{\text{zk}}}$, then Construction 8.2 is HVZK with error $2^k \cdot \zeta_{\mathcal{C}} + (n + 2k) \cdot \zeta_{\mathcal{C}_{\text{zk}}}$ and query complexity $(t, (t_{\text{zk}})_{i \in [n]})$.

Construction 8.2.

- **Inputs.** The verifier receives explicit input $\mathbf{x} = ((\mu, \text{st}, (\text{st}_i)_{i \in [n]}), ((\mu_i)_{i \in [n]}, (\text{st}_{\circ, i})_{i \in [n]}))$ and oracle access to implicit input $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$. In the honest case, the prover receives as input $\mathbf{x}$ and $\mathbf{y}$ as well as witness $\mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$.
- **Interaction phase.**

- **Sumcheck IOR.** The prover and verifier run the zero-knowledge sumcheck IOR from $\mathcal{R}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{zk}, \mathbf{sl}}$ to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \mathbf{sl}'}$ in Construction 6.3, leading to:

$$\begin{aligned} \mathbf{x}' &:= \left((\mu', \mathbf{st}', (\mathbf{st}'_i)_{i \in [n+k]}), (\mu'_i, \mathbf{st}'_{o,i})_{i \in [n+k]} \right), \\ \mathbf{y}' &:= (\text{Fold}(f, \gamma), (\xi_i)_{i \in [n]}, (s_j)_{j \in [k]}), \\ (\text{in the honest case}) \quad \mathbf{w}' &:= (\text{Fold}(\mathbf{f}, \gamma), \text{Fold}(\mathbf{r}, \gamma), (\xi_i, \mathbf{r}_i)_{i \in [n]}, (\mathbf{s}_j, \mathbf{r}'_j)_{j \in [k]}), \\ \mathbf{sl}' &:= (\times(\odot_k \mathbf{sl}), (\times(\mathbf{sl}_i))_{i \in [n]}, (\mathbf{sl}_{ze_{\ell_{zk}}^*})_{j \in [k]}, (\mathbf{sl}_{o,i})_{i \in [n]}, (\mathbf{sl}_{id})_{j \in [k]}). \end{aligned}$$

Above $\gamma \in \mathbb{F}^k$ is the randomness sampled during the sumcheck protocol.

- **Non-succinct IOPP.** The prover and verifier run the IOPP for the relation $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \mathbf{sl}'}$ in Construction 7.2 on $(\mathbf{x}', \mathbf{y}', \mathbf{w}')$, with repetition parameters (t, t_{zk}) .

Proof of Theorem 8.1. Let IOR_1 be Construction 6.3 and let IOR_2 be Construction 7.2 instantiated on $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \mathbf{sl}'}$ with $n + k$ mask oracles. Construction 8.2 is the concatenation $\text{IOR}_2 \circ \text{IOR}_1$.

The stated round complexity, prover communication, verifier queries, prover/verifier time, and round-by-round errors are obtained by combining the corresponding bounds from Theorems 6.2 and 7.1. We now establish HVZK:

- By Theorem 7.1, IOR_2 has a simulator $\mathbf{S}_2$ with error $\epsilon_2 := \zeta_{\mathcal{C}^{\equiv 2^k}} + (n + k) \cdot \zeta_{\mathcal{C}_{zk}}$ and query complexity $(t, (t_{zk})_{i \in [n+k]})$. In the concatenated protocol, the first-oracle queries handled through $\mathbf{S}_2$ are induced by queries to the original oracle f , which is a codeword of $\mathcal{C}^{\equiv 2^k}$.
- Let $\mathbf{t} := (t, (t_{zk})_{i \in [n]})$, and define $\mathcal{D}_{\mathbf{S}_2}$ as in Definition 4.4, i.e., $\mathcal{D}_{\mathbf{S}_2}^y(\cdot) := \mathbf{S}_2^y(\cdot)$. By the query bound of $\mathbf{S}_2$, $\mathcal{D}_{\mathbf{S}_2}$ makes at most t queries to the first oracle of $\mathbf{y}'$, at most t_{zk} queries to each of the next n mask oracles, and at most t_{zk} queries to each of the k new mask oracles. Hence $\mathcal{D}_{\mathbf{S}_2} \in \mathcal{D}^{\leq (t, (t_{zk})_{j \in [k]})}$. By Theorem 6.2, IOR_1 is HVZK for this distinguisher via a simulator $\mathbf{S}_1$ with error $\epsilon_1 := k \cdot \zeta_{\mathcal{C}_{zk}}$ and query complexity $(t, (t_{zk})_{i \in [n]})$.
- Apply Theorem 4.5 with $\mathcal{R}_{\text{start}} := \mathcal{R}_{\mathcal{C}^{\equiv 2^k}, \mathcal{C}_{zk}, \mathbf{sl}}, \mathcal{R}_{\text{int}} := \mathcal{R}_{\mathcal{C}, \mathcal{C}_{zk}, \mathbf{sl}'}, \mathcal{R}_{\text{fin}} := \mathcal{R}_{\text{triv}}$. The composed protocol is HVZK with query complexity $(t, (t_{zk})_{i \in [n]})$ and error at most $\epsilon_1 + \epsilon_2 = \zeta_{\mathcal{C}^{\equiv 2^k}} + (n + 2k) \cdot \zeta_{\mathcal{C}_{zk}}$. By Claim 3.23, $\zeta_{\mathcal{C}^{\equiv 2^k}} \leq 2^k \cdot \zeta_{\mathcal{C}}$, and therefore

$$\epsilon_1 + \epsilon_2 \leq 2^k \cdot \zeta_{\mathcal{C}} + (n + 2k) \cdot \zeta_{\mathcal{C}_{zk}}.$$

In sum Construction 8.2 is an IOPP with HVZK simulation error $2^k \cdot \zeta_{\mathcal{C}} + (n + 2k) \cdot \zeta_{\mathcal{C}_{zk}}$. $\square$

9 Zero-knowledge code-switching

We introduce an honest-verifier zero-knowledge code-switching protocol. To do so, we first introduce the variant of out-of-domain samples that we use in Section 9.1, the notion of private zero-evaders in Section 9.2, and a measure of the code-switching efficiency in Section 9.3.

9.1 Out-of-domain samples

We refine the notion of out-of-domain samples for a linear code (see [BGKS20] and their later uses in [ACFY24; ACFY25; BCFW25]) to account for zero-knowledge encodings. Specifically, in this paper we need to ensure that the “disagreement” is on the message part of the zero-knowledge code and not (only) the randomness part.

Lemma 9.1. *Let $\mathcal{C} \subseteq \Sigma^m$ be an $\mathbb{F}$ -additive code with zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$, $f \in \Sigma^m$ be a word, and $\delta \in [0, 1]$ be a distance parameter. Let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t \times \ell}$ be a zero-evader with error $\varepsilon_{\text{zero}}$ (Definition 3.3). Then,*

$$\Pr_{\rho \leftarrow D_{\text{ze}}} \left[\begin{array}{l} \exists u, v \in \Lambda(\mathcal{C}, f, \delta) \text{ s.t.} \\ \mathbf{u} \neq \mathbf{v} \wedge \text{ze}(\rho) \cdot \mathbf{u} = \text{ze}(\rho) \cdot \mathbf{v} \\ \text{where } (\mathbf{u}, \mathbf{r}) := \text{Enc}_{\mathcal{C}}^{-1}(u), (\mathbf{v}, \mathbf{r}') := \text{Enc}_{\mathcal{C}}^{-1}(v) \end{array} \right] \leq \frac{|\Lambda(\mathcal{C}, \delta)|^2}{2} \cdot \varepsilon_{\text{zero}}.$$

Proof. Let $u, v \in \Lambda(\mathcal{C}, f, \delta)$ be distinct codewords and $(\mathbf{u}, \mathbf{r}) := \text{Enc}_{\mathcal{C}}^{-1}(u)$ and $(\mathbf{v}, \mathbf{r}') := \text{Enc}_{\mathcal{C}}^{-1}(v)$. We consider two cases: (i) if $\mathbf{u} = \mathbf{v}$, we are done; (ii) if $\mathbf{u} \neq \mathbf{v}$, since ze is a zero-evader $\text{ze}(\rho) \cdot \mathbf{u} = \text{ze}(\rho) \cdot \mathbf{v}$ with probability at most $\varepsilon_{\text{zero}}$. The bound follows by taking a union bound over the at most $\binom{|\Lambda(\mathcal{C}, \delta)|}{2}$ distinct pairs of codewords in $\Lambda(\mathcal{C}, f, \delta)$. $\square$

9.2 Private zero-evaders

We consider zero-evaders with a *privacy guarantee*: the inner product with a vector consisting of a private portion and a random portion reveals little or no information about the private portion.

Definition 9.2. *A zero-evader $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t \times \ell}$ is (r, ζ_{ze}) -private if there exists a simulator $\mathbf{S}_{\text{ze}}$ such that for every $\mathbf{f} \in \mathbb{F}^{\ell-r}$ the following two distributions have statistical distance at most ζ_{ze} :*

$$\left\{ (\rho, \mathbf{y}) \mid \begin{array}{l} \rho \leftarrow D_{\text{ze}}, \mathbf{r} \leftarrow \mathbb{F}^r \\ \mathbf{y} = \text{ze}(\rho) \cdot \begin{bmatrix} \mathbf{f} \\ \mathbf{r} \end{bmatrix} \end{array} \right\} \text{ and } \mathbf{S}_{\text{ze}}.$$

By considering the all-zero zero-evader, one can see that there are zero-evaders with good privacy guarantees but trivial zero-evading error. In fact any zero-evader can be modified to have *perfect* privacy guarantees while essentially preserving the zero-evading error.

Lemma 9.3. *Let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{t \times \ell}$ be a zero-evader with error $\varepsilon_{\text{zero}}$. For every $r \in \mathbb{N}$ with $r \geq t$, there exists a zero-evader $\text{ze}': D'_{\text{ze}} \rightarrow \mathbb{F}^{t \times (\ell+r)}$ with error $\max\{\varepsilon_{\text{zero}}, \frac{1}{|\mathbb{F}|^t - 1}\}$ that is $(r, 0)$ -private.*

Proof. Let

$$\mathcal{M}_{t,r} := \left\{ \mathbf{M} \in \mathbb{F}^{t \times r} : \text{the columns of } \mathbf{M} \text{ span } \mathbb{F}^t \right\}.$$

Define $D'_{\text{ze}} := D_{\text{ze}} \times \mathcal{M}_{t,r}$ and

$$\text{ze}'(\rho, \mathbf{M}) := [\text{ze}(\rho), \mathbf{M}].$$

Privacy error. Fix any $\mathbf{f} \in \mathbb{F}^\ell$. Let $\mathbf{S}_{\text{ze}}$ be the simulator that samples $(\rho, \mathbf{M}) \leftarrow D'_{\text{ze}}$ and $\mathbf{r} \leftarrow \mathbb{F}^r$. Let $\mathbf{t}_\rho := \text{ze}(\rho) \cdot \mathbf{f} \in \mathbb{F}^t$. The sample from the real distribution is

$$((\rho, \mathbf{M}), \mathbf{y}) \quad \text{where} \quad \mathbf{y} = \text{ze}'(\rho, \mathbf{M}) \cdot \begin{bmatrix} \mathbf{f} \\ \mathbf{r} \end{bmatrix} = \mathbf{t}_\rho + \mathbf{M}\mathbf{r}.$$

Since $\mathbf{M} \in \mathcal{M}_{t,r}$ has columns spanning $\mathbb{F}^t$, the linear map $\mathbf{r} \mapsto \mathbf{M}\mathbf{r}$ is surjective onto $\mathbb{F}^t$. Therefore, when $\mathbf{r}$ is uniform in $\mathbb{F}^r$, the value $\mathbf{M}\mathbf{r}$ is uniform in $\mathbb{F}^t$, and hence $\mathbf{y} = \mathbf{t}_\rho + \mathbf{M}\mathbf{r}$ is uniform in $\mathbb{F}^t$, even conditioned on $(\rho, \mathbf{M})$. Thus the joint distribution of $((\rho, \mathbf{M}), \mathbf{y})$ is identical to the distribution produced by the simulator.

Zero-evader error. Let $\mathbf{v} \neq \mathbf{0}$ and write $\mathbf{v} = (\mathbf{s}, \mathbf{r})^\top$ for $\mathbf{s} \in \mathbb{F}^\ell$ and $\mathbf{r} \in \mathbb{F}^r$. Note that $\text{ze}'(\rho, \mathbf{M}) \cdot \mathbf{v} = \text{ze}(\rho) \cdot \mathbf{s} + \mathbf{M} \cdot \mathbf{r}$. We consider two cases.

- If $\mathbf{r} = \mathbf{0}$ then $\mathbf{s} \neq \mathbf{0}$ and then

$$\Pr[\text{ze}'(\rho, \mathbf{M}) \cdot \mathbf{v} = \mathbf{0}] = \Pr[\text{ze}(\rho) \cdot \mathbf{s} = \mathbf{0}] \leq \varepsilon_{\text{zero}}.$$

- If $\mathbf{r} \neq \mathbf{0}$, write $\mathbf{t}_\rho = \text{ze}(\rho) \cdot \mathbf{s}$, in which case

$$\Pr[\text{ze}'(\rho, \mathbf{M}) \cdot \mathbf{v} = \mathbf{0}] = \Pr[\mathbf{M} \cdot \mathbf{r} = \mathbf{t}_\rho].$$

Since $\mathbf{r} \neq \mathbf{0}$, there exists an invertible matrix $\mathbf{R} \in \mathbb{F}^{r \times r}$ such that $\mathbf{R} \cdot \mathbf{r} = \mathbf{e}_1$. Right-multiplication by $\mathbf{R}$ preserves the property that the columns span $\mathbb{F}^t$, hence the map $\mathbf{M} \mapsto \mathbf{M}\mathbf{R}$ is a bijection on $\mathcal{M}_{t,r}$ and therefore

$$\Pr[\mathbf{M}\mathbf{r} = \mathbf{t}_\rho] = \Pr[\mathbf{M}\mathbf{e}_1 = \mathbf{t}_\rho].$$

Denote by $\mathbf{m}_1$ the first column of $\mathbf{M}$, so $\mathbf{M}\mathbf{e}_1 = \mathbf{m}_1$. We upper bound $\Pr[\mathbf{m}_1 = \mathbf{t}_\rho]$.

If $\mathbf{t}_\rho \neq \mathbf{0}$, then for every $\mathbf{u} \in \mathbb{F}^t \setminus \{\mathbf{0}\}$ the probability $\Pr[\mathbf{m}_1 = \mathbf{u}]$ is the same by left-invariance: for any invertible $\mathbf{L} \in \mathbb{F}^{t \times t}$, the map $\mathbf{M} \mapsto \mathbf{L}\mathbf{M}$ is a bijection on $\mathcal{M}_{t,r}$ and sends $\mathbf{m}_1$ to $\mathbf{L} \cdot \mathbf{m}_1$. Hence $\Pr[\mathbf{m}_1 = \mathbf{u}]$ is constant over all nonzero $\mathbf{u}$, and therefore

$$\Pr[\mathbf{m}_1 = \mathbf{t}_\rho] \leq \frac{1}{|\mathbb{F}|^t - 1}.$$

If $\mathbf{t}_\rho = \mathbf{0}$, then

$$\Pr_{\mathbf{M} \leftarrow \mathcal{M}_{t,r}}[\mathbf{m}_1 = \mathbf{0}] = \frac{|\mathcal{M}_{t,r-1}|}{|\mathcal{M}_{t,r}|} = \prod_{i=0}^{t-1} \frac{|\mathbb{F}|^{r-1} - |\mathbb{F}|^i}{|\mathbb{F}|^r - |\mathbb{F}|^i} \leq |\mathbb{F}|^{-t},$$

where the equality follows by observing that fixing $\mathbf{m}_1 = \mathbf{0}$ leaves a uniformly random matrix in $\mathbb{F}^{t \times (r-1)}$ whose columns span $\mathbb{F}^t$, and the inequality holds since for all i , $\frac{|\mathbb{F}|^{r-1} - |\mathbb{F}|^i}{|\mathbb{F}|^r - |\mathbb{F}|^i} \leq \frac{|\mathbb{F}|^{r-1}}{|\mathbb{F}|^r} = |\mathbb{F}|^{-1}$.

Combining the two subcases, for all $\mathbf{t}_\rho \in \mathbb{F}^t$ we obtain

$$\Pr_{\mathbf{M} \leftarrow \mathcal{M}_{t,r}}[\mathbf{M}\mathbf{r} = \mathbf{t}_\rho] \leq \max \left\{ \frac{1}{|\mathbb{F}|^t - 1}, |\mathbb{F}|^{-t} \right\} = \frac{1}{|\mathbb{F}|^t - 1},$$

and hence, for $\mathbf{r} \neq \mathbf{0}$,

$$\Pr[\text{ze}'(\rho, \mathbf{M}) \cdot \mathbf{v} = \mathbf{0}] \leq \frac{1}{|\mathbb{F}|^t - 1}.$$

□

9.3 Code-switching complexity

We define succinct linear forms induced by the generating matrices of (the encoding map of) the code. The complexity of computing such succinct linear forms will directly impact the prover and verifier time of the code-switch protocol that we design.

Definition 9.4. Let $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\ell)^m$ be an $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$ and generating matrix $\mathbf{G}_{\mathcal{C}} = \begin{bmatrix} \mathbf{G}_{\mathcal{C}}^{\#} & \mathbf{G}_{\mathcal{C}}^{\$} \end{bmatrix}$. We define two linear forms:

- $\text{sl}_{\mathcal{C}}^{\#}$ takes as input a state $x \in [m \cdot \iota]$ and outputs $\mathbf{G}_{\mathcal{C}}^{\#}[x, \cdot] \in \mathbb{F}^\ell$ ($\mathbf{G}_{\mathcal{C}}^{\#}$'s x -th row);
- $\text{sl}_{\mathcal{C}}^{\$}$ takes as input a state $x \in [m \cdot \iota]$ and outputs $\mathbf{G}_{\mathcal{C}}^{\$}[x, \cdot] \in \mathbb{F}^r$ ($\mathbf{G}_{\mathcal{C}}^{\$}$'s x -th row).

(A minor abuse of notation for simplicity: $\text{sl}_{\mathcal{C}}^{\#}, \text{sl}_{\mathcal{C}}^{\$}$ actually depend on $\text{Enc}_{\mathcal{C}}$ rather than $\mathcal{C}$.)

In our protocols the prover time will depend on $t_{\text{sl}_{\mathcal{C}}^{\#}}$ and $t_{\text{sl}_{\mathcal{C}}^{\$}}$, and the verifier time will depend on $t_{\odot_k(\text{sl}_{\mathcal{C}}^{\#})}$ and $t_{\text{sl}_{\mathcal{C}}^{\$}}$ for some $k \in \mathbb{N}$.

For some linear codes such as Reed–Solomon codes over explicit domains, the prover costs can be linear $O(\ell + \log m + r)$ while the verifier costs can be logarithmic $O(\log m + r)$ (this fact is implicit in [ACFY25; NA25] and explicit in [BFRW25]).

Lemma 9.5. Let $\mathcal{C} := \text{RS}[\mathbb{F}, \mathcal{L}, \ell + r]$ be a Reed–Solomon code with $|\mathcal{L}| = m$ and such that there exists a bijection $\varphi: [m] \rightarrow \mathcal{L}$ computable in $O(\log m)$ field operations.² There exists a r -query perfect zero-knowledge encoding for $\mathcal{C}$ such that: (i) if ℓ is a power-of-two, for every $i \in [\log \ell]$, $t_{\odot_i(\text{sl}_{\mathcal{C}}^{\#})} = O(\ell/2^i + i + \log m)$; (ii) $t_{\text{sl}_{\mathcal{C}}^{\$}} = O(r + \log m)$.

Proof. Consider the encoding map $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \mathcal{C}$ that takes as input $(\mathbf{f}, \mathbf{r})$, interprets them as coefficients of polynomials $\hat{\mathbf{f}}, \hat{\mathbf{r}}$ and outputs $(\hat{\mathbf{f}} + X^\ell \cdot \hat{\mathbf{r}})(\mathcal{L})$. From prior work (e.g. [BCL22]), $\text{Enc}_{\mathcal{C}}$ is a r -query perfect zero-knowledge encoding for $\mathcal{C}$. The encoding matrix of $\text{Enc}_{\mathcal{C}}$ is $\mathbf{G}_{\mathcal{C}} = [\mathbf{G}_{\mathcal{C}}^{\#}, \mathbf{G}_{\mathcal{C}}^{\$}] = [\varphi(x)^{j-1}]_{x \in [m], j \in [\ell+r]}$.

Assume that ℓ is a power of two, $x \in [m]$, $\gamma \in \mathbb{F}^i$, and consider $\odot_i \text{sl}_{\mathcal{C}}^{\#}(x, \gamma)$. By definition, for every $\mathbf{c} \in \{0, 1\}^{\log \ell - i}$,

$$\begin{aligned} (\odot_i \text{sl}_{\mathcal{C}}^{\#}(x, \gamma))[\mathbf{c}] &= \sum_{\mathbf{b} \in \{0, 1\}^i} \text{eq}(\mathbf{b}, \gamma) \cdot \mathbf{G}_{\mathcal{C}}^{\#}[x, \mathbf{b}, \mathbf{c}] \\ &= \sum_{\mathbf{b} \in \{0, 1\}^i} \text{eq}(\mathbf{b}, \gamma) \cdot \varphi(x)^{\sum_{t \in [i]} b_t \cdot 2^{t-1} + \sum_{t \in [\log \ell - i]} c_t \cdot 2^{i+t-1}} \\ &= \sum_{\mathbf{b} \in \{0, 1\}^i} \text{eq}(\mathbf{b}, \gamma) \cdot \prod_{t \in [i]} \varphi(x)^{b_t \cdot 2^{t-1}} \cdot \prod_{t \in [\log \ell - i]} \varphi(x)^{c_t \cdot 2^{i+t-1}} \\ &= \left(\prod_{t \in [\log \ell - i]} \varphi(x)^{c_t \cdot 2^{i+t-1}} \right) \cdot \left(\sum_{\mathbf{b} \in \{0, 1\}^i} \text{eq}(\mathbf{b}, \gamma) \cdot \prod_{t \in [i]} ((1 - b_t) \cdot 1 + b_t \cdot \varphi(x)^{2^{t-1}}) \right) \\ &= \left(\prod_{t \in [\log \ell - i]} \varphi(x)^{c_t \cdot 2^{i+t-1}} \right) \cdot \prod_{t \in [i]} (1 - \gamma_t + \gamma_t \cdot \varphi(x)^{2^{t-1}}). \end{aligned}$$

Where we have used the fact that one of the quantities was multilinear in $\mathbf{b}$ and $\sum_{\mathbf{b}} \text{eq}(\mathbf{b}, \gamma) \cdot \hat{p}(\mathbf{b}) = \hat{p}(\gamma)$ for any multilinear polynomial $\hat{p}$. This yields an algorithm to compute $\odot_i \text{sl}_{\mathcal{C}}^{\#}(x, \gamma)$:

²This class includes many commonly used Reed–Solomon codes over finite field, but excludes (for example) Reed–Solomon codes where $\mathcal{L}$ is a random subset of the field.

1. Compute $\varphi(x)$ in $O(\log m)$ field operations.
2. Compute $\prod_{t \in [i]} (1 - \gamma_t + \gamma_t \cdot \varphi(x)^{2^{t-1}})$ in $O(i)$ field operations.
3. Compute $\left(\prod_{t \in [\log \ell - i]} \varphi(x)^{c_t \cdot 2^{i+t-1}} \right)_{c \in \{0,1\}^{\log \ell - i}}$ in $O(\ell/2^i)$ field operations using a dynamic programming algorithm (e.g., [VSBW13; BCFFMMZ25]).
4. Compute the final expression in $O(\ell/2^i)$ field operations.

To conclude the lemma, one can see that for every $x \in [m]$, $\mathbf{G}_C^s[x, \cdot] = [\varphi(x)^{\ell+i-1}]_{j \in [r]}$ can be computed in $O(\log m + r)$ field operations. $\square$

9.4 Code-switching IOR

We describe an IOR from $\mathcal{R}_{C, C_{zk}, \mathbf{sl}}$ to $\mathcal{R}_{C', C_{zk}, \mathbf{sl}'}$.

Theorem 9.6. *Let:*

- $n \in \mathbb{N}$ be a number of functions.
- $t, t_{\text{ood}} \in \mathbb{N}$ be repetition parameters.
- $C \subseteq \Sigma^m \equiv (\mathbb{F}^\ell)^m$ be an $\mathbb{F}$ -additive code with zero-knowledge encoding $\text{Enc}_C: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$.
- $C' \subseteq (\Sigma')^{m'} \equiv (\mathbb{F}^{\ell'})^{m'}$ be a $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{C'}: \mathbb{F}^\ell \times \mathbb{F}^{r'} \rightarrow (\Sigma')^{m'}$.
- $C_{zk} \subseteq \Sigma_{zk}^{m_{zk}} \equiv (\mathbb{F}^{\ell_{zk}})^{m_{zk}}$ be a $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{C_{zk}}: \mathbb{F}^{\ell_{zk}} \times \mathbb{F}^{r_{zk}} \rightarrow \Sigma_{zk}^{m_{zk}}$.
- $\mathbf{sl} = (\mathbf{sl}, (\mathbf{sl}_i)_{i \in [n]}, (\mathbf{sl}_{o,i})_{i \in [n]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i \in \langle \mathbb{F}^{\ell_{zk}} \rangle$, $\mathbf{sl}_{o,i} \in \langle \mathbb{F}^{t_i \times \ell_{zk}} \rangle$.
- Let $\mathbf{ze}: D_{\mathbf{ze}} \rightarrow \mathbb{F}^{1+t_{\text{ood}}+t \cdot \iota}$ be a zero-evader with error $\varepsilon_{\text{zero}}$ and let $\mathbf{ze}_{\leftrightarrow}: D_{\mathbf{ze}} \rightarrow \mathbb{F}^{t_{\text{ood}}}$ denote the first t_{ood} coefficients of $\mathbf{ze}$ (skipping the first one).
- Let $\mathbf{ze}_{\text{ood}}: D_{\text{ood}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times (\ell + \ell_{zk})}$ be a zero-evader with error ε_{ood} . Write $\mathbf{ze}_{\text{ood}} = (\mathbf{ze}_{\text{ood}}^{\leftarrow}, \mathbf{ze}_{\text{ood}}^{\rightarrow})$ where $\mathbf{ze}_{\text{ood}}^{\leftarrow}: D_{\text{ood}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times \ell}$ and $\mathbf{ze}_{\text{ood}}^{\rightarrow}: D_{\text{ood}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times \ell_{zk}}$ denote the first and last components of $\mathbf{ze}_{\text{ood}}$. For every $i \in [t_{\text{ood}}]$ denote by $\mathbf{ze}_{\text{ood}}^{\leftarrow, i}$ (resp. $\mathbf{ze}_{\text{ood}}^{\rightarrow, i}$) the zero-evader obtained by restricting $\mathbf{ze}_{\text{ood}}^{\leftarrow}$ (resp. $\mathbf{ze}_{\text{ood}}^{\rightarrow}$) to the i -th row.

Define

$$\begin{aligned} \mathbf{sl}' &:= \mathbf{ze}[\times(\mathbf{sl}), (\mathbf{ze}_{\text{ood}}^{\leftarrow, i})_{i \in [t_{\text{ood}}]}, (\mathbf{sl}_C^\#)_{i \in [t], l \in [\iota]}] \\ \mathbf{sl}'_{n+1} &:= \mathbf{ze}_{\leftrightarrow}[(\mathbf{sl}_{\text{ze}_{\text{ood}}^{\rightarrow, i}})_{i \in [t_{\text{ood}}]}, (0^{\ell_{zk}-r}(\mathbf{sl}_C^\#))_{i \in [t], l \in [\iota]}] \\ \mathbf{sl}' &:= \left(\mathbf{sl}', (\times(\mathbf{sl}_i))_{i \in [n]}, \mathbf{sl}'_{n+1}, (\mathbf{sl}_{o,i})_{i \in [n]}, \mathbf{sl}_{\text{id}} \right). \end{aligned}$$

Construction 9.7 is an IOR from $\mathcal{R}_{C, C_{zk}, \mathbf{sl}}$ to $\mathcal{R}_{C', C_{zk}, \mathbf{sl}'}$ with the following properties.

- Round complexity: 2.
- Prover communication (in field elements): $m' \cdot \iota' + m_{zk} \cdot \iota_{zk} + t_{\text{ood}}$. Of these,
 - $m' \cdot \iota'$ are sent as an oracle over Σ' ,
 - $m_{zk} \cdot \iota_{zk}$ are sent as an oracle over Σ_{zk} , and
 - t_{ood} are sent as a non-oracle message.
- Query complexity: t queries to the first input oracle.
- Prover time (in field operations): $O(\text{tenc}(C') + \text{tenc}(C_{zk}) + \text{teval}(\mathbf{ze}) + \text{teval}(\mathbf{ze}_{\text{ood}}) + \sum_{i \in [n]} t_{\text{sl}_i})$.
- Verifier time (in field operations): $O(\text{teval}(\mathbf{ze}) + t_{\text{ood}})$.

- **Round-by-round security:** For every $\delta, \delta', \delta_{\text{zk}} \in (0, 1)$, Construction 9.7 has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}}, \tilde{\mathcal{R}}_{\mathcal{C}', \mathcal{C}_{\text{zk}}, \mathbf{sl}'}^{\delta', \delta_{\text{zk}}})$ and errors

$$\left(\frac{(|\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\text{zk}}, \delta_{\text{zk}})|)^2}{2} \cdot \varepsilon_{\text{ood}}, (1 - \delta)^t, |\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+1}, \delta_{\text{zk}})| \cdot \varepsilon_{\text{zero}} \right).$$

The total extraction time is $O(\text{tenc}(\mathcal{C}))$.

- **Zero-knowledge:** If ze_{ood} is a $(\ell_{\text{zk}} - r, \zeta_{\text{ze}})$ -private zero-evader, $\mathcal{C}'$ has a t' -query zero-knowledge encoding with error $\zeta_{\mathcal{C}'}$, and $\mathcal{C}_{\text{zk}}$ has a t_{zk} -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_{\text{zk}}}$, then, for every $\mathbf{t} \in \mathbb{N}^n$, Construction 9.7 is HVZK for $\mathcal{D}^{\leq(t', \mathbf{t}, t_{\text{zk}})}$ with error $\zeta_{\mathcal{C}'} + \zeta_{\text{ze}} + \zeta_{\mathcal{C}_{\text{zk}}}$ and query complexity $(t, \mathbf{t})$.

Construction 9.7.

- **Inputs.** The verifier receives explicit input $\mathbf{x} = ((\mu, \mathbf{st}, (\mathbf{st}_i)_{i \in [n]}), ((\boldsymbol{\mu}_i)_{i \in [n]}, (\mathbf{st}_{\text{o}, i})_{i \in [n]}))$ and oracle access to implicit input $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$. In the honest case, the prover receives as input $\mathbf{x}$ and $\mathbf{y}$ as well as witness $\mathbf{w} = (\mathbf{f}, \mathbf{r}, (\xi_i, \mathbf{r}_i)_{i \in [n]})$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$.

- **Interaction phase.**

1. **Send witness.** The prover sends $g \in (\Sigma')^{m'}$ and $s \in \Sigma_{\text{zk}}^{m_{\text{zk}}}$. In the honest case, the prover samples $\mathbf{r}' \leftarrow \mathbb{F}^{r'}$, $\mathbf{s} \leftarrow \mathbb{F}^{\ell_{\text{zk}} - r}$, $\mathbf{r}'' \leftarrow \mathbb{F}^{r_{\text{zk}}}$ and computes $g := \text{Enc}_{\mathcal{C}'}(\mathbf{f}, \mathbf{r}')$ and $s := \text{Enc}_{\mathcal{C}_{\text{zk}}}((\mathbf{r}, \mathbf{s}), \mathbf{r}'')$.
2. **Out-of-domain samples.** The verifier samples and sends $\rho_{\text{ood}} \leftarrow D_{\text{ood}}$.
3. **Out-of-domain answers.** The prover sends $\mathbf{y} \in \mathbb{F}^{t_{\text{ood}}}$. In the honest case the prover computes

$$\mathbf{y} := \text{ze}_{\text{ood}}(\rho_{\text{ood}}) \cdot \begin{bmatrix} \mathbf{f} \\ \mathbf{r} \\ \mathbf{s} \end{bmatrix}.$$

4. **In-domain queries and batching.** The verifier samples and sends $x_1, \dots, x_t \leftarrow [m]$ and $\rho \leftarrow D_{\text{ze}}$. Set $\boldsymbol{\nu} := \text{ze}(\rho)$ and for $i \in [t]$, let $\mathbf{x}_i := \text{binary}(x_i)$.
- **Decision phase.** The verifier queries $f(x_1), \dots, f(x_t) \in \Sigma$, and computes

$$\mu' := \boldsymbol{\nu}_1 \cdot \mu + \sum_{i \in [t_{\text{ood}}]} \boldsymbol{\nu}_{1+i} \cdot y_i + \sum_{i \in [t]} \sum_{l \in [\iota]} \boldsymbol{\nu}_{1+t_{\text{ood}}+i \cdot \iota + l} \cdot f(x_i)_l.$$

The verifier further sets $\mathbf{st}' := \mathbf{st}'_{n+1} := (\rho, \mathbf{st}, (\rho_{\text{ood}})_{i \in [t_{\text{ood}}]}, (x_{1,l}, \dots, x_{t,l})_{l \in [\iota]}),$ where $x_{j,l} := \iota \cdot x_j + l$. The verifier outputs the following new explicit instance and implicit instance and in the honest case the prover outputs the following new witness:

$$\begin{aligned} \mathbf{x}' &:= ((\mu', \mathbf{st}', (\boldsymbol{\nu}_1, \mathbf{st}_1), \dots, (\boldsymbol{\nu}_1, \mathbf{st}_n), \mathbf{sl}'_{n+1}), ((\boldsymbol{\mu}_i)_{i \in [n]}, (\mathbf{st}_{\text{o}, i})_{i \in [n]}), (0, 0^{\ell_{\text{zk}}})) , \\ \mathbf{y}' &:= (g, (\xi_i)_{i \in [n]}, s), \\ \mathbf{w}' &:= ((\mathbf{f}, \mathbf{r}'), (\xi_i, \mathbf{r}_i)_{i \in [n]}, ((\mathbf{r}, \mathbf{s}), \mathbf{r}'')). \end{aligned} \tag{9}$$

Completeness. We show that the construction is perfectly complete. In an honest execution

$$\begin{aligned}\mu &= \langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle \\ \forall i \in [t_{\text{ood}}] : y_i &= \langle (\mathbf{f}, \mathbf{r}, \mathbf{s}), \text{ze}_{\text{ood}}(\rho_{\text{ood}})[i] \rangle = \langle \mathbf{f}, \text{ze}_{\text{ood}}^{\leftarrow, i}(\rho_{\text{ood}}) \rangle + \langle (\mathbf{r}, \mathbf{s}), \text{ze}_{\text{ood}}^{\rightarrow, i}(\rho_{\text{ood}}) \rangle \\ \forall i \in [t], \forall l \in [l] : f(x_i)_l &= \langle \mathbf{f}, \mathbf{G}_{\mathcal{C}}^{\#}[x_{i,l}, \cdot] \rangle + \langle \mathbf{r}, \mathbf{G}_{\mathcal{C}}^{\mathbf{s}}[x_{i,l}, \cdot] \rangle \\ &= \langle \mathbf{f}, \mathbf{G}_{\mathcal{C}}^{\#}[x_{i,l}, \cdot] \rangle + \langle (\mathbf{r}, \mathbf{s}), (\mathbf{G}_{\mathcal{C}}^{\mathbf{s}}[x_{i,l}, \cdot], \mathbf{0}) \rangle.\end{aligned}$$

By linearity then,

$$\begin{aligned}\mu' &= \left\langle \mathbf{f}, \nu_1 \cdot \text{sl}(\text{st}) + \sum_{i \in [t_{\text{ood}}]} \nu_{1+i} \cdot \text{ze}_{\text{ood}}^{\leftarrow, i}(\rho_{\text{ood}}) + \sum_{i \in [t]} \sum_{l \in [l]} \nu_{1+t_{\text{ood}}+i \cdot l + l} \cdot \mathbf{G}_{\mathcal{C}}^{\#}[x_{i,l}, \cdot] \right\rangle \\ &\quad + \sum_{i \in [n]} \langle \xi_i, \nu_1 \cdot \text{sl}_i(\text{st}_i) \rangle \\ &\quad + \left\langle (\mathbf{r}, \mathbf{s}), \sum_{i \in [t_{\text{ood}}]} \nu_{1+i} \cdot \text{ze}_{\text{ood}}^{\rightarrow, i}(\rho_{\text{ood}}) + \sum_{i \in [t]} \sum_{l \in [l]} \nu_{1+t_{\text{ood}}+i \cdot l + l} \cdot (\mathbf{G}_{\mathcal{C}}^{\mathbf{s}}[x_{i,l}, \cdot], \mathbf{0}) \right\rangle.\end{aligned}$$

9.5 Zero-knowledge

Lemma 9.8. *If ze_{ood} is a $(\ell_{\text{zk}} - r, \zeta_{\text{ze}})$ -private zero-evader, $\mathcal{C}'$ has a t' -query zero-knowledge encoding with error $\zeta_{\mathcal{C}'}$, and $\mathcal{C}_{\text{zk}}$ has a t_{zk} -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_{\text{zk}}}$, then, for every $\mathbf{t} \in \mathbb{N}^n$, Construction 9.7 is HVZK for $\mathfrak{D}^{\leq(t', \mathbf{t}, t_{\text{zk}})}$ with error $\zeta_{\mathcal{C}'} + \zeta_{\text{ze}} + \zeta_{\mathcal{C}_{\text{zk}}}$ and query complexity $(t, \mathbf{t})$.*

Proof. We describe the simulator $\mathbf{S}$ for Construction 9.7.

$\mathbf{S}^{\mathbf{y}, \mathcal{D}}(\mathbf{x})$:

1. Parse $\mathbf{y}$ as $\mathbf{y} = (f, (\xi_i)_{i \in [n]})$.
2. Sample $x_1, \dots, x_t \leftarrow [m]$ and $\rho \leftarrow D_{\text{ze}}$.
3. Sample $(\rho_{\text{ood}}, \mathbf{y}) \leftarrow \mathbf{S}_{\text{ze}_{\text{ood}}}$.
4. Query $f(x_1), \dots, f(x_t)$ and derive $\mathbf{x}'$ as in Equation 9.
5. Run $\mathcal{D}$ on $\mathbf{x}'$, answering queries to $\mathbf{y}'$ as follows:
 - When $\mathcal{D}$ queries one of $(\xi_i)_{i \in [n]}$, query the corresponding oracle accordingly (denote these answers as Q_{ξ}).
 - When $\mathcal{D}$ queries g , answer according to $\text{Sim}_{\mathcal{C}'}$ (denote these answers as Q_g).
 - When $\mathcal{D}$ queries s , answer according to $\text{Sim}_{\mathcal{C}_{\text{zk}}}$ (denote these answers as Q_s).
6. Let out denote $\mathcal{D}$'s output.
7. Output $(\rho, (x_1, \dots, x_t), (f(x_1), \dots, f(x_t)), Q_{\xi}, Q_g, Q_s, \mathbf{y}, \text{out})$.

The proof follows by a sequence of hybrids $H_0, \dots, H_3$. The first hybrid H_0 is the real experiment. In the second hybrid H_1 , the distribution of $(\rho_{\text{ood}}, \mathbf{y})$ is replaced to be sampled by $\mathbf{S}_{\text{ze}_{\text{ood}}}$; the statistical distance between H_0 and H_1 is then bounded by ζ_{ze} . In the third hybrid H_2 , the queries to g are replaced by answers according to $\text{Sim}_{\mathcal{C}'}$; the statistical distance between H_1 and H_2 is then bounded by $\zeta_{\mathcal{C}'}$. In the fourth hybrid H_3 , the queries to s are replaced by answers according to $\text{Sim}_{\mathcal{C}_{\text{zk}}}$, and thus the experiment is simulated experiment; the statistical distance between H_2 and H_3 is then bounded by $\zeta_{\mathcal{C}_{\text{zk}}}$. $\square$

9.6 Round-by-round knowledge soundness

Lemma 9.9. *For every $\delta, \delta', \delta_{zk} \in (0, 1)$, Construction 9.7 has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{zk}, \mathbf{sl}}^{\delta, \delta_{zk}}, \tilde{\mathcal{R}}_{\mathcal{C}', \mathcal{C}_{zk}, \mathbf{sl}'}^{\delta', \delta_{zk}})$ and errors*

$$\left(\frac{(|\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{zk}, \delta_{zk})|)^2}{2} \cdot \varepsilon_{\text{ood}}, (1 - \delta)^t, |\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{zk}^{\equiv n+1}, \delta_{zk})| \cdot \varepsilon_{\text{zero}} \right).$$

The total extraction time is $O(\text{tenc}(\mathcal{C}))$.

Proof. We give notation, describe the extractors, describe a state function, and finally we establish the RBR knowledge soundness errors (based on the extractor and state function).

Notation. Let $\mathbb{x} = ((\mu, \text{st}_0, (\text{st}_i)_{i \in [n]}), (\boldsymbol{\mu}_i, \text{st}_{o,i})_{i \in [n]})$ and $\mathbb{y} = (f, (\xi_i)_{i \in [n]})$. A complete transcript of the protocol has the form $\text{tr} = ((g, s), \rho_{\text{ood}}, \mathbf{y}, (x_1, \dots, x_t, \rho))$.

Extractors. We describe the round-by-round extractors from the last round to the first.

1. **Final round.** $\mathbf{E}_{\text{rbr}}(\mathbb{x}, \mathbb{y}, \text{tr} \parallel (x_1, \dots, x_t, \rho), \mathbf{w})$ outputs the witness $\mathbf{w}$.

2. **Out-of-domain samples.** $\mathbf{E}_{\text{rbr}}(\mathbb{x}, \mathbb{y}, \text{tr} \parallel \rho_{\text{ood}}, \mathbf{w})$:

- (a) Parse $\mathbf{w}$ as $((\bar{g}, \mathbf{g}, \mathbf{r}'), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}, \mathbf{s}, \mathbf{r}''))$.
- (b) Parse $\mathbf{s}$ as $(\mathbf{r}, \mathbf{s}') \in \mathbb{F}^r \times \mathbb{F}^{\ell_{zk}-r}$.
- (c) Compute $\bar{f} := \text{Enc}_{\mathcal{C}}(\mathbf{g}, \mathbf{r})$.
- (d) Output $((\bar{f}, \mathbf{g}, \mathbf{r}), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]})$.

3. **Initial round.**

- (a) Parse $\mathbf{w}$ as $((\bar{f}, \mathbf{f}, \mathbf{r}), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]})$.
- (b) Set $\mathbb{w} := ((\mathbf{f}, \mathbf{r}), (\boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]})$.
- (c) Set $\mathbb{y}^* := (\bar{f}, (\bar{\xi}_i)_{i \in [n]})$.
- (d) Output $(\mathbb{w}, \mathbb{y}^*)$.

The state function. We define a state function KState .

0. **Initial transcript:** We set $\text{KState}(\mathbb{x}, \mathbb{y}, \emptyset, \mathbf{w}) = 1$ if and only if $(\mathbb{x}, \mathbb{y}, \mathbf{w}) \in \tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{zk}, \mathbf{sl}}^{\delta, \delta_{zk}}$, that is

$$\mathbf{w} = ((\bar{f}, \mathbf{f}, \mathbf{r}), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]})$$

is such that:

- $\Delta((f, \xi_1, \dots, \xi_n), (\bar{f}, \bar{\xi}_1, \dots, \bar{\xi}_n)) \leq (\delta, \delta_{zk}, \dots, \delta_{zk})$;
- $\bar{f} = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r}), \forall i \in [n] \bar{\xi}_i = \text{Enc}_{\mathcal{C}_{zk}}(\boldsymbol{\xi}_i, \mathbf{r}_i)$;
- $\forall i \in [n] \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \boldsymbol{\xi}_i = \boldsymbol{\mu}_i$;
- $\langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \boldsymbol{\xi}_i, \text{sl}_i(\text{st}_i) \rangle = \mu$.

1. **Out-of-domain samples:** The transcript has the form $\text{tr} = ((g, s))$. The verifier samples $\rho_{\text{ood}} \leftarrow D_{\text{ood}}$. We set $\text{KState}(\mathbb{x}, \mathbb{y}, \text{tr} \parallel \rho_{\text{ood}}, \mathbf{w}) = 1$ if and only if either of the following hold:

- (a) $\mathbf{w} = ((\bar{f}, \mathbf{f}, \mathbf{r}), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]})$ is such that:
- $\Delta((f, \xi_1, \dots, \xi_n), (\bar{f}, \bar{\xi}_1, \dots, \bar{\xi}_n)) \leq (\delta, \delta_{z_k}, \dots, \delta_{z_k})$;
 - $\bar{f} = \text{Enc}_C(\mathbf{f}, \mathbf{r}), \forall i \in [n] \bar{\xi}_i = \text{Enc}_{C_{z_k}}(\boldsymbol{\xi}_i, \mathbf{r}_i)$;
 - $\forall i \in [n] \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \boldsymbol{\xi}_i = \boldsymbol{\mu}_i$;
 - $\langle \mathbf{f}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \boldsymbol{\xi}_i, \text{sl}_i(\text{st}_i) \rangle = \mu$.
- (b) there exist distinct $(\mathbf{g}_0, \mathbf{s}_0) \neq (\mathbf{g}_1, \mathbf{s}_1)$ and $(\mathbf{r}'_0, \mathbf{r}''_0, \mathbf{r}'_1, \mathbf{r}''_1)$ such that $\Delta(g, \text{Enc}_{C'}(\mathbf{g}_0, \mathbf{r}'_0)) \leq \delta'$ and $\Delta(s, \text{Enc}_{C_{z_k}}(\mathbf{s}_0, \mathbf{r}''_0)) \leq \delta_{z_k}$, $\Delta(g, \text{Enc}_{C'}(\mathbf{g}_1, \mathbf{r}'_1)) \leq \delta'$ and $\Delta(s, \text{Enc}_{C_{z_k}}(\mathbf{s}_1, \mathbf{r}''_1)) \leq \delta_{z_k}$ and

$$\mathbf{ze}_{\text{ood}}(\rho_{\text{ood}}) \cdot \begin{bmatrix} \mathbf{g}_0 \\ \mathbf{s}_0 \end{bmatrix} = \mathbf{ze}_{\text{ood}}(\rho_{\text{ood}}) \cdot \begin{bmatrix} \mathbf{g}_1 \\ \mathbf{s}_1 \end{bmatrix}.$$

2. **In-domain samples:** The transcript has the form $\text{tr} = ((g, s), \rho_{\text{ood}}, \mathbf{y})$. The verifier samples $x_1, \dots, x_t \leftarrow [m]$. We set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \| (x_1, \dots, x_t), \mathbf{w}) = 1$ if and only if

$$\mathbf{w} = ((\bar{g}, \mathbf{g}, \mathbf{r}'), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}, \mathbf{s}, \mathbf{r}''))$$

is such that:

- $\Delta((g, \xi_1, \dots, \xi_n, s), (\bar{g}, \bar{\xi}_1, \dots, \bar{\xi}_n, \bar{s})) \leq (\delta', \delta_{z_k}, \dots, \delta_{z_k})$;
- $\bar{g} = \text{Enc}_{C'}(\mathbf{g}, \mathbf{r}'), \forall i \in [n] \bar{\xi}_i = \text{Enc}_{C_{z_k}}(\boldsymbol{\xi}_i, \mathbf{r}_i), \bar{s} = \text{Enc}_{C_{z_k}}(\mathbf{s}, \mathbf{r}'')$;
- $\langle \mathbf{g}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \boldsymbol{\xi}_i, \text{sl}_i(\text{st}_i) \rangle = \mu$;
- $\forall i \in [n], \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \boldsymbol{\xi}_i = \boldsymbol{\mu}_i$;
- $\forall i \in [t_{\text{ood}}], \langle (\mathbf{g}, \mathbf{s}), \mathbf{ze}_{\text{ood}}(\rho_{\text{ood}})[i] \rangle = y_i$;
- $\forall i \in [t], l \in [l], \langle \mathbf{g}, \mathbf{G}_C^\# [x_{i,l}, \cdot] \rangle + \langle \mathbf{s}, (\mathbf{G}_C^\# [x_{i,l}, \cdot], \mathbf{0}) \rangle = f(x_i)_l$.

3. **Combination randomness:** The transcript has the form $\text{tr} = ((g, s), \rho_{\text{ood}}, \mathbf{y}, (x_1, \dots, x_t))$. The verifier samples $\rho \leftarrow D_{\text{ze}}$. We set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \| \rho, \mathbf{w}) = 1$ if and only if

$$\mathbf{w} = ((\bar{g}, \mathbf{g}, \mathbf{r}'), (\bar{\xi}_i, \boldsymbol{\xi}_i, \mathbf{r}_i)_{i \in [n]}, (\bar{s}, \mathbf{s}, \mathbf{r}''))$$

is such that:

- $\Delta((g, \xi_1, \dots, \xi_n, s), (\bar{g}, \bar{\xi}_1, \dots, \bar{\xi}_n, \bar{s})) \leq (\delta', \delta_{z_k}, \dots, \delta_{z_k})$;
- $\bar{g} = \text{Enc}_{C'}(\mathbf{g}, \mathbf{r}'), \forall i \in [n] \bar{\xi}_i = \text{Enc}_{C_{z_k}}(\boldsymbol{\xi}_i, \mathbf{r}_i), \bar{s} = \text{Enc}_{C_{z_k}}(\mathbf{s}, \mathbf{r}'')$;
- $\forall i \in [n] \text{sl}_{o,i}(\text{st}_{o,i}) \cdot \boldsymbol{\xi}_i = \boldsymbol{\mu}_i$;
- the following holds

$$\begin{aligned} \mu' = & \left\langle \mathbf{g}, \boldsymbol{\nu}_1 \cdot \text{sl}(\text{st}) + \sum_{i \in [t_{\text{ood}}]} \boldsymbol{\nu}_{1+i} \cdot \mathbf{ze}_{\text{ood}}^{\leftarrow, i}(\rho_{\text{ood}}) + \sum_{i \in [t]} \sum_{l \in [l]} \boldsymbol{\nu}_{1+t_{\text{ood}}+i \cdot l + l} \cdot \mathbf{G}_C^\# [x_{i,l}, \cdot] \right\rangle \\ & + \boldsymbol{\nu}_1 \cdot \sum_{i \in [n]} \langle \boldsymbol{\xi}_i, \text{sl}_i(\text{st}_i) \rangle \\ & + \left\langle \mathbf{s}, \sum_{i \in [t_{\text{ood}}]} \boldsymbol{\nu}_{1+i} \cdot \mathbf{ze}_{\text{ood}}^{\rightarrow, i}(\rho_{\text{ood}}) + \sum_{i \in [t]} \sum_{l \in [l]} \boldsymbol{\nu}_{1+t_{\text{ood}}+i \cdot l + l} \cdot (\mathbf{G}_C^\# [x_{i,l}, \cdot], \mathbf{0}) \right\rangle. \end{aligned}$$

Bounding the errors. We bound the RBR errors for the protocol.

1. **Out-of-domain samples.** We show that

$$\Pr \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \rho_{\text{ood}}, \mathbf{w}) = 1 \end{array} \right] \leq \frac{(|\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\text{zk}}, \delta_{\text{zk}})|)^2}{2} \cdot \varepsilon_{\text{ood}}.$$

Note that Item 1a is independent of ρ_{ood} , and so it cannot hold. It must then be that Item 1b holds, but the probability of this holding is bound by Lemma 9.1.

2. **In-domain samples.** We show that

$$\Pr \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel (x_1, \dots, x_t), \mathbf{w}) = 1 \end{array} \right] \leq (1 - \delta)^t.$$

Fix $\mathbf{w}$ such that $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel (x_1, \dots, x_t), \mathbf{w}) = 1$ and $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0$. Then, it must be that for $i \in [n]$ we have that $\text{sl}_{\text{o},i}(\text{st}_{\text{o},i}) \cdot \xi_i = \mu_i$ and this must also hold in Item 1a.

Note that $\Delta((g, s), (\bar{g}, \bar{s})) \leq (\delta', \delta_{\text{zk}})$, and so by Item 1b there must be a unique choice of $(\mathbf{g}, \mathbf{s})$ in $\mathbf{w}$ such that for every $i \in [t_{\text{ood}}]$ it holds that $\langle (\mathbf{g}, \mathbf{s}), \mathbf{ze}_{\text{ood}}(\rho_{\text{ood}})[i] \rangle = y_i$ (note also that every condition involving $x_1, \dots, x_t$ only depends on the $(\mathbf{g}, \mathbf{s})$ portion of the witness).

Now, suppose that for this $(\mathbf{g}, \mathbf{r}', \mathbf{s}, \mathbf{r}^*)$ it held that

$$\langle \mathbf{g}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle = \mu,$$

and parse $\mathbf{s} = (\mathbf{r}^*, \mathbf{s}')$. Let $\bar{f} = \text{Enc}_{\mathcal{C}}(\mathbf{g}, \mathbf{r}^*)$. It must be that $\Delta(f, \bar{f}) > \delta$, as else it contradicts the assumption $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0$. But then, the probability that for every $i \in [t], l \in [l]$ it holds that $f(x_i)_l = \bar{f}(x_i)_l = \langle \mathbf{g}, \mathbf{G}_{\mathcal{C}}^{\#}[x_{i,l}, \cdot] \rangle + \langle \mathbf{r}^*, \mathbf{G}_{\mathcal{C}}^{\#}[x_{i,l}, \cdot] \rangle$ is at most $(1 - \delta)^t$.

3. **Combination randomness.** We show that

$$\Pr \left[\exists \mathbf{w} : \begin{array}{l} \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \rho, \mathbf{w}) = 1 \end{array} \right] \leq |\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+1}, \delta_{\text{zk}})| \cdot \varepsilon_{\text{zero}}.$$

Fix $\mathbf{w}$ such that $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbb{X}, \mathbb{Y}, \text{tr}, \mathbf{w})) = 0$. Then, it must be that at least one of the three conditions are not satisfied, namely either:

$$\langle \mathbf{g}, \text{sl}(\text{st}) \rangle + \sum_{i \in [n]} \langle \xi_i, \text{sl}_i(\text{st}_i) \rangle \neq \mu,$$

- or for some $i \in [t_{\text{ood}}]$ it holds that $\langle (\mathbf{g}, \mathbf{s}), \mathbf{ze}_{\text{ood}}(\rho_{\text{ood}})[i] \rangle \neq y_i$.
- or for some $i \in [t], l \in [l]$ it holds that $\langle \mathbf{g}, \mathbf{G}_{\mathcal{C}}^{\#}[x_{i,l}, \cdot] \rangle + \langle \mathbf{s}, (\mathbf{G}_{\mathcal{C}}^{\#}[x_{i,l}, \cdot], \mathbf{0}) \rangle \neq f(x_i)_l$.

Then, the probability that $\text{KState}(\mathbb{X}, \mathbb{Y}, \text{tr} \parallel \rho, \mathbf{w}) = 1$ is at most $\varepsilon_{\text{zero}}$. Taking a union bound over the $|\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\text{zk}}^{\equiv n+1}, \delta_{\text{zk}})|$ possible choices of $\mathbf{w}$ concludes the proof. $\square$

9.7 Proof of Theorem 2

We combine Construction 6.3 and Construction 9.7 to give a formal statement and proof of Theorem 2.

Theorem 9.10. *Let:*

- $n \in \mathbb{N}$;
- $k, t, t_{\text{ood}} \in \mathbb{N}$;
- $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\ell)^m$ be an $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$;
- $\mathcal{C}_{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{zk}}} \equiv (\mathbb{F}^{\ell_{\text{zk}}})^{m_{\text{zk}}}$ be a $\mathbb{F}$ -additive code with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{zk}}}: \mathbb{F}^{\ell_{\text{zk}}} \times \mathbb{F}^{r_{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{zk}}}$;
- $\mathcal{C}' \subseteq (\Sigma')^{m'} \equiv (\mathbb{F}^{\ell'})^{m'}$ be a $\mathbb{F}$ -additive code with zero-knowledge encoding $\text{Enc}_{\mathcal{C}'}: \mathbb{F}^\ell \times \mathbb{F}^{r'_{\text{zk}}} \rightarrow (\Sigma')^{m'}$;
- $\mathbf{sl} = (\mathbf{sl}, (\mathbf{sl}_i)_{i \in [n]}, (\mathbf{sl}_{\text{o},i})_{i \in [n]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i \in \langle \mathbb{F}^{\ell_{\text{zk}}} \rangle$, $\mathbf{sl}_{\text{o},i} \in \langle \mathbb{F}^{t_i \times \ell_{\text{zk}}} \rangle$;
- $\mathbf{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^{1+t_{\text{ood}}+t_{\text{e}}}$ be a zero-evader with error $\varepsilon_{\text{zero}}$, and let $\mathbf{ze}_{\leftrightarrow}: D_{\text{ze}} \rightarrow \mathbb{F}^{t_{\text{ood}}}$ denote the first t_{ood} coefficients of $\mathbf{ze}$ (skipping the first one);
- $\mathbf{ze}_{\text{ood}}: D_{\text{ood}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times (\ell + \ell_{\text{zk}})}$ be a zero-evader with error ε_{ood} , and write $\mathbf{ze}_{\text{ood}} = (\mathbf{ze}_{\text{ood}}^{\leftarrow}, \mathbf{ze}_{\text{ood}}^{\rightarrow})$ with $\mathbf{ze}_{\text{ood}}^{\leftarrow}: D_{\text{ood}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times \ell}$ and $\mathbf{ze}_{\text{ood}}^{\rightarrow}: D_{\text{ood}} \rightarrow \mathbb{F}^{t_{\text{ood}} \times \ell_{\text{zk}}}$. For $i \in [t_{\text{ood}}]$, denote by $\mathbf{ze}_{\text{ood}}^{\leftarrow,i}$ and $\mathbf{ze}_{\text{ood}}^{\rightarrow,i}$ the corresponding row-restricted zero-evaders.

Define

$$\begin{aligned} \mathbf{sl}' &:= \mathbf{ze}[\times(\odot_k \mathbf{sl}), (\mathbf{ze}_{\text{ood}}^{\leftarrow,i})_{i \in [t_{\text{ood}}]}, (\mathbf{sl}_c^{\#})_{i \in [t], l \in [l]}], \\ \mathbf{sl}'_{n+k+1} &:= \mathbf{ze}_{\leftrightarrow}[(\mathbf{sl}_{\text{ze}^{\rightarrow,i}})_{i \in [t_{\text{ood}}]}, (0^{\ell_{\text{zk}}-r}(\mathbf{sl}_c^{\#}))_{i \in [t], l \in [l]}] \\ \mathbf{sl}' &:= \left(\mathbf{sl}', (\times(\mathbf{sl}_i))_{i \in [n]}, (\times(\mathbf{sl}_{\text{ze}^{\star}}))_{j \in [k]}, \mathbf{sl}'_{n+k+1}, (\mathbf{sl}_{\text{o},i})_{i \in [n]}, (\mathbf{sl}_{\text{id}})_{j \in [k+1]} \right). \end{aligned}$$

There exists an IOR from $\mathcal{R}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}}$ to $\mathcal{R}_{\mathcal{C}', \mathcal{C}_{\text{zk}}, \mathbf{sl}'}$ with the following properties:

- **Round complexity:** $k + 3$.
- **Prover communication (in field elements):** $k \cdot (\ell_{\text{zk}} + m_{\text{zk}} \cdot \ell_{\text{zk}}) + 1 + m' \cdot \ell' + m_{\text{zk}} \cdot \ell_{\text{zk}} + t_{\text{ood}}$. Of these, $m' \cdot \ell'$ are sent as an oracle over Σ' , $(k+1) \cdot m_{\text{zk}} \cdot \ell_{\text{zk}}$ are sent as an oracle over Σ_{zk}^{k+1} and $k \cdot \ell_{\text{zk}} + t_{\text{ood}} + 1$ are sent as non-oracle messages.
- **Query complexity:** t queries to the first input oracle.
- **Prover time (in field operations):** $O(\ell \cdot 2^k + t_{\text{sl}} + \sum_{i \in [n]} t_{\text{sl}_i} + \text{tenc}(\mathcal{C}') + k \cdot \text{tenc}(\mathcal{C}_{\text{zk}}) + \text{teval}(\mathbf{ze}) + \text{teval}(\mathbf{ze}_{\text{ood}}))$.
- **Verifier time (in field operations):** $O(k \cdot \ell_{\text{zk}} + \text{teval}(\mathbf{ze}))$.
- **Round-by-round security:** For $\delta, \delta', \delta_{\text{zk}} \in (0, 1)$, the protocol has RBR knowledge soundness with relaxation $(\tilde{\mathcal{R}}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}}, \tilde{\mathcal{R}}_{\mathcal{C}', \mathcal{C}_{\text{zk}}, \mathbf{sl}'}^{\delta', \delta_{\text{zk}}})$ with errors

$$\left(\frac{|\Lambda(\mathcal{C} \equiv 2^k, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}} \equiv n+k, \delta_{\text{zk}})|}{|\mathbb{F}|}, \left(\epsilon_{\text{mca}}(\mathcal{C} \equiv 2^{k-j}, \delta) + \frac{\ell_{\text{zk}} \cdot |\Lambda(\mathcal{C} \equiv 2^{k-(j-1)}, \delta)| \cdot |\Lambda(\mathcal{C}_{\text{zk}} \equiv n+k, \delta_{\text{zk}})|}{|\mathbb{F}|} \right)_{j \in [k]}, \right. \\ \left. \frac{(|\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\text{zk}}, \delta_{\text{zk}})|)^2}{2} \cdot \varepsilon_{\text{ood}}, (1 - \delta)^t, |\Lambda(\mathcal{C}', \delta')| \cdot |\Lambda(\mathcal{C}_{\text{zk}} \equiv n+k+1, \delta_{\text{zk}})| \cdot \varepsilon_{\text{zero}} \right)$$

and extraction time $O(\sum_{j \in [k]} \text{tcor}(\mathcal{C} \equiv 2^{k-j}) + \text{tenc}(\mathcal{C}))$.

- **Zero-knowledge:** Suppose that $\text{char}(\mathbb{F}) \neq 2$, $\ell_{\text{zk}} \geq 2$, and:
 - $\mathbf{ze}_{\text{ood}}$ is a $(\ell_{\text{zk}} - r, \zeta_{\text{ze}})$ -private zero-evader;

- $\text{Enc}_{C'}$ is a t' -query zero-knowledge encoding with error $\zeta_{C'}$;
 - $\text{Enc}_{C_{zk}}$ is a t_{zk} -query zero-knowledge encoding with error $\zeta_{C_{zk}}$.
- Then, for every $\mathbf{t} \in \mathbb{N}^n$, the protocol is HVZK for $\mathfrak{D}^{\leq(t', \mathbf{t}, (t_{zk})_{j \in [k+1]})}$ with error $\zeta_{C'} + \zeta_{ze} + (k+1) \cdot \zeta_{C_{zk}}$ and simulator query complexity $(t, \mathbf{t})$.

Proof. Let $\text{IOR}_{sc}: \mathcal{R}_{C \equiv 2^k, C_{zk}, \mathbf{sl}} \rightarrow \mathcal{R}_{C, C_{zk}, \mathbf{sl}_{sc}}$ be the IOR from Theorem 6.2 and let $\text{IOR}_{cs}: \mathcal{R}_{C, C_{zk}, \mathbf{sl}_{sc}} \rightarrow \mathcal{R}_{C', C_{zk}, \mathbf{sl}'}$ be the IOR from Theorem 9.6 (with $n+k$ mask oracles). Define $\text{IOR} := \text{IOR}_{cs} \circ \text{IOR}_{sc}$.

The round, communication, and time bounds follow by adding the corresponding bounds of the two theorems. The round-by-round knowledge soundness statement (including the error vector and extraction time) follows from Theorem 4.5 together with the two constituent RBR guarantees.

For zero-knowledge, fix any $\mathcal{D} \in \mathfrak{D}^{\leq(t', \mathbf{t}, (t_{zk})_{j \in [k+1]})}$, and let $\mathbf{S}_{cs}$ be the simulator from the HVZK guarantee of IOR_{cs} . Set $\mathcal{D}_* := \mathcal{D}_{\mathbf{S}_{cs}}$. By the query bounds of $\mathbf{S}_{cs}$, $\mathcal{D}_*$ queries (i) f at most t times; (ii) the i -th original mask at most t_i times for $i \in [n]$; and (iii) each of the k masks introduced by sumcheck at most t_{zk} times. Hence $\mathcal{D}_*$ is in the query class required by Theorem 6.2. Applying that theorem gives a simulator $\mathbf{S}_{sc}$ with error $k \cdot \zeta_{C_{zk}}$ and with exactly the claimed query complexity to the original implicit instance.

Apply Theorem 4.5 to IOR_{sc} and IOR_{cs} . The composed simulator error is

$$(k \cdot \zeta_{C_{zk}}) + (\zeta_{C'} + \zeta_{ze} + \zeta_{C_{zk}}) = \zeta_{C'} + \zeta_{ze} + (k+1) \cdot \zeta_{C_{zk}}.$$

□

10 Zero-knowledge IOPP for constrained codes

We construct a zero-knowledge IOPP for constrained codes. This yields a formal statement and proof of Theorem 4, and can be seen as a zero-knowledge variant (and generalization) of both WHIR [ACFY25] and Ligerito [NA25].

Batched linear forms. We are going to iteratively use the IOR from $\mathcal{R}_{\mathcal{C} \equiv 2^k, \mathcal{C}_{zk}, \mathbf{sl}}$ to $\mathcal{R}_{\mathcal{C}', \mathcal{C}_{zk}, \mathbf{sl}'}$ in Theorem 9.10; in this IOR the prover incurs (among others) the cost $t_{\mathbf{sl}} + \sum_{i \in [n]} t_{\mathbf{sl}_i}$ associated to $\mathbf{sl} = (\mathbf{sl}, \mathbf{sl}_1, \dots, \mathbf{sl}_n)$. In particular, the second invocation after the first incurs this cost relative to the *new linear form* $\mathbf{sl}'$ (defined in Theorem 9.10). A similar cost is also incurred by the verifier. In both cases, the prover and verifier costs associated to $\mathbf{sl}'$ involve *batch of linear forms*.

Definition 10.1. Let $\mathbf{sl} \in \langle \mathbb{F}^{t \times n} \rangle$ and let $b \in \mathbb{N}$. We let $\mathbf{b}(\mathbf{sl}, b)$ denote the number of field operation to compute $\sum_{i \in [b]} \nu_i \cdot \mathbf{sl}(\mathbf{st}_i) \in \mathbb{F}^{t \times n}$ given $(\mathbf{st}_1, \dots, \mathbf{st}_b)$ and $\nu \in \mathbb{F}^b$. Note that $\mathbf{b}(\mathbf{sl}, b) \leq b \cdot t_{\mathbf{sl}} + O(b \cdot t \cdot n)$.

For many linear forms (e.g., the one in Section 9.3) the cost to compute such batches can be much smaller than $O(b \cdot t \cdot n)$. To see this, fix a matrix $\mathbf{M} \in \mathbb{F}^{m \times \ell}$ and let $|\mathbf{M}|$ denote the number of field operations to compute $\mathbf{M} \cdot \mathbf{v}$ for any $\mathbf{v} \in \mathbb{F}^\ell$. Let $\mathbf{sl}_{\mathbf{M}}$ be linear form mapping $x \in [m]$ to $\mathbf{M}[x, \cdot] \in \mathbb{F}^\ell$. We argue that $\mathbf{b}(\mathbf{sl}_{\mathbf{M}}, b) = O(b + |\mathbf{M}|)$. Fix $x_1, \dots, x_b$ and ν . Then $\sum_i \nu_i \cdot \mathbf{M}[x_i, \cdot] = \mathbf{M}^\top \cdot \left(\sum_{i \in [b]} \nu_i \cdot \mathbf{e}_{x_i} \right)$ where for $i \in [m]$, the vector $\mathbf{e}_i \in \mathbb{F}^m$ is the unit vector with 1 in position i and 0 everywhere else. This quantity can be computed in $O(b + |\mathbf{M}^\top|)$ field operations, which is $O(b + |\mathbf{M}|)$ operations by the transposition principle [Bor57; Ber].

Main theorem. The main theorem in this section follows by iteratively applying the zero-knowledge sumcheck IOR in Section 6 and the zero-knowledge code-switching IOR in Section 9.

Theorem 10.2. Fix the following:

- $n \in \mathbb{N}$ and $\mathbf{n} \in \mathbb{N}$ with $\mathbf{n} \geq 2$;
- $\forall i \in [\mathbf{n}], \mathcal{C}_i \subseteq \Sigma_i^{m_i} \equiv (\mathbb{F}^{\ell_i})^{m_i}$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_i}: \mathbb{F}^{\ell_i} \times \mathbb{F}^{r_i} \rightarrow \Sigma_i^{m_i}$;
- $\mathcal{C}_{zk} \subseteq \Sigma_{zk}^{m_{zk}} \equiv (\mathbb{F}^{\ell_{zk}})^{m_{zk}}$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{zk}}: \mathbb{F}^{\ell_{zk}} \times \mathbb{F}^{r_{zk}} \rightarrow \Sigma_{zk}^{m_{zk}}$ with $\ell_{zk} \geq \max_{i \in [\mathbf{n}]} r_i$;
- $\mathbf{sl}_0 = (\mathbf{sl}, (\mathbf{sl}_i)_{i \in [\mathbf{n}]}, (\mathbf{sl}_{o,i})_{i \in [\mathbf{n}]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i \in \langle \mathbb{F}^{\ell_{zk}} \rangle$, $\mathbf{sl}_{o,i} \in \langle \mathbb{F}^{t_i \times \ell_{zk}} \rangle$;
- $\forall i \in [\mathbf{n} - 1]$:
 - $k_i, t_i, t_{\text{ood},i} \in \mathbb{N}$;
 - $\mathbf{ze}_i: D_{\mathbf{ze},i} \rightarrow \mathbb{F}^{1+t_{\text{ood},i}+t_i}$ is a zero-evader with error $\varepsilon_{\text{zero},i}$;
 - $\mathbf{ze}_{\text{ood},i}: D_{\text{ood},i} \rightarrow \mathbb{F}^{t_{\text{ood},i} \times (\ell_i + \ell_{zk})}$ is a zero-evader with error $\varepsilon_{\text{ood},i}$;
- $t_n, t_{zk} \in \mathbb{N}$;
- set $k_n := 0$, and assume $2^{k_{i+1}} \cdot \ell_{i+1} \geq \ell_i$ for every $i \in [\mathbf{n} - 1]$;
- set $K_j := \sum_{i \leq j} k_i$, $N_i := n + K_i$ and $(K, N) := (K_{\mathbf{n}-1}, N_{\mathbf{n}-1})$.

There exists an IOPP for $\mathcal{R}_{\mathcal{C} \equiv 2^{k_1}, \mathcal{C}_{zk}, \mathbf{sl}_0}$ with the following properties.

- Round complexity: $K + 3 \cdot (\mathbf{n} - 1) + 2$ rounds.
- Prover communication (in field elements):

$$\sum_{i \in [\mathbf{n}-1]} \left(k_i \cdot (\ell_{zk} + m_{zk} \cdot \ell_{zk}) + 1 + 2^{k_{i+1}} \cdot m_{i+1} \cdot \ell_{i+1} + m_{zk} \cdot \ell_{zk} + t_{\text{ood},i} \right) + m_n \cdot \ell_n + N \cdot m_{zk} \cdot \ell_{zk} + \ell_n + r_n + N \cdot (\ell_{zk} + r_{zk}).$$

Of these:

- $(N + \sum_{i \in [n-1]} k_i) \cdot m_{z_k} \cdot \iota_{z_k}$ are sent as an oracle over alphabet $\Sigma_{z_k}^{N + \sum_{i \in [n-1]} k_i}$,
- $\forall i \in [n-1], 2^{k_{i+1}} \cdot m_{i+1} \cdot \iota_{i+1}$ are sent as oracles over alphabet $(\Sigma_{i+1})^{2^{k_{i+1}}}$,
- $m_n \cdot \iota_n$ are sent as an oracle over alphabet Σ_n ,
- $\sum_{i \in [n-1]} (k_i \cdot \ell_{z_k} + t_{\text{ood},i} + 1) + \ell_n + r_n + N \cdot (\ell_{z_k} + r_{z_k})$ are sent as non-oracle messages.
- **Query complexity:** $\sum_{i \in [n]} t_i + N \cdot t_{z_k}$ oracle queries in total.
- **Prover time (in field operations):**

$$O \left(\ell_1 \cdot 2^{k_1} + t_{\text{sl}} + \sum_{i \in [n]} t_{\text{sl}_i} + \sum_{i \in [n-1]} \left(\begin{array}{l} \text{tenc}(\mathcal{C}_{i+1}^{\equiv 2^{k_{i+1}}}) + k_i \cdot \text{tenc}(\mathcal{C}_{z_k}) \\ + \text{teval}(\text{ze}_i) + \text{teval}(\text{ze}_{\text{ood},i}) \\ + \text{b}(\text{sl}_{\mathcal{C}_i}^\#, t_i \cdot \iota_i) + \text{b}(\text{sl}_{\mathcal{C}_i}^s, t_i \cdot \iota_i) \end{array} \right) + \text{tenc}(\mathcal{C}_n) + N \cdot \text{tenc}(\mathcal{C}_{z_k}) \right).$$

- **Verifier time (in field operations):**

$$O \left(t_{\odot_{K^{\text{sl}}}} + \sum_{i \in [n]} t_{\text{sl}_i} + \sum_{i \in [n-1]} \left(\begin{array}{l} k_i \cdot \ell_{z_k} + \text{teval}(\text{ze}_i) \\ + \text{b}(\odot_{K-K_i} \text{sl}_{\mathcal{C}_i}^\#, t_i \cdot \iota_i) + \text{b}(\text{sl}_{\mathcal{C}_i}^s, t_i \cdot \iota_i) \\ + \text{b}(\odot_{K-K_i} \text{sl}_{\text{ze}_{\text{ood},i}}^{\leftarrow}, t_{\text{ood},i}) + \text{b}(\text{sl}_{\text{ze}_{\text{ood},i}}^{\rightarrow}, t_{\text{ood},i}) \end{array} \right) + \text{tenc}(\mathcal{C}_n) + N \cdot \text{tenc}(\mathcal{C}_{z_k}) \right).$$

- **Round-by-round security:** For every $\delta_{z_k} \in (0, 1)$ and $(\delta_i)_{i \in [n]} \in (0, 1)^n$, the protocol has RBR knowledge soundness with relaxation $\tilde{\mathcal{R}}_{\mathcal{C}_1^{\equiv 2^{k_1}}, \mathcal{C}_{z_k}, \text{sl}_0}^{\delta_1, \delta_{z_k}}$ with the errors $\epsilon_{\text{CS},1} \|\cdots\| \epsilon_{\text{CS},n-1} \|\epsilon_{\text{base}}$ defined below and total extraction time

$$O \left(\sum_{i \in [n-1]} \left(\sum_{h \in [k_i]} \text{tcor}(\mathcal{C}_i^{\equiv 2^{k_i-h}}) + \text{tenc}(\mathcal{C}_i) \right) + \text{tcor}(\mathcal{C}_n) + N_{n-1} \cdot \text{tcor}(\mathcal{C}_{z_k}) \right).$$

- For every $i \in [n-1]$, define

$$\epsilon_{\text{CS},i} := \left(\begin{array}{l} \frac{|\Lambda(\mathcal{C}_i^{\equiv 2^{k_i}}, \delta_i)| \cdot |\Lambda(\mathcal{C}_{z_k}^{\equiv N_i}, \delta_{z_k})|}{|\mathbb{F}|}, \\ \left(\epsilon_{\text{mca}}(\mathcal{C}_i^{\equiv 2^{k_i-h}}, \delta_i) + \frac{\ell_{z_k} \cdot |\Lambda(\mathcal{C}_i^{\equiv 2^{k_i-(h-1)}}, \delta_i)| \cdot |\Lambda(\mathcal{C}_{z_k}^{\equiv N_i}, \delta_{z_k})|}{|\mathbb{F}|} \right)_{h \in [k_i]}, \\ \frac{\left(|\Lambda(\mathcal{C}_{i+1}^{\equiv 2^{k_{i+1}}}, \delta_{i+1})| \cdot |\Lambda(\mathcal{C}_{z_k}, \delta_{z_k})| \right)^2}{2} \cdot \epsilon_{\text{ood},i}, \\ (1 - \delta_i)^{t_i}, \\ |\Lambda(\mathcal{C}_{i+1}^{\equiv 2^{k_{i+1}}}, \delta_{i+1})| \cdot |\Lambda(\mathcal{C}_{z_k}^{\equiv N_{i+1}}, \delta_{z_k})| \cdot \epsilon_{\text{zero},i} \end{array} \right),$$

- Define

$$\epsilon_{\text{base}} := \left(\begin{array}{l} \epsilon_{\text{mca}}(\mathcal{C}_n, \delta_n) + \epsilon_{\text{mca}}(\mathcal{C}_{z_k}^{\equiv N}, \delta_{z_k}) + \frac{|\Lambda(\mathcal{C}_n^{\equiv 2}, \delta_n)| \cdot |\Lambda(\mathcal{C}_{z_k}^{\equiv 2 \cdot N_{n-1}}, \delta_{z_k})|}{|\mathbb{F}|}, \\ \max \{ (1 - \delta_n)^{t_n}, (1 - \delta_{z_k})^{t_{z_k}} \} \end{array} \right).$$

- **Zero-knowledge:** Suppose that $\text{char}(\mathbb{F}) \neq 2$, $\ell_{z_k} \geq 2$, and:
 - for every $i \in [n-1]$, $\text{ze}_{\text{ood},i}$ is a $(\ell_{z_k} - r_i, \zeta_{\text{ze},i})$ -private zero-evader;
 - for every $i \in [2, n]$, $\mathcal{C}_i$ has a t_i -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_i}$;
 - $\mathcal{C}_{z_k}$ has a t_{z_k} -query zero-knowledge encoding with error $\zeta_{\mathcal{C}_{z_k}}$.

Then the protocol is honest-verifier zero-knowledge with error

$$\sum_{i \in [n-1]} \left(\zeta_{\mathcal{C}_{i+1}^{\equiv 2^{k_{i+1}}}} + \zeta_{\text{ze},i} + (k_i + 1) \cdot \zeta_{\mathcal{C}_{z_k}} \right) + \zeta_{\mathcal{C}_n} + N \cdot \zeta_{\mathcal{C}_{z_k}}$$

and simulator query complexity $(t_1, (t_{z_k})_{i \in [n]})$.

Proof. Let $\mathbf{sl}_0$ be the initial tuple from the theorem statement and for each $i \in [n-1]$ let $\mathbf{sl}_i$ be the output tuple after stage i . For each $i \in [n-1]$, apply Theorem 9.10 with

$$(\mathcal{C}, \mathcal{C}', n, k, t, t_{\text{ood}}) = (\mathcal{C}_i, \mathcal{C}_{i+1}^{\equiv 2^{k_i+1}}, N_{i-1}, k_i, t_i, t_{\text{ood},i})$$

and with zero-evaders $\mathbf{ze}_i$ and $\mathbf{ze}_{\text{ood},i}$, obtaining an IOR IOR_i from $\mathcal{R}_{\mathcal{C}_i^{\equiv 2^{k_i}}, N_{i-1}, \mathcal{C}_{zk}, \mathbf{sl}_{i-1}}$ to $\mathcal{R}_{\mathcal{C}_{i+1}^{\equiv 2^{k_i+1}}, N_{i-1}, \mathcal{C}_{zk}, \mathbf{sl}_i}$. Then $\text{IOR}_{\text{cs}} := \text{IOR}_1 \circ \dots \circ \text{IOR}_{n-1}$ is an IOR from $\mathcal{R}_{\mathcal{C}_1^{\equiv 2^{k_1}}, \mathcal{C}_{zk}, \mathbf{sl}_0}$ to $\mathcal{R}_{\mathcal{C}_n, \mathcal{C}_{zk}, \mathbf{sl}_{n-1}}$.

Apply the IOPP IOR_{base} from Theorem 7.1 to $\mathcal{R}_{\mathcal{C}_n, \mathcal{C}_{zk}, \mathbf{sl}_{n-1}}$ with $n := N$, $t := t_n$, and t_{zk} as in the statement. The final IOPP for $\mathcal{R}_{\mathcal{C}_1^{\equiv 2^{k_1}}, \mathcal{C}_{zk}, \mathbf{sl}_0}$ is $\text{IOR}_{\text{cs}} \circ \text{IOR}_{\text{base}}$.

The stated round/query/communication/time bounds and the round-by-round error vector (including extraction time) follow by careful accounting. We focus on establishing zero knowledge.

Zero-knowledge. We establish HVZK for all distinguishers by building simulators $\mathbf{S}_n, \mathbf{S}_{n-1}, \dots, \mathbf{S}_1$, and applying Theorem 4.5 at each step. For each stage $i \in [n-1]$, define

$$\epsilon_i := \zeta_{\mathcal{C}_{i+1}^{\equiv 2^{k_i+1}}} + \zeta_{\mathbf{ze},i} + (k_i + 1) \cdot \zeta_{\mathcal{C}_{zk}}.$$

Also define tail-error terms: for $s \in [n]$, $E_s := (\sum_{i=s}^{n-1} \epsilon_i) + \zeta_{\mathcal{C}_n} + N \cdot \zeta_{\mathcal{C}_{zk}}$; for $s = n$ the sum is empty, so $E_n = \zeta_{\mathcal{C}_n} + N \cdot \zeta_{\mathcal{C}_{zk}}$.

- *Base case.* Let $\mathbf{S}_n$ be the simulator for IOR_{base} from Theorem 7.1; the simulator has error E_n and queries the main oracle at most t_n times and each of the N mask oracles at most t_{zk} times. As IOR_{base} is an IOPP, simulation works (trivially) for all distinguishers.
- *Inductive step* ($i = n-1$ down to 1). Suppose we have already constructed a simulator $\mathbf{S}_{i+1}$ for the IOPP $\text{IOR}_{i+1} \circ \dots \circ \text{IOR}_{n-1} \circ \text{IOR}_{\text{base}}$ that has simulation error E_{i+1} and that queries the $\mathcal{C}_{i+1}^{\equiv 2^{k_i+1}}$ -oracle at most t_{i+1} times and each of the N_i mask oracles at most t_{zk} times.

Fix any distinguisher $\mathcal{D}$ and let $\mathcal{D}_{\mathbf{S}_{i+1}}$ be as in Definition 4.4, i.e., $\mathcal{D}_{\mathbf{S}_{i+1}}^y(\cdot) := \mathbf{S}_{i+1}^{y,\mathcal{D}}(\cdot)$. By the query bounds of $\mathbf{S}_{i+1}$, $\mathcal{D}_{\mathbf{S}_{i+1}}$ queries the $\mathcal{C}_{i+1}^{\equiv 2^{k_i+1}}$ -oracle at most t_{i+1} times and each of the N_i output mask oracles at most t_{zk} times; so $\mathcal{D}_{\mathbf{S}_{i+1}} \in \mathfrak{D}^{\leq(t_{i+1}, (t_{zk})_{j \in [N_i]})}$. Thus $\mathfrak{D}_{\mathbf{S}_{i+1}} \subseteq \mathfrak{D}^{\leq(t_{i+1}, (t_{zk})_{j \in [N_i]})}$.

The IOR IOR_i from Theorem 9.10 (instantiated with $n = N_{i-1}$ input masks) is HVZK with respect to $\mathfrak{D}^{\leq(t_{i+1}, (t_{zk})_{j \in [N_i]})}$: the k_i mask oracles introduced during the sumcheck phase are each queried by $\mathcal{D}_{\mathbf{S}_{i+1}}$ at most t_{zk} times (since they belong to the N_i output masks), satisfying $t_{N_{i-1}+j} \leq t_{zk}$ for every $j \in [k_i]$ as required. Since $\mathfrak{D}_{\mathbf{S}_{i+1}} \subseteq \mathfrak{D}^{\leq(t_{i+1}, (t_{zk})_{j \in [N_i]})}$, IOR_i is in particular HVZK with respect to $\mathfrak{D}_{\mathbf{S}_{i+1}}$.

Let $\mathbf{S}_i$ be simulator from Theorem 9.10, with error ϵ_i and query complexity at most t_i to the $\mathcal{C}_i^{\equiv 2^{k_i}}$ -oracle and at most t_{zk} to each of the N_{i-1} input mask oracles. Applying Theorem 4.5 with $\text{IOR}_1 := \text{IOR}_i$ (simulator $\mathbf{S}_i$) and $\text{IOR}_2 := \text{IOR}_{i+1} \circ \dots \circ \text{IOR}_{n-1} \circ \text{IOR}_{\text{base}}$ (simulator $\mathbf{S}_{i+1}$) yields a simulator for the IOPP $\text{IOR}_i \circ (\text{IOR}_{i+1} \circ \dots \circ \text{IOR}_{n-1} \circ \text{IOR}_{\text{base}})$ that is HVZK with ZK error $\epsilon_i + E_{i+1} = E_i$.

- *Conclusion.* Unrolling the induction to $i = 1$ gives a simulator $\mathbf{S} := \mathbf{S}_1$ for the full protocol. $\mathbf{S}$ queries the original implicit instance $(f, (\xi_i)_{i \in [n]})$ at most t_1 times for f and at most t_{zk} times for each ξ_i , as guaranteed by Theorem 9.10 for stage 1. The total ZK error is E_1 , which equals

$$\sum_{i \in [n-1]} \left(\zeta_{\mathcal{C}_{i+1}^{\equiv 2^{k_i+1}}} + \zeta_{\mathbf{ze},i} + (k_i + 1) \cdot \zeta_{\mathcal{C}_{zk}} \right) + \zeta_{\mathcal{C}_n} + N \cdot \zeta_{\mathcal{C}_{zk}}.$$

Efficiency. The efficiency of the construction follows by accumulating the costs of the building blocks throughout, and computing the cost of succinct linear form at each step (in particular, the codeswitches costs in see Section 9.3). The latter add the following terms to the prover/verifier time — the prover needs to compute a batch of size $t_i \cdot \iota_i$ of the linear forms $\mathbf{sl}_{c_i}^\#, \mathbf{sl}_{c_i}^\$$ for all $i \in [\mathbf{n} - 1]$, as this must be proved by subsequent reductions. Similarly, the verifier must compute the (folded) succinct linear forms associated with those codeswitches, which entails $\mathbf{b}(\odot_{K-K_i} \mathbf{sl}_{c_i}^\#, t_i) + \mathbf{b}(\mathbf{sl}_{c_i}^\$, t_i)$ field operations (as these are also computable in a batch). $\square$

11 Zero-knowledge reduction from R1CS

We describe a reduction from $\mathcal{R}_{\text{R1CS}}$ to testing proximity to a constrained code relation, similar to the one in Definition 5.8. The reduction is similar to the Σ -IOP from [ACFY25], combined with a masking technique similar to the one used for the zero-knowledge sumcheck in Section 6.

Definition 11.1. *Let:*

- $n_{\text{in}}, n_{\text{out}} \in \mathbb{N}$;
- $\mathcal{C} \subseteq \mathbb{F}^m$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$;
- $\mathcal{C}_{\text{out}}^{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{out}}^{\text{zk}}}$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{out}}^{\text{zk}}}: \mathbb{F}^{\ell_{\text{out}}^{\text{zk}}} \times \mathbb{F}^{r_{\text{out}}^{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{out}}^{\text{zk}}}$;
- $\mathcal{C}_{\text{in}}^{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{in}}^{\text{zk}}}$ with a zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{in}}^{\text{zk}}}: \mathbb{F}^{\ell_{\text{in}}^{\text{zk}}} \times \mathbb{F}^{r_{\text{in}}^{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{in}}^{\text{zk}}}$;
- $\mathbf{sl} = (\mathbf{sl}, (\mathbf{sl}_i^{\text{in}})_{i \in [n_{\text{in}}]}, (\mathbf{sl}_{o,i}^{\text{in}})_{i \in [n_{\text{in}}]}, (\mathbf{sl}_{o,i}^{\text{out}})_{i \in [n_{\text{out}}]})$ where $\mathbf{sl} \in \langle \mathbb{F}^\ell \rangle$, $\mathbf{sl}_i^{\text{in}} \in \langle \mathbb{F}^{\ell_{\text{in}}^{\text{zk}}} \rangle$, $\mathbf{sl}_{o,i}^{\text{in}} \in \langle \mathbb{F}^{2 \times \ell_{\text{in}}^{\text{zk}}} \rangle$, $\mathbf{sl}_{o,i}^{\text{out}} \in \langle \mathbb{F}^{\ell_{\text{out}}^{\text{zk}}} \rangle$.

Define:

$$\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{in}}^{\text{zk}}, \mathcal{C}_{\text{out}}^{\text{zk}}, \mathbf{sl}} := \left\{ \begin{array}{l} \mathbb{X} = \left(\begin{array}{l} (\mu, \mathbf{st}, (\mathbf{st}_i^{\text{in}})_{i \in [n_{\text{in}}]}), \\ ((\mu_i^{\text{in}}, \mathbf{st}_{o,i}^{\text{in}})_{i \in [n_{\text{in}}]}), \\ ((\mu_i^{\text{out}}, \mathbf{st}_{o,i}^{\text{out}})_{i \in [n_{\text{out}}]}) \end{array} \right), \\ \mathbb{Y} = (f, (\xi_i)_{i \in [n_{\text{in}}]}, (\xi_i)_{i \in [n_{\text{out}}]}), \\ \mathbb{W} = (\mathbf{f}, \mathbf{r}, (\xi_i, \bar{\mathbf{r}}_i)_{i \in [n_{\text{in}}]}, (\xi_i, \mathbf{r}_i)_{i \in [n_{\text{out}}]}) \end{array} \mid \begin{array}{l} f = \text{Enc}_{\mathcal{C}}(\mathbf{f}, \mathbf{r}) \\ \forall i \in [n_{\text{in}}]: \xi_i = \text{Enc}_{\mathcal{C}_{\text{in}}^{\text{zk}}}(\bar{\xi}_i, \bar{\mathbf{r}}_i) \\ \forall i \in [n_{\text{out}}]: \xi_i = \text{Enc}_{\mathcal{C}_{\text{out}}^{\text{zk}}}(\xi_i, \mathbf{r}_i) \\ \forall i \in [n_{\text{in}}]: \mathbf{sl}_{o,i}^{\text{in}}(\mathbf{st}_i^{\text{in}}) \cdot \xi_i = \mu_i^{\text{in}} \\ \forall i \in [n_{\text{out}}]: \mathbf{sl}_{o,i}^{\text{out}}(\mathbf{st}_{o,i}^{\text{out}}) \cdot \xi_i = \mu_i^{\text{out}} \\ \langle \mathbf{f}, \mathbf{sl}(\mathbf{st}) \rangle + \sum_{i \in [n_{\text{in}}]} \langle \bar{\xi}_i, \mathbf{sl}_i^{\text{in}}(\mathbf{st}_i^{\text{in}}) \rangle = \mu \end{array} \right\}.$$

For every $\delta, \delta_{\text{zk}} \in (0, 1)$ we define a corresponding relaxed relation for proximity testing:

$$\tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{in}}^{\text{zk}}, \mathcal{C}_{\text{out}}^{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}} := \left\{ (\mathbb{X}, \mathbb{Y}, (\mathbb{W}, \bar{\mathbb{Y}})) \mid \Delta(\mathbb{Y}, \bar{\mathbb{Y}}) \leq (\delta, \underbrace{\delta_{\text{zk}}, \dots, \delta_{\text{zk}}}_{n_{\text{in}} + n_{\text{out}} \text{ terms}}) \wedge (\mathbb{X}, \bar{\mathbb{Y}}, \mathbb{W}) \in \mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{in}}^{\text{zk}}, \mathcal{C}_{\text{out}}^{\text{zk}}, \mathbf{sl}} \right\}.$$

This definition is a minor extension of Definition 5.8 that we use to illustrate the different roles each zero-knowledge code plays in the reduction. By padding the shorter mask messages (and the corresponding succinct-evaluation states/targets) with zeros to a common length, both mask codes (the inner and outer codes) can be the same code, returning to the relation in Definition 5.8.

Definition 11.2. *The relation $\mathcal{R}_{\text{R1CS}}$ is the set of all pairs $((\mathbb{F}, n_0, \ell, A, B, C, v), w)$ where $\mathbb{F}$ is a finite field, $n_0, \ell \in \mathbb{N}$, A, B, C are $(\ell + n_0) \times (\ell + n_0)$ matrices over $\mathbb{F}$, $v \in \mathbb{F}^{n_0}$, and $w \in \mathbb{F}^\ell$, such that for all $i \in [(\ell + n_0)]$:*

$$\left(\sum_{j=1}^{(\ell+n_0)} A_{i,j} \cdot z_j \right) \cdot \left(\sum_{j=1}^{(\ell+n_0)} B_{i,j} \cdot z_j \right) = \left(\sum_{j=1}^{(\ell+n_0)} C_{i,j} \cdot z_j \right)$$

for $z := (v, w) \in \mathbb{F}^{(\ell+n_0)}$.

Theorem 11.3. *Assume $\ell = n_0$ and that ℓ is a power of 2.*

- Let $\mathcal{C} \subseteq \Sigma^m \equiv (\mathbb{F}^\ell)^m$ be a code with zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \Sigma^m$.
- Let $\mathcal{C}_{\text{out}}^{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{out}}^{\text{zk}}} \equiv (\mathbb{F}^{\ell_{\text{out}}^{\text{zk}}})^{m_{\text{out}}^{\text{zk}}}$ be a code with zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{out}}^{\text{zk}}}: \mathbb{F}^{\ell_{\text{out}}^{\text{zk}}} \times \mathbb{F}^{r_{\text{out}}^{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{out}}^{\text{zk}}}$.

- Let $\mathcal{C}_{\text{in}}^{\text{zk}} \subseteq \Sigma_{\text{zk}}^{m_{\text{in}}^{\text{zk}}} \equiv (\mathbb{F}_{\text{in}}^{\iota_{\text{in}}^{\text{zk}}})^{m_{\text{in}}^{\text{zk}}}$ be a code with zero-knowledge encoding $\text{Enc}_{\mathcal{C}_{\text{in}}^{\text{zk}}}: \mathbb{F}_{\text{in}}^{\ell_{\text{in}}^{\text{zk}}} \times \mathbb{F}_{\text{in}}^{r_{\text{in}}^{\text{zk}}} \rightarrow \Sigma_{\text{zk}}^{m_{\text{in}}^{\text{zk}}}$.
- Let $\text{ze}: D_{\text{ze}} \rightarrow \mathbb{F}^3$ be a zero-evader with error $\varepsilon_{\text{zero}}$.

Define:

$$\begin{aligned} \mathbf{sl} &:= \left(\mathbf{sl}, \left(\mathbf{sl}_{M,i}^{\text{in}} \right)_{M \in \{A,B,C\}, i \in [\log \ell + 1]}, \left(\mathbf{sl}_{o,M,i}^{\text{in}} \right)_{M \in \{A,B,C\}, i \in [\log \ell + 1]}, \left(\mathbf{sl}_{o,i}^{\text{out}} \right)_{i \in [\log \ell + 1]} \right) \\ \text{where } \mathbf{sl}_{o,M,i}^{\text{in}} &:= (\mathbf{sl}_{\text{id}}, \mathbf{sl}_{\text{id}})^{\top}, \\ \mathbf{sl}_{o,i}^{\text{out}} &:= \mathbf{sl}_{\text{id}}, \\ \mathbf{sl} &:= \mathbf{sl}[\text{ze}, (\mathbf{sl}_{\text{id}}, \mathbf{sl}_{\text{id}}, \mathbf{sl}_{\text{id}})], \\ \mathbf{sl}_{M,i}^{\text{in}} &:= \mathbf{sl}_{\text{id}}. \end{aligned}$$

There exists an IOR from $\mathcal{R}_{\text{R1CS}}$ to $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{in}}^{\text{zk}}, \mathcal{C}_{\text{out}}^{\text{zk}}, \mathbf{sl}}$ with the following properties.

- Round complexity: $O(\log \ell)$.
- Prover communication (in field elements): $m \cdot \iota + 3(\log \ell + 1) \cdot m_{\text{in}}^{\text{zk}} \cdot \iota_{\text{in}}^{\text{zk}} + (\log \ell + 1) \cdot m_{\text{out}}^{\text{zk}} \cdot \iota_{\text{out}}^{\text{zk}} + (\log \ell + 1) \cdot (\ell_{\text{out}}^{\text{zk}} + 1) + 4$. Of these,
 - $m \cdot \iota$ are sent as an oracle over alphabet Σ ,
 - $3(\log \ell + 1) \cdot m_{\text{in}}^{\text{zk}} \cdot \iota_{\text{in}}^{\text{zk}} + (\log \ell + 1) \cdot m_{\text{out}}^{\text{zk}} \cdot \iota_{\text{out}}^{\text{zk}}$ are sent as mask oracles over alphabet Σ_{zk} (consisting of $3(\log \ell + 1)$ inner-mask oracles and $\log \ell + 1$ outer-mask oracles), and
 - $(\log \ell + 1) \cdot (\ell_{\text{out}}^{\text{zk}} + 1) + 4$ are sent as non-oracle messages.
- Verifier queries: none (a queryless IOR).
- Prover time (in field operations): $O(\text{tenc}(\mathcal{C}) + \log \ell \cdot (\text{tenc}(\mathcal{C}_{\text{out}}^{\text{zk}}) + \text{tenc}(\mathcal{C}_{\text{in}}^{\text{zk}})) + \ell)$.
- Verifier time (in field operations): $O(\log \ell \cdot \ell_{\text{out}}^{\text{zk}} + \ell)$.
- **Round-by-round security:** For every $\delta, \delta_{\text{zk}} \in (0, 1)$, Construction 11.4 has RBR knowledge soundness with relaxation $(\mathcal{R}_{\text{R1CS}}, \tilde{\mathcal{R}}_{\mathcal{C}, \mathcal{C}_{\text{in}}^{\text{zk}}, \mathcal{C}_{\text{out}}^{\text{zk}}, \mathbf{sl}}^{\delta, \delta_{\text{zk}}})$ and errors

$$\left(\frac{(\ell_{\text{out}}^{\text{zk}} + 1) \cdot |\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda|}{|\mathbb{F}|}, \left(\frac{|\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda| \cdot \ell_{\text{out}}^{\text{zk}}}{|\mathbb{F}|} \right)_{j \in [\log \ell + 1]}, \frac{|\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda| \cdot \ell_{\text{out}}^{\text{zk}}}{|\mathbb{F}| - 2}, |\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda| \cdot \varepsilon_{\text{zero}} \right)$$

where $|\Lambda| := |\Lambda((\mathcal{C}_{\text{out}}^{\text{zk}})^{\equiv \log \ell + 1}, \delta_{\text{zk}})| \cdot |\Lambda((\mathcal{C}_{\text{in}}^{\text{zk}})^{\equiv 3(\log \ell + 1)}, \delta_{\text{zk}})|$. The total extraction time is $O(\log \ell \cdot (\text{tcor}(\mathcal{C}_{\text{out}}^{\text{zk}}) + \text{tcor}(\mathcal{C}_{\text{in}}^{\text{zk}})))$.

- **Zero-knowledge:** If $\text{char}(\mathbb{F}) \neq 2$, $\ell_{\text{out}}^{\text{zk}} \geq 2 \cdot \ell_{\text{in}}^{\text{zk}}$, $\ell_{\text{in}}^{\text{zk}} \geq 4$, and $\text{Enc}_{\mathcal{C}}, \text{Enc}_{\mathcal{C}_{\text{out}}^{\text{zk}}}, \text{Enc}_{\mathcal{C}_{\text{in}}^{\text{zk}}}$ are t -query zero-knowledge encodings with error ζ , then the reduction is HVZK for $\mathfrak{D}^{\leq t}$ with error $(4 \log \ell + 5) \cdot \zeta$ (and no query complexity as $\mathcal{R}_{\text{R1CS}}$ has no implicit instance).

Construction 11.4. Let $\mathbb{F}$ be a field. Consider the following ingredients and notation.

- We assume that ℓ is a power of 2. Sometimes we refer to elements of $\{0, 1\}^{\log \ell}$ as elements in $[\ell]$. Implicitly, we assume a bijection between the two domains and use it as appropriate to translate between them. We also identify $\{0, 1\}^{\log \ell + 1}$ with $[(\ell + n_0)]$ (recall that we assume $(\ell + n_0) = 2\ell$).
- We let $\hat{f}_v \in \mathbb{F}^{<2}[\mathbf{x}_1, \dots, \mathbf{x}_{\log \ell}]$ be the unique multilinear polynomial satisfying $\hat{f}_v(b) = v(b)$ for every $b \in \{0, 1\}^{\log \ell}$.
- For every $M \in \{A, B, C\}$ let $\hat{f}_M \in \mathbb{F}^{<2}[\mathbf{x}_1, \dots, \mathbf{x}_{\log \ell + 1}, \mathbf{y}_1, \dots, \mathbf{y}_{\log \ell + 1}]$ be the unique multilinear polynomial such that

$$\hat{f}_M(a, b) = M[a, b] \quad \text{for all } a, b \in \{0, 1\}^{\log \ell + 1}.$$

Inputs. The honest prover receives $((\mathbb{F}, n_0, \ell, A, B, C, v), w) \in \mathcal{R}_{\text{R1CS}}$ and the verifier receives $(\mathbb{F}, n_0, \ell, A, B, C, v)$. Define $\text{st}_1 := (1, 0^{\ell_{\text{in}}^{\text{zk}}-1})$ and $\text{st}_2 := (0, 1, 0^{\ell_{\text{in}}^{\text{zk}}-2})$.

Interaction phase.

1. **Witness hiding masks.** For each $M \in \{A, B, C\}, i \in [\log \ell + 1]$ send $\bar{\xi}_{M,i} \in \Sigma_{\text{zk}}^{m_{\text{in}}^{\text{zk}}}$. In the honest case, the prover samples a coefficient vector $\bar{\xi}_{M,i} \in \mathbb{F}^{\ell_{\text{in}}^{\text{zk}}}$ (equivalently, a polynomial in $\mathbb{F}^{<\ell_{\text{in}}^{\text{zk}}}[\mathbf{X}]$), conditioned on $\bar{\xi}_{M,i}(0) = \bar{\xi}_{M,i}(1) = 0$, samples $\bar{r}_{M,i} \in \mathbb{F}^{r_{\text{in}}^{\text{zk}}}$, and sends $\bar{\xi}_{M,i} := \text{Enc}_{C_{\text{in}}^{\text{zk}}}(\bar{\xi}_{M,i}, \bar{r}_{M,i})$.
Succinct evaluation: define $\mu_{M,i}^{\text{in}} = (0, 0)^\top$ and $\text{st}_{\circ, M,i}^{\text{in}} = (\text{st}_1, \text{st}_2)^\top$.

2. **Send witness encoding.** The prover sends a $f \in \Sigma^m$. In the honest case, the prover sets $\mathbf{f} := w$, samples $\mathbf{r} \in \mathbb{F}^r$, and sends $f := \text{Enc}_C(\mathbf{f}, \mathbf{r})$, where $\hat{f}_w$ is the unique multilinear extension of w , i.e.

$$\hat{f}_w(b) = w(b) \quad \forall b \in \{0, 1\}^{\log \ell}.$$

3. **Sumcheck.** The prover and verifier engage in a sumcheck protocol. In the honest case, the prover defines the following polynomials:

- *Boolean extension of the witness/input:*

$$\hat{f}_z(\mathbf{X}_1, \dots, \mathbf{X}_{\log \ell + 1}) := (1 - \mathbf{X}_{\log \ell + 1}) \cdot \hat{f}_v(\mathbf{X}_1, \dots, \mathbf{X}_{\log \ell}) + \mathbf{X}_{\log \ell + 1} \cdot \hat{f}_w(\mathbf{X}_1, \dots, \mathbf{X}_{\log \ell}).$$

- *Constraint polynomial:* Define:

$$\begin{aligned} \hat{g}(\mathbf{X}) := & \left(\sum_{b \in \{0,1\}^{\log \ell + 1}} \hat{f}_A(\mathbf{X}, b) \hat{f}_z(b) + \left(\sum_{i=1}^{\log \ell + 1} \bar{\xi}_{A,i}(\mathbf{X}_i) \right) \right) \cdot \left(\sum_{b \in \{0,1\}^{\log \ell + 1}} \hat{f}_B(\mathbf{X}, b) \hat{f}_z(b) + \left(\sum_{i=1}^{\log \ell + 1} \bar{\xi}_{B,i}(\mathbf{X}_i) \right) \right) \\ & - \left(\sum_{b \in \{0,1\}^{\log \ell + 1}} \hat{f}_C(\mathbf{X}, b) \hat{f}_z(b) + \left(\sum_{i=1}^{\log \ell + 1} \bar{\xi}_{C,i}(\mathbf{X}_i) \right) \right). \end{aligned}$$

4. **Sumcheck masks.** For each $j \in \{1, \dots, \log \ell + 1\}$ send $\xi_j \in \Sigma_{\text{zk}}^{m_{\text{out}}^{\text{zk}}}$, and send one value $\tilde{\mu}$. In the honest case, for each j the prover samples a coefficient vector $\xi_j \in \mathbb{F}^{\ell_{\text{out}}^{\text{zk}}}$ (equivalently, a polynomial in $\mathbb{F}^{<\ell_{\text{out}}^{\text{zk}}}[\mathbf{X}]$), samples $\mathbf{r}_j \in \mathbb{F}^{r_{\text{out}}^{\text{zk}}}$, and sends $\xi_j := \text{Enc}_{C_{\text{out}}^{\text{zk}}}(\xi_j, \mathbf{r}_j)$. Then it sends

$$\tilde{\mu} := \sum_{a \in \{0,1\}^{\log \ell + 1}} (\xi_1(a_1) + \dots + \xi_{\log \ell + 1}(a_{\log \ell + 1})).$$

5. **Sumcheck combining randomness.** The verifier sends $\varepsilon \leftarrow \mathbb{F}$ and $\mathbf{r} \leftarrow \mathbb{F}^{\log \ell + 1}$.

6. **Sumcheck messages.** The prover and the verifier engage with a sumcheck protocol on the claim $\sum_{a \in \{0,1\}^{\log \ell + 1}} \varepsilon \cdot \hat{g}(a) \cdot \text{eq}(\mathbf{r}, a) + \left(\sum_{i=1}^{\log \ell + 1} \xi_i(a_i) \right) = \tilde{\mu}$. Let $\alpha := \emptyset$. For each $j \in \{1, \dots, \log \ell + 1\}$:

- The prover sends a polynomial $\hat{h}_j \in \mathbb{F}^{<\ell_{\text{out}}^{\text{zk}}}[\mathbf{X}]$. In the honest case, $\hat{h}_j(\mathbf{X})$ is equal to the following:

$$\sum_{a \in \{0,1\}^{\log \ell + 1 - j}} \varepsilon \cdot \hat{g}(\alpha, \mathbf{X}, a) \cdot \text{eq}(\mathbf{r}, (\alpha, \mathbf{X}, a)) + \left(\sum_{i=1}^{j-1} \xi_i(\alpha_i) \right) + \xi_j(\mathbf{X}) + \left(\sum_{i=j+1}^{\log \ell + 1} \xi_i(a_i) \right)$$

where $\hat{g}(\alpha, \mathbf{X}, a)$ is:

$$\begin{aligned} \hat{g}(\alpha, \mathbf{X}, a) &:= \left(\sum_{b \in \{0,1\}^{\log \ell + 1}} \hat{f}_A((\alpha, \mathbf{X}, a), b) \hat{f}_z(b) + \left(\left(\sum_{i=1}^{j-1} \bar{\xi}_{A,i}(\alpha_i) \right) + \bar{\xi}_{A,j}(\mathbf{X}) + \left(\sum_{i=j+1}^{\log \ell + 1} \bar{\xi}_{A,i}(a_i) \right) \right) \right) \\ &\cdot \left(\sum_{b \in \{0,1\}^{\log \ell + 1}} \hat{f}_B((\alpha, \mathbf{X}, a), b) \hat{f}_z(b) + \left(\left(\sum_{i=1}^{j-1} \bar{\xi}_{B,i}(\alpha_i) \right) + \bar{\xi}_{B,j}(\mathbf{X}) + \left(\sum_{i=j+1}^{\log \ell + 1} \bar{\xi}_{B,i}(a_i) \right) \right) \right) \\ &- \left(\sum_{b \in \{0,1\}^{\log \ell + 1}} \hat{f}_C((\alpha, \mathbf{X}, a), b) \hat{f}_z(b) + \left(\left(\sum_{i=1}^{j-1} \bar{\xi}_{C,i}(\alpha_i) \right) + \bar{\xi}_{C,j}(\mathbf{X}) + \left(\sum_{i=j+1}^{\log \ell + 1} \bar{\xi}_{C,i}(a_i) \right) \right) \right) \end{aligned}$$

- The verifier checks that $\sum_{a \in \{0,1\}} \hat{h}_j(a) = \hat{h}_{j-1}(\alpha_{j-1})$ (for $j = 1$, the verifier compares to $\tilde{\mu}$).
- The verifier samples $\alpha_j \leftarrow \mathbb{F}$ (except for the last round where $\alpha_{\log \ell + 1} \leftarrow \mathbb{F} \setminus \{0,1\}$) and updates $\alpha := (\alpha \| \alpha_j)$.

7. **Reveal outer masks evaluations.** The prover sends $\forall j \in \{1, \dots, \log \ell + 1\}$, $\tilde{\mu}_j$. In the honest case, $\tilde{\mu}_j = \xi_j(\alpha_j)$.

Succinct evaluation: define $\mu_j^{\text{out}} = \tilde{\mu}_j$ and $\text{st}_{\alpha,j}^{\text{out}} = \text{pow}(\alpha_j)$.

8. **Sumcheck's final check.** The prover sends v_M for every $M \in \{A, B, C\}$. In the honest case:

$$\begin{aligned} v_M &:= \sum_{b \in \{0,1\}^{\log \ell + 1}} \hat{f}_M(\alpha, b) \cdot \hat{f}_z(b) + \left(\sum_{i=1}^{\log \ell + 1} \bar{\xi}_{M,i}(\alpha_i) \right) \\ &= \sum_{b \in \{0,1\}^{\log \ell}} \left(\hat{f}_M(\alpha, b, 0) \cdot \hat{f}_v(b) + \hat{f}_M(\alpha, b, 1) \cdot \hat{f}_w(b) \right) + 2 \cdot \left(\sum_{i=1}^{\log \ell + 1} \bar{\xi}_{M,i}(\alpha_i) \right) \end{aligned}$$

The verifier checks that:

$$\varepsilon \cdot (v_A \cdot v_B - v_C) \cdot \text{eq}(\mathbf{r}, \alpha) + \sum_{j=1}^{\log \ell + 1} \tilde{\mu}_j = \hat{h}_{\log \ell + 1}(\alpha_{\log \ell + 1})$$

Finally, the verifier computes:

$$u_M = \sum_{b \in \{0,1\}^{\log \ell}} \left(\hat{f}_M(\alpha, b, 0) \cdot \hat{f}_v(b) \right).$$

9. **Joint constraint.** The verifier samples and sends $\rho \leftarrow \mathbb{F}$.

Succinct evaluation: Let $\text{st}_M = M$ for all $M \in \{A, B, C\}$ (note that we abuse the notation but we mean that the state is the succinct description of the matrix) and $\text{sl}_M = \text{sl}_{\text{id}}$ ³. Set: $\text{sl} = \text{sl}[\text{ze}, (\text{sl}_A, \text{sl}_B, \text{sl}_C)]$ and $\text{st} = (\text{st}_A, \text{st}_B, \text{st}_C, \rho)$. Define also, for each $M \in \{A, B, C\}$ and $i \in [\log \ell + 1]$, $\text{sl}_{M,i}^{\text{in}} = \text{sl}_{\text{id}}$ and $\text{st}_{M,i}^{\text{in}} = (\text{pow}(\alpha_i), \text{ze}(\rho)_M)$, and set $\mu = \sum_{M \in \{A, B, C\}} \text{ze}(\rho)_M \cdot (v_M - u_M)$ (where $\text{ze}(\rho)_M \in \mathbb{F}$ denotes the coordinate associated with $M \in \{A, B, C\}$).

³Note the slight abuse of notation where we treat the entire description of the matrix as the succinct description from which we compute the MLE as in the beginning of the construction.

10. **Output claim:** The prover and the verifier both implicitly define the output instance for $\mathcal{R}_{\mathcal{C}, \mathcal{C}_{\text{in}}^{\text{zk}}, \mathcal{C}_{\text{out}}^{\text{zk}}, \text{sl}}$:

$$\begin{aligned} \mathbb{x} &= ((\mu, \text{st}, (\text{st}_{M,i}^{\text{in}})_{M \in \{A,B,C\}, i \in [\log \ell + 1]}), (\mu_{M,i}^{\text{in}}, \text{st}_{o,M,i}^{\text{in}})_{M \in \{A,B,C\}, i \in [\log \ell + 1]}, (\mu_i^{\text{out}}, \text{st}_{o,i}^{\text{out}})_{i \in [\log \ell + 1]}) \\ \mathbb{y} &= (f, (\bar{\xi}_{M,i})_{M \in \{A,B,C\}, i \in [\log \ell + 1]}, (\xi_i)_{i \in [\log \ell + 1]}). \end{aligned}$$

In the honest case, the prover also defines the witness:

$$\mathbb{w} = (\mathbf{f}, \mathbf{r}, (\bar{\xi}_{M,i}, \bar{r}_{M,i})_{M \in \{A,B,C\}, i \in [\log \ell + 1]}, (\xi_i, \mathbf{r}_i)_{i \in [\log \ell + 1]}).$$

Proof sketch of Theorem 11.3. We sketch the proof for round-by-round knowledge soundness, and honest-verifier zero-knowledge.

Round-by-round knowledge soundness. Ignoring masks, the protocol follows the Σ -IOP analysis for $\mathcal{R}_{\text{R1CS}}$ in [ACFY25]. The differences are the added inner/outer masking layers and the final **ze**-based joint constraint, and these are handled with the same tools as in Section 6. Writing

$$|\Lambda| := |\Lambda((\mathcal{C}_{\text{out}}^{\text{zk}})^{\equiv \log \ell + 1}, \delta_{\text{zk}})| \cdot |\Lambda((\mathcal{C}_{\text{in}}^{\text{zk}})^{\equiv 3(\log \ell + 1)}, \delta_{\text{zk}})|$$

as in the theorem statement, the round-by-round error vector is

$$\left(\frac{(\ell_{\text{out}}^{\text{zk}} + 1) \cdot |\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda|}{|\mathbb{F}|}, \left(\frac{|\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda| \cdot \ell_{\text{out}}^{\text{zk}}}{|\mathbb{F}|} \right)_{j \in [\log \ell + 1]}, \frac{|\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda| \cdot \ell_{\text{out}}^{\text{zk}}}{|\mathbb{F}| - 2}, |\Lambda(\mathcal{C}, \delta)| \cdot |\Lambda| \cdot \varepsilon_{\text{zero}} \right).$$

The first coordinate corresponds to the initial masked-sumcheck consistency check, the vector coordinates correspond to the per-round sumcheck checks for rounds $j \in [\log \ell + 1]$, the third coordinate corresponds to the final check at $\alpha_{\log \ell + 1} \in \mathbb{F} \setminus \{0, 1\}$ (hence denominator $|\mathbb{F}| - 2$), and the fourth coordinate comes from the final random linear combination with **ze** in the joint constraint. Extraction time is $O(\log \ell \cdot (\text{tcor}(\mathcal{C}_{\text{out}}^{\text{zk}}) + \text{tcor}(\mathcal{C}_{\text{in}}^{\text{zk}})))$.

Honest-verifier zero-knowledge. Fix any $\mathcal{D} \in \mathcal{D}^{\leq t}$ and define a simulator for its view. Let $\mathbf{S}_{\mathcal{C}_{\text{out}}^{\text{zk}}}$ and $\mathbf{S}_{\mathcal{C}_{\text{in}}^{\text{zk}}}$ denote the simulators for the t -query zero-knowledge encodings of $\mathcal{C}_{\text{out}}^{\text{zk}}$ and $\mathcal{C}_{\text{in}}^{\text{zk}}$ (with error ζ). If $\mathcal{D}$ also queries f , we use the analogous simulator $\mathbf{S}_{\mathcal{C}}$ for $\text{Enc}_{\mathcal{C}}$.

$\mathbf{S}^{\mathcal{D}}(\mathbb{F}, \ell, A, B, C, v)$:

1. Sample verifier randomness exactly as in the protocol: $\varepsilon \leftarrow \mathbb{F}, \mathbf{r} \leftarrow \mathbb{F}^{\log \ell + 1}, (\alpha_1, \dots, \alpha_{\log \ell}) \leftarrow \mathbb{F}^{\log \ell}, \alpha_{\log \ell + 1} \leftarrow \mathbb{F} \setminus \{0, 1\}, \rho \leftarrow \mathbb{F}$.
2. Sample $(v_A, v_B, v_C) \leftarrow \mathbb{F}^3$.
3. Sample $(\tilde{\mu}, (\hat{h}_j)_{j \in [\log \ell + 1]}, (\tilde{\mu}_j)_{j \in [\log \ell + 1]})$ uniformly from $\mathcal{T}(v_A, v_B, v_C)$ (defined below).
4. Compute the explicit output instance $\mathbb{x}$ exactly as in Items 9 and 10, using the sampled values (v_A, v_B, v_C) and the public quantities $(u_M)_{M \in \{A,B,C\}}$ computed from (A, B, C, v) and α , and setting

$$\mu := \sum_{M \in \{A,B,C\}} \text{ze}(\rho)_M \cdot (v_M - u_M).$$

(Also set all inner and outer targets/states as in the construction.)

5. Run $\mathcal{D}$ on $\mathbb{x}$, giving it oracle access to $\mathbb{y} = (f, (\bar{\xi}_{M,i})_{M,i}, (\xi_j)_j)$, and answer its (adaptive) oracle queries as follows:
 - (a) for each (M, i) , answer queries to $\bar{\xi}_{M,i}$ using $\mathbf{S}_{\mathcal{C}_{\text{in}}^{\text{zk}}}$;

- (b) for each j , answer queries to ξ_j using $\mathbf{S}_{\text{out}}^{\text{zk}}$;
 - (c) answer queries to f using $\mathbf{S}_C$.
6. Output the simulated view (including $\mathcal{D}$'s query/answer transcript).

Fix verifier randomness $(\varepsilon, \mathbf{r}, \alpha, \rho)$. For any fixed triple $(v_A, v_B, v_C) \in \mathbb{F}^3$, define $\mathcal{T}(v_A, v_B, v_C)$ to be the set of all tuples $(\tilde{\mu}, (\hat{h}_j)_{j \in [\log \ell + 1]}, (\tilde{\mu}_j)_{j \in [\log \ell + 1]})$ that satisfy

$$\begin{aligned} \hat{h}_1(0) + \hat{h}_1(1) &= \tilde{\mu}, \\ \forall j \in \{2, \dots, \log \ell + 1\} : \quad \hat{h}_j(0) + \hat{h}_j(1) &= \hat{h}_{j-1}(\alpha_{j-1}), \end{aligned}$$

and

$$\varepsilon \cdot (v_A \cdot v_B - v_C) \cdot \text{eq}(\mathbf{r}, \alpha) + \sum_{j=1}^{\log \ell + 1} \tilde{\mu}_j = \hat{h}_{\log \ell + 1}(\alpha_{\log \ell + 1}).$$

For fixed (v_A, v_B, v_C) , these are linear constraints in the coefficients of $(\hat{h}_j)_j$ and in $(\tilde{\mu}, \tilde{\mu}_1, \dots, \tilde{\mu}_{\log \ell + 1})$, hence $\mathcal{T}(v_A, v_B, v_C)$ is the solution set of a linear system (i.e., an affine subspace). Also, $\mathcal{T}(v_A, v_B, v_C) \neq \emptyset$ for every (v_A, v_B, v_C) : choose $\hat{h}_1, \dots, \hat{h}_{\log \ell + 1}$ recursively so that $\hat{h}_j(0) + \hat{h}_j(1) = \hat{h}_{j-1}(\alpha_{j-1})$ (with $\hat{h}_1(0) + \hat{h}_1(1) = \tilde{\mu}$), set $\tilde{\mu}_1 = \dots = \tilde{\mu}_{\log \ell} = 0$, and then define $\tilde{\mu}_{\log \ell + 1}$ to satisfy the final equation. Hence the simulator's sampling in Item 3 is well-defined.

We compare distributions in two claims, conditioning on verifier randomness.

Outer transcript claim. Fix the prover's inner-layer randomness (thus fixing $\hat{g}$ and the values (v_A, v_B, v_C) from Item 8). In the honest execution, the tuple $(\tilde{\mu}, (\hat{h}_j)_j, (\tilde{\mu}_j)_j)$ is an affine function of the outer-mask coefficients, and the verifier's checks are exactly the constraints defining $\mathcal{T}(v_A, v_B, v_C)$. Under $\ell_{\text{out}}^{\text{zk}} \geq 2 \cdot \ell_{\text{in}}^{\text{zk}}$ and $\ell_{\text{in}}^{\text{zk}} \geq 4$, the same linear-algebra argument as in Lemma 6.4 gives: the image is $\mathcal{T}(v_A, v_B, v_C)$, and every point in $\mathcal{T}(v_A, v_B, v_C)$ has the same number of preimages. Therefore uniform outer masks induce the uniform distribution on $\mathcal{T}(v_A, v_B, v_C)$, which matches the simulator's choice in Item 3.

Value claim. Using the decomposition in Item 8, in the honest execution

$$v_M = \sum_{b \in \{0,1\}^{\log \ell}} \left(\hat{f}_M(\alpha, b, 0) \cdot \hat{f}_v(b) + \hat{f}_M(\alpha, b, 1) \cdot \hat{f}_w(b) \right) + 2 \cdot \sum_{i=1}^{\log \ell + 1} \bar{\xi}_{M,i}(\alpha_i) \quad (M \in \{A, B, C\}),$$

where each $\bar{\xi}_{M,i}$ is sampled uniformly subject to $\bar{\xi}_{M,i}(0) = \bar{\xi}_{M,i}(1) = 0$. For each M , condition on all inner masks except $\bar{\xi}_{M, \log \ell + 1}$, and rewrite

$$v_M = \text{const}_M + 2 \cdot \bar{\xi}_{M, \log \ell + 1}(\alpha_{\log \ell + 1}).$$

Because $\alpha_{\log \ell + 1} \in \mathbb{F} \setminus \{0, 1\}$ and $\ell_{\text{in}}^{\text{zk}} \geq 4$, the value $\bar{\xi}_{M, \log \ell + 1}(\alpha_{\log \ell + 1})$ is uniform over $\mathbb{F}$; and because $\text{char}(\mathbb{F}) \neq 2$, multiplication by 2 is a bijection on $\mathbb{F}$. Hence each v_M is uniform over $\mathbb{F}$ conditioned on public randomness and witness, and by independence of the three masks at index $\log \ell + 1$, the triple (v_A, v_B, v_C) is jointly uniform over $\mathbb{F}^3$ and independent of witness/input. This matches the simulator's choice in Item 2.

Combining the two claims, the sumcheck part is perfectly simulated.

Finally, we replace the honest encodings of the oracles $(\bar{\xi}_{M,i})_{M,i}$, $(\xi_j)_j$, and f , one oracle at a time, by $\mathbf{S}_{\text{in}}^{\text{zk}}$, $\mathbf{S}_{\text{out}}^{\text{zk}}$, and $\mathbf{S}_C$ respectively (similarly to the hybrid argument in Lemma 6.4). Each replacement changes $\mathcal{D}$'s view by at most ζ in statistical distance, because each encoding is t -query zero-knowledge and $\mathcal{D} \in \mathcal{D}^{\leq t}$. There are exactly $3(\log \ell + 1) + (\log \ell + 1) + 1 = 4\log \ell + 5$ such oracles, so by a union bound the simulation error is at most $(4\log \ell + 5) \cdot \zeta = O(\log \ell \cdot \zeta)$. $\square$

12 High-distance codes from dispersers

We discuss the use of distance amplification via disperser graphs [ABNNR92; GRS12], and its use for certain efficiency gains.

12.1 Preliminaries

The following two definitions and subsequent proposition are from [GRS12].

Definition 12.1. Let $G = (L \cup R, E)$ be a bipartite graph with $|L| = |R| = m$. We say that G is a (ε, γ) -**disperser** if for every $S \subseteq L$ with $|S| \geq (1 - \varepsilon)m$ the neighborhood $N(S) := \{j \in R \mid \exists i \in S : (i, j) \in E\}$ satisfies $|N(S)| \geq (1 - \gamma)|R| = (1 - \gamma)m$.

Definition 12.2. Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a code with minimum distance $\delta(\mathcal{C}) \geq 1 - \varepsilon$. Let $G = (L \cup R, E)$ be a (ε, γ) -disperser with $|L| = |R| = m$, and assume G is right-regular of degree d . For every $j \in R$, fix an ordered list $(i_{j,1}, i_{j,2}, \dots, i_{j,d}) \in L^d$ of its d left incidences (allowing repetitions when G has multi-edges). Define the **amplification map** $A_G: \mathbb{F}^m \rightarrow (\mathbb{F}^d)^m$ by $(A_G(c))_j := (c_{i_{j,1}}, \dots, c_{i_{j,d}})$ for each $j \in R$.

Proposition 12.3 (Code-amplification lemma). Under the hypotheses of Definition 12.2, the amplified code $G(\mathcal{C}) := \{A_G(c) \mid c \in \mathcal{C}\} \subseteq (\mathbb{F}^d)^m$ has minimum distance $\delta(G(\mathcal{C})) \geq 1 - \gamma$.

Proof. Take distinct $c, c' \in \mathcal{C}$ and let $S := \{i \in L \mid c_i \neq c'_i\}$. Since $\delta(\mathcal{C}) \geq 1 - \varepsilon$, we have $|S| \geq (1 - \varepsilon)m$ and hence $|N(S)| \geq (1 - \gamma)m$ by Definition 12.1. For any $j \in N(S)$ there exists $k \in \{1, \dots, d\}$ with $i_{j,k} \in S$, so the j -th symbols of $A_G(c)$ and $A_G(c')$ differ in coordinate k of the d -tuple. Therefore the number of right indices where the amplified codewords differ is at least $|N(S)| \geq (1 - \gamma)m$. $\square$

Below we show via a standard probabilistic method argument that random graphs are good dispersers (similar lemmas can be found in [Coh91; TUZ07]). The edges of the graph are sampled at random such that each right vertex has d left-neighbors. The following gives a lower bound on the required degree to guarantee the parameters of the disperser.

Proposition 12.4 (Probabilistic sampling of dispersers). Let L, R be vertex sets with $|L| = |R| = m$. Construct a random bipartite graph $G = (L \cup R, E)$ (with multi-edges) by choosing, for each $j \in R$, a multiset of d neighbors in L uniformly at random with replacement (right-degree d). If

$$d > \frac{\lceil (1 - \varepsilon)m \rceil \ln \frac{me}{\lceil (1 - \varepsilon)m \rceil} + \lceil \gamma m \rceil \ln \frac{me}{\lceil \gamma m \rceil} - \ln \alpha}{\lceil \gamma m \rceil \cdot (-\ln \varepsilon)},$$

then, with probability at least $1 - \alpha$, the graph G is a (ε, γ) -disperser.

Proof. G fails to be a (ε, γ) -disperser iff there exist $S \subseteq L$ and $T \subseteq R$ with $|S| \geq \lceil (1 - \varepsilon)m \rceil$ and $|T| \geq \lceil \gamma m \rceil$ such that every neighbor of each $j \in T$ lies in $L \setminus S$, i.e., $N(S) \subseteq R \setminus T$ (equivalently, $|N(S)| \leq (1 - \gamma)m$). We may assume $|S| = \lceil (1 - \varepsilon)m \rceil$ and $|T| = \lceil \gamma m \rceil$.

Fix S, T of sizes $|S| = \lceil (1 - \varepsilon)m \rceil$ and $|T| = \lceil \gamma m \rceil$. Let $p := \frac{m - |S|}{m} \leq \varepsilon$ be the probability that a single random neighbor (one draw into L) lands in $L \setminus S$. For a fixed $j \in T$, the probability that all d independently sampled neighbors (with replacement) lie in $L \setminus S$ is at most $p^d \leq \varepsilon^d$.

Independence across $j \in T$ yields $\Pr[\text{all neighbors of all } j \in T \text{ avoid } S] \leq (\varepsilon^d)^{|T|} = \varepsilon^{d\lceil \gamma m \rceil}$. By a union bound over all choices of S and T of these sizes, $\Pr[\text{failure}] \leq \binom{m}{\lceil (1-\varepsilon)m \rceil} \binom{m}{\lceil \gamma m \rceil} \varepsilon^{d\lceil \gamma m \rceil}$. Using $\binom{N}{k} \leq (\frac{Ne}{k})^k$ gives $\Pr[\text{failure}] \leq \left(\frac{me}{\lceil (1-\varepsilon)m \rceil}\right)^{\lceil (1-\varepsilon)m \rceil} \left(\frac{me}{\lceil \gamma m \rceil}\right)^{\lceil \gamma m \rceil} \varepsilon^{d\lceil \gamma m \rceil}$. We conclude by making the right-hand side at most α . $\square$

Use in our protocols. Our protocols can be instantiated with any $\mathbb{F}$ -additive code with a zero-knowledge encoding, including distance-amplified codes. Although amplified codewords are over a larger alphabet, they remain $\mathbb{F}$ -additive: a vector in $(\mathbb{F}^d)^m$ is equivalently a vector in $\mathbb{F}^{d \cdot m}$. There are generic results on mutual correlated agreement for every $\mathbb{F}$ -additive code [GKL24; GCXK25; BCGM25]. Moreover, if $\mathcal{C} \subseteq \mathbb{F}^m$ has a t -query zero-knowledge encoding $\text{Enc}_{\mathcal{C}}: \mathbb{F}^\ell \times \mathbb{F}^r \rightarrow \mathbb{F}^m$, then the amplified code $G(\mathcal{C}) = \{A_G(c) : c \in \mathcal{C}\}$ has a $\lfloor t/d \rfloor$ -query zero-knowledge encoding: each query to one amplified symbol reveals d base-code symbols. In sum, distance-amplified codes can be used in our constructions both as the zero-knowledge masking code $\mathcal{C}_{zk}$ and as the main code $\mathcal{C}$ (for example in Theorem 8.1).

12.2 Amplified Reed–Solomon code

We describe how distance amplification can be used to improve the encoding time of codes, in particular Reed–Solomon codes, given any target distance. The distance of the codes used in the protocols plays a crucial role in the number of queries made (and hence the resulting argument size after compilation via Merkle commitments), and it is beneficial to have large distance (that is, to have a code with distance $1 - \gamma$ with small γ).

In general, distance amplification allows one to take any code, and obtain any distance using graphs with sufficiently large degrees (which correspond to sufficiently large alphabet for the output code). In the following, we highlight the resulting code from distance amplification on the Reed–Solomon code, which allows for faster encoding for a target distance.

Definition 12.5 (Amplified Reed–Solomon code). *Fix $c > 1$ and a message length $\ell \in \mathbb{N}$. Set $m := \lceil c\ell \rceil$, choose a field $\mathbb{F}$ with $|\mathbb{F}| \geq m$, and fix an evaluation domain $[m] \subseteq \mathbb{F}$ of size m . Let $L := [m]$ and let $G = (L \cup R, E)$ be a right-regular bipartite graph of degree d that is a (ε, γ) -dispenser with $\varepsilon := \frac{\ell-1}{m}$. The code $\text{ARS}[\mathbb{F}, m, \ell, G]$ is defined via this encoding: for a message $\text{msg} \in \mathbb{F}^\ell$, set $u := \text{Enc}_{\text{RS}}(\text{msg}) \in \text{RS}[\mathbb{F}, m, \ell]$ and output $A_G(u) \in (\mathbb{F}^d)^{|R|}$ (following Definition 12.2).*

The following claim follows by Definition 12.2 and Proposition 12.3.

Claim 12.6 (Parameters of the amplified RS code). *The code in Definition 12.5 has alphabet $\mathbb{F}^d$, block length $|R|$ (in alphabet symbols), and minimum distance at least $1 - \gamma$.*

Encoding time for two ways to reach relative distance $1 - \gamma$. We want to encode a message $\text{msg} \in \mathbb{F}^\ell$ into a codeword with minimum distance at least $1 - \gamma$. We compare two methods.

1. Plain Reed–Solomon: Choose an evaluation domain of size $\frac{\ell-1}{\gamma}$ (and a field $\mathbb{F}$ large enough) so that $\delta(\text{RS}[\mathbb{F}, \frac{\ell-1}{\gamma}, \ell]) \geq 1 - \gamma$. Encoding msg consists of (i) interpolating the degree- $(\ell - 1)$ polynomial from the ℓ message symbols, and (ii) evaluating it at all points of the chosen domain. Assuming FFT-style algorithms on $[m]$, this requires at most

$$c_{\text{FFT}} \cdot \ell \cdot \log_2 \ell + c_{\text{FFT}} \cdot \left(\frac{\ell-1}{\gamma}\right) \cdot \log_2 \left(\frac{\ell-1}{\gamma}\right)$$

field operations. (If Enc_{RS} treats the message as polynomial coefficients, omit the interpolation term and keep only the evaluation term.)

2. Amplified Reed–Solomon (with factor $c := m/\ell$ and a right-regular disperser of degree d from the main bound): First encode $\text{msg} \in \mathbb{F}^\ell$ as a Reed–Solomon codeword on an evaluation domain of size $c \cdot \ell$; this takes at most $c_{\text{FFT}} \cdot \ell \cdot \log_2 \ell + c_{\text{FFT}} \cdot (c \cdot \ell) \cdot \log_2 (c \cdot \ell)$ field operations (again, drop the interpolation term if the message is given as coefficients). Then apply the amplification map A_G using the disperser graph; this step performs $d \cdot |R|$ copy operations. If, as in Definition 12.5, one sets $|R| = c \cdot \ell$, this amplification step costs $d \cdot c \cdot \ell$ symbols. Overall, the encoding time amounts to:

$$c_{\text{FFT}} \cdot \ell \cdot \log_2 \ell + c_{\text{FFT}} \cdot (c \cdot \ell) \cdot \log_2 (c \cdot \ell) + d \cdot c \cdot \ell.$$

For the amplified option, we instantiate the disperser degree at the smallest integer strictly larger than the lower bound in Proposition 12.4 with $n := c \cdot \ell$, $\varepsilon := \frac{\ell-1}{c \cdot \ell}$, γ as in the target distance, and any fixed $\alpha \in (0, 1)$ for the construction’s failure probability. Concretely, take d to be the least integer bigger than

$$\frac{(1 - \varepsilon)n \cdot \ln \frac{ne}{(1-\varepsilon)n} + \gamma n \cdot \ln \frac{ne}{\gamma n} - \ln \alpha}{\gamma n \cdot (-\ln \varepsilon)} \quad \text{with} \quad n = c \cdot \ell, \quad \varepsilon = \frac{\ell - 1}{c \cdot \ell}.$$

With this choice, the amplified variant is faster than plain RS exactly when

$$\underbrace{c_{\text{FFT}} \cdot (c \cdot \ell) \cdot \log_2 (c \cdot \ell)}_{\text{RS on size } c \cdot \ell} + \underbrace{d \cdot c \cdot \ell}_{\text{amplification copies}} \leq \underbrace{c_{\text{FFT}} \cdot \left(\frac{\ell - 1}{\gamma} \right) \cdot \log_2 \left(\frac{\ell - 1}{\gamma} \right)}_{\text{RS on size } (\ell-1)/\gamma}.$$

Intuitively, when γ is small (demanding very large plain-RS block length), the right-hand side grows like $\frac{\ell}{\gamma} \log_2 \frac{\ell}{\gamma}$, while the extra cost of amplification grows only linearly in $\frac{\ell}{\gamma}$ through $d \cdot c \cdot \ell$ (since d from the lemma scales like $\frac{(1-\frac{1}{c}) \ln \frac{e}{1-\frac{1}{c}} + \gamma \ln \frac{e}{\gamma}}{\gamma \ln c}$ with $n = c \cdot \ell$). Thus, for small γ and fixed c , the amplified RS method is cheaper.

Benefits of amplified RS. Amplified Reed–Solomon codes offer benefits in time and rate.

- *No large FFTs.* The FFT is for size $\Theta(\ell)$ rather than $\Theta(\ell/\gamma)$. This improves encoding time, by trading a $\Theta(\ell/\gamma)$ -size FFT for a $\Theta(c\ell)$ -size FFT plus a $\Theta(\ell/\gamma)$ -time pass to gather neighbors.
- *Space complexity.* The *encoding space* required for ARS is also decreased, if one has efficient access to the graph G . Even if one were to use in-place FFTs, the encoding space of the Reed–Solomon code at that distance is ℓ/γ , while we can afford smaller FFTs requiring only $O(\ell)$ size. Storing G in memory takes $O(\frac{\ell}{\gamma} \log \ell)$, making the space complexity slightly *worse* than in-place FFT implementations. Since strongly-explicit dispersers (where the neighborhood of a vertex could be computed in polylogarithmic time) with $d = O(1/\gamma)$ are not known, one would have to store the sampled G in memory. An alternative approach would be to *stream* G , performing the alphabet aggregation for the neighbors of each vertex when their neighborhoods are streamed. This streaming option is much less clear for FFTs. Furthermore, in practice, one could use a PRG that, in principle, replaces the notion of a strongly explicit construction, as the neighbor set of each vertex could be sampled “on the fly” in a reproducible way.

12.3 Distance amplification and Merkle-compiled argument size

We discuss potential efficiency gains from distance amplification for argument size. While the proposed application here increases the encoding time (as opposed to the decrease in encoding time mentioned in the previous section), we show that it decreases overall argument size.

A case study. To investigate this point, we consider an example, which is based on the codeword batching protocol from [BCFW25]. In this protocol, the prover sends as an oracle a purported codeword, and the verifier makes t independent spot-checks to an oracle. When the protocol’s soundness relies on the fact that two distinct codewords disagree on a $1 - \varepsilon$ fraction of coordinates (i.e., the underlying code has relative distance at least $1 - \varepsilon$), then each independent check catches a cheating strategy with probability at least $1 - \varepsilon$, and hence the overall soundness error is at most ε^t . Thus, to drive the soundness error below $2^{-\lambda}$, it suffices to take $t \geq \frac{\lambda}{-\log_2 \varepsilon}$.

The cost of a Merkle commitment. Consider compiling such a protocol into an argument by committing to the codeword via a Merkle commitment whose leaves are the codeword symbols. Each verifier query is then answered by the leaf value and the authentication path, i.e., the sibling hashes along the root-to-leaf path. For a codeword of block length m (in symbols), the path has length $\lceil \log_2 m \rceil$. Hence, measured as “one field symbol plus one hash digest” units, the total proof communication contributed by these t oracle queries is $t \cdot (1 + \lceil \log_2 m \rceil)$, up to constant factors that depend only on the fixed hash function.

Merkle compilation cost for an amplified code. Now replace the underlying code $\mathcal{C} \subseteq \mathbb{F}^m$ by its amplified version $G(\mathcal{C}) \subseteq (\mathbb{F}^d)^m$ from Definition 12.2, where G is a right-regular (ε, γ) -disperser of degree d and $|R| = m$. By Proposition 12.3, the amplified code has relative distance at least $1 - \gamma$, so the same “independent spot-check” reasoning yields soundness error at most γ^t , and to achieve soundness error below $2^{-\lambda}$ it suffices to take $t \geq \frac{\lambda}{-\log_2 \gamma}$. After compilation, each query now opens a *single amplified symbol*, i.e., an element of $\mathbb{F}^d$, together with an authentication path of length $\lceil \log_2 m \rceil$. Hence the argument size contributed by the t oracle queries becomes $t \cdot (d + \lceil \log_2 m \rceil)$.

In the disperser-based setting of Definitions 12.1 and 12.2, the Merkle depth terms coincide and amplification yields an improvement for this part of the argument when

$$d < \left(1 + \lceil \log_2 m \rceil\right) \cdot \frac{-\log_2 \gamma}{-\log_2 \varepsilon} - \lceil \log_2 m \rceil. \quad (10)$$

Instantiating d via Proposition 12.4. Fix any target $\gamma \in (0, 1)$ and any $\alpha \in (0, 1)$. Let d be the least integer strictly larger than the lower bound from Proposition 12.4. To assess whether Equation 10 can hold for the *best* (i.e., smallest) degree guaranteed by Proposition 12.4, take d to be the least integer strictly larger than the bound of Proposition 12.4. Ignoring ceiling effects and the additive $-\ln \alpha$ term (which is divided by γm and is negligible once m is large), the bound has the following dominant form:

$$d \approx \frac{(1 - \varepsilon) \ln \frac{e}{1 - \varepsilon}}{\gamma \cdot (-\ln \varepsilon)} + \frac{\ln \frac{e}{\gamma}}{-\ln \varepsilon}.$$

In particular, for fixed ε this degree grows proportionally to $1/\gamma$ as $\gamma \rightarrow 0$. Since the required number of openings decreases only as $1/(-\log_2 \gamma)$, very small γ forces d so large that Equation 10 eventually fails: in this regime, amplifying to extremely high relative distance makes each Merkle leaf so large that the reduction in the number of openings cannot compensate.

Concrete parameters. The condition in Equation 10 *can* hold for moderate target distances. For instance, take $m = 2^{20}, \alpha = 2^{-40}, \lambda = 128, \varepsilon = \frac{1}{3}$. Then the plain compilation needs $t \approx 81$ openings and the query/opening contribution is approximately $81 \cdot (1 + 20) \approx 1.70 \cdot 10^3$ units. If we target $\gamma = 0.1$ (relative distance 0.9), the smallest d certified by Proposition 12.4 is $d = 12$, and the amplified compilation needs $t \approx 39$ openings, contributing about $39 \cdot (12 + 20) \approx 1.23 \cdot 10^3$ units, a reduction of roughly 27%. In contrast, if we target $\gamma = 0.01$ (relative distance 0.99), the same calculation yields $d = 91$ and $t \approx 19$, giving about $19 \cdot (91 + 20) \approx 2.14 \cdot 10^3$ units, which is *larger* than plain. These examples illustrate that, under Proposition 12.4, the Merkle-query part improves only when the requested distance increase is substantial but not so extreme that the required d becomes the dominant term in each opening.

We compare the prover’s slowdown when using distance amplification. To compare *prover* runtime, we separate (a) committing to the oracle (encoding, plus building the Merkle commitment) from (b) answering the t openings. In the comparison we ignore the base encoding time of $\mathcal{C}$ (it is the same in both variants) and we count the total number of field-symbol copies/hashes.

As before, we assume that $m = 2^{20}$ so the Merkle commitment has 2^{20} leaves and $2^{20} - 1$ internal nodes. In the *plain* case, committing to $\mathcal{C} \subseteq \mathbb{F}^m$ requires hashing 2^{20} leaves (each containing one field symbol) and $2^{20} - 1$ internal nodes, i.e., about $(2^{20}) + (2^{20} - 1) = 2,097,151$ hash computations. Answering $t \approx 81$ openings transmits 81 field symbols and $81 \cdot 20$ sibling hashes, i.e., $81 \cdot (1 + 20) = 1,701$ units in the same model. Overall, the plain prover’s Merkle-related work is $\approx 2,098,852$ units.

For $\gamma = 0.1$, the probabilistic-method bound yields $d = 12$ and $t \approx 39$. In the *amplified* case, beyond the same base encoding, the prover must (i) compute A_G (about $dm = 12 \cdot 2^{20} = 12,582,912$ symbol copies), and (ii) hash 2^{20} leaves each containing $d = 12$ field symbols (about another 12,582,912 symbol-hash units), plus the same $2^{20} - 1$ internal-node hashes. Answering the openings transmits 39 leaves of size 12 plus $39 \cdot 20$ sibling hashes, i.e., $39 \cdot (12 + 20) = 1,248$ units. Altogether, this totals roughly $12,582,912 + 12,582,912 + (2^{20} - 1) + 1,248 = 26,215,647$ units, i.e., about a 12.5 $\times$ increase over the plain prover time in this model. In other words, even though the *communication* in openings improves (roughly 27% above), the *prover* becomes slower because it must materialize and commit to d -times larger leaves.

Acknowledgments

The authors are supported in part by the Ethereum Foundation, the Sui Foundation, and Polygon Labs.

References

- [ABNNR92] N. Alon, J. Bruck, J. Naor, M. Naor, and R.M. Roth. “Construction of asymptotically good low-rate error-correcting codes through pseudo-random graphs”. In: *IEEE Transactions on Information Theory* 38.2 (1992), pp. 509–516. DOI: 10.1109/18.119713.
- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “STIR: Reed–Solomon Proximity Testing with Fewer Queries”. In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO ’24. 2024, pp. 380–413.
- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification”. In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EURO-CRYPT ’25. 2025.
- [ACY23] Gal Arnon, Alessandro Chiesa, and Eylon Yogev. “IOPs with Inverse Polynomial Soundness Error”. In: *64th IEEE Annual Symposium on Foundations of Computer Science , FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023*. IEEE, 2023, pp. 752–761.
- [AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup”. In: *Proceedings of the 24th ACM Conference on Computer and Communications Security*. CCS ’17. 2017, pp. 2087–2104.
- [ARR25] Noor Athamnah, Noga Ron-Zewi, and Ron D. Rothblum. “Linear Prover IOPs in Log Star Rounds”. In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC ’25. 2025, pp. 335–368.
- [BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. “Fast Reed–Solomon Interactive Oracle Proofs of Proximity”. In: *Proceedings of the 45th International Colloquium on Automata, Languages and Programming*. ICALP ’18. 2018, 14:1–14:17.
- [BBHR19] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. “Scalable Zero Knowledge with No Trusted Setup”. In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO ’19. 2019, pp. 733–764.
- [BBHV22] Laasya Bangalore, Rishabh Bhadauria, Carmit Hazay, and Muthuramakrishnan Venkitasubramaniam. “On Black-Box Constructions of Time and Space Efficient Sublinear Arguments from Symmetric-Key Primitives”. In: *Proceedings of the 20th Theory of Cryptography Conference*. TCC ’22. 2022, pp. 417–446.
- [BCFFMMZ25] Anubhav Baweja, Alessandro Chiesa, Elisabetta Fedele, Giacomo Fenzi, Pratyush Mishra, Tushar Mopuri, and Andrew Zitek-Estrada. “Time-Space Tradeoffs for Sumcheck”. In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC ’25. 2025, pp. 37–70.
- [BCFGRS17] Eli Ben-Sasson, Alessandro Chiesa, Michael A. Forbes, Ariel Gabizon, Michael Riabzev, and Nicholas Spooner. “Zero Knowledge Protocols from Succinct Constraint Detection”. In: *Proceedings of the 15th Theory of Cryptography Conference*. TCC ’17. 2017, pp. 172–206.

- [BCFRRZ25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. “Blaze: Fast SNARKs from Interleaved RAA Codes”. In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’25. 2025.
- [BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. “Linear-Time Accumulation Schemes”. In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC ’25. 2025, pp. 369–399.
- [BCG20] Jonathan Bootle, Alessandro Chiesa, and Jens Groth. “Linear-Time Arguments with Sub-linear Verification from Tensor Codes”. In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC ’20. 2020, pp. 19–46.
- [BCGGHJ17] Jonathan Bootle, Andrea Cerulli, Essam Ghadafi, Jens Groth, Mohammad Hajiabadi, and Sune K. Jakobsen. “Linear-Time Zero-Knowledge Proofs for Arithmetic Circuit Satisfiability”. In: *Proceedings of the 23rd International Conference on the Theory and Applications of Cryptology and Information Security*. ASIACRYPT ’17. 2017, pp. 336–365.
- [BCGGRS19] Eli Ben-Sasson, Alessandro Chiesa, Lior Goldberg, Tom Gur, Michael Riabzev, and Nicholas Spooner. “Linear-Size Constant-Query IOPs for Delegating Computation”. In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC ’19. 2019, pp. 494–521.
- [BCGM25] Sarah Bordage, Alessandro Chiesa, Ziyi Guan, and Ignacio Manzur. *All Polynomial Generators Preserve Distance with Mutual Correlated Agreement*. Cryptology ePrint Archive, Paper 2025/2051. 2025.
- [BCGV16] Eli Ben-Sasson, Alessandro Chiesa, Ariel Gabizon, and Madars Virza. “Quasilinear-Size Zero Knowledge from Linear-Algebraic PCPs”. In: *Proceedings of the 13th Theory of Cryptography Conference*. TCC ’16-A. 2016, pp. 33–64.
- [BCIKS20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. “Proximity Gaps for Reed–Solomon Codes”. In: *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’20. 2020, pp. 900–909.
- [BCL22] Jonathan Bootle, Alessandro Chiesa, and Siqi Liu. “Zero-Knowledge IOPs with Linear-Time Prover and Polylogarithmic-Time Verifier”. In: *Proceedings of the 41st Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’22. 2022, pp. 275–304.
- [BCRSVW19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. “Aurora: Transparent Succinct Arguments for R1CS”. In: *Proceedings of the 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’19. 2019, pp. 103–128.
- [BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. “Interactive Oracle Proofs”. In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC ’16-B. 2016, pp. 31–60.
- [BFRW25] Benedikt Bünz, Giacomo Fenzi, Ron D. Rothblum, and William Wang. *TensorSwitch: Nearly Optimal Polynomial Commitments from Tensor Codes*. Cryptology ePrint Archive, Report 2025/2065. 2025.
- [BGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. “DEEP-FRI: Sampling Outside the Box Improves Soundness”. In: *Proceedings of the 11th Innovations in Theoretical Computer Science Conference*. ITCS ’20. 2020, 5:1–5:32.
- [BMMS25a] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. *FICS and FACS: Fast IOPs and Accumulation via Code-Switching*. Cryptology ePrint Archive, Report 2025/737. 2025.

- [BMMS25b] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. *Query-Optimal IOPPs for Linear-Time Encodable Codes*. Cryptology ePrint Archive, Report 2025/1588. 2025.
- [BMNW25] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. “Arc: Accumulation for Reed–Solomon Codes”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25. 2025.
- [Ber] Daniel J. Bernstein. *The transposition principle*. URL: <https://cr.yp.to/transposition.html>.
- [Bor57] Jan L. Bordewijk. “Inter-reciprocity applied to electrical networks”. In: *Applied Scientific Research, Section B* 6.1 (1957), pp. 1–74.
- [CCHLRR18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. *Fiat–Shamir From Simpler Assumptions*. Cryptology ePrint Archive, Report 2018/1004. 2018.
- [CFS17] Alessandro Chiesa, Michael A. Forbes, and Nicholas Spooner. *A Zero Knowledge Sumcheck and its Applications*. <https://arxiv.org/abs/1704.02086>. arXiv:1704.02086 [cs.CC]. 2017. DOI: 10.48550/arXiv.1704.02086.
- [CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. “Succinct Arguments in the Quantum Random Oracle Model”. In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC ’19. 2019, pp. 1–29.
- [COS20] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. “Fractal: Post-Quantum and Transparent Recursive Proofs from Holography”. In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EURO-CRYPT ’20. 2020, pp. 769–793.
- [CTY11] Graham Cormode, Justin Thaler, and Ke Yi. “Verifying computations with streaming interactive proofs”. In: *Proceedings of the VLDB Endowment* 5.1 (2011), pp. 25–36.
- [CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL: <https://github.com/hash-based-snargs-book>.
- [Coh91] Aviad Cohen. “Disperser Graphs, Deterministic Amplification and Imperfect Random Sources”. PhD Thesis, PDF available online at <https://www.math.ias.edu/~avi/STUDENTS/acthesis.pdf>. PhD thesis. Hebrew University of Jerusalem, 1991.
- [DL25] Hugo Delavenne and Louise Lallemand. *Codes on any Cayley Graph have an Interactive Oracle Proof of Proximity*. arXiv preprint. 2025. arXiv: 2508.10510 [cs.CC].
- [DM11] Irit Dinur and Or Meir. “Derandomized Parallel Repetition via Structured PCPs”. In: *Computational Complexity* 20.2 (2011), pp. 207–327. DOI: 10.1007/s00037-011-0013-5. URL: <https://link.springer.com/article/10.1007/s00037-011-0013-5>.
- [DMR25] Hugo Delavenne, Tanguy Medevielle, and Élina Roussel. “Interactive Oracle Proofs of Proximity to Codes on Graphs”. In: *2025 IEEE International Symposium on Information Theory*. ISIT ’25. 2025, pp. 1–6.
- [DP24] Benjamin E. Diamond and Jim Posen. *Proximity Testing with Logarithmic Randomness*. Cryptology ePrint Archive, Report 2023/630. 2024.
- [DR04] Irit Dinur and Omer Reingold. “Assignment Testers: Towards a Combinatorial Proof of the PCP Theorem”. In: *Proceedings of the 45th Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’04. 2004, pp. 155–164.
- [Dia25] Benjamin Diamond. *Zero-Knowledge Polynomial Commitment in Binary Fields*. Cryptology ePrint Archive, Report 2025/1015. 2025.

- [Din07] Irit Dinur. “The PCP theorem by gap amplification”. In: *Journal of the ACM* 54.3 (2007), p. 12.
- [Fs24] Matteo Frigo and abhi shelat. *Anonymous credentials from ECDSA*. Cryptology ePrint Archive, Report 2024/2010. 2024.
- [GCXK25] Yiwen Gao, Dongliang Cai, Yang Xu, and Haibin Kan. *From List-Decodability to Proximity Gaps*. Tech. rep. 2025/870. Cryptology ePrint Archive, 2025. URL: <https://eprint.iacr.org/2025/870>.
- [GGR11] Parikshit Gopalan, Venkatesan Guruswami, and Prasad Raghavendra. “List Decoding Tensor Products and Interleaved Codes”. In: *SIAM Journal on Computing* 40.5 (2011), pp. 1432–1462.
- [GI05] Venkatesan Guruswami and Piotr Indyk. “Linear-time encodable/decodable codes with near-optimal rate”. In: *IEEE Transactions on Information Theory* 51.10 (2005). Preliminary version appeared in STOC ’03., pp. 3393–3400.
- [GKL24] Yiwen Gao, Haibin Kan, and Yuan Li. *Linear Proximity Gap for Linear Codes within the 1.5 Johnson Bound*. Cryptology ePrint Archive, Report 2024/1810. 2024.
- [GLSTW23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. “Brakedown: Linear-time and field-agnostic SNARKs for R1CS”. In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO ’23. 2023.
- [GOS24] Tom Gur, Jack O’Connor, and Nicholas Spooner. “Perfect Zero-Knowledge PCPs for #P”. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. STOC ’24. 2024, pp. 1724–1730.
- [GOS25] Tom Gur, Jack O’Connor, and Nicholas Spooner. “A Zero-Knowledge PCP Theorem”. In: *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*. STOC ’25. 2025, pp. 986–994.
- [GRS12] Venkatesan Guruswami, Atri Rudra, and Madhu Sudan. *Essential coding theory*. Vol. 2. 1. 2012.
- [GS00] Oded Goldreich and Shmuel Safra. “A Combinatorial Consistency Lemma with Application to Proving the PCP Theorem”. In: *SIAM Journal on Computing* 29.4 (2000). Earlier version: FOCS 1997, pp. 1132–1154. DOI: 10.1137/S0097539797315744.
- [GUV09] Venkatesan Guruswami, Christopher Umans, and Salil P. Vadhan. “Unbalanced expanders and randomness extractors from Parvaresh–Vardy codes”. In: *Journal of the ACM* 56.4 (2009), 20:1–20:34.
- [Goo] *Opening up ‘Zero-Knowledge Proof’ technology to promote privacy in age assurance*. <https://blog.google/technology/safety-security/opening-up-zero-knowledge-proof-technology-to-promote-privacy-in-age-assurance/>. 2025.
- [HK24] Ulrich Haböck and Al Kindi. *A note on adding zero-knowledge to STARK*. Cryptology ePrint Archive, Report 2024/1037. 2024.
- [IKW09] Russell Impagliazzo, Valentine Kabanets, and Avi Wigderson. “New Direct-Product Testers and 2-Query PCPs”. In: *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*. Preliminary version appeared as ECCC TR08-016. 2009, pp. 131–140. DOI: 10.1145/1536414.1536436.
- [ISVW13] Yuval Ishai, Amit Sahai, Michael Viderman, and Mor Weiss. “Zero Knowledge LTCs and Their Applications”. In: *Proceedings of the 16th International Workshop on Approximation Algorithms for Combinatorial Optimization Problems, and of the 17th International Workshop on Randomization and Computation*. APPROX-RANDOM ’13. 2013, pp. 607–622.

- [Lig] *Reshaping KYC/AML in Web3*. <https://ligero-inc.com/kyc-aml>. 2025.
- [MZ25] Dor Minzer and Kai Zhe Zheng. “Improved Round-by-round Soundness IOPs via Reed–Muller Codes”. In: *Proceedings of the 66th Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’25. 2025, pp. 1286–1294.
- [Mei12] Or Meir. “Combinatorial PCPs with Short Proofs”. In: *Proceedings of the 26th Annual IEEE Conference on Computational Complexity*. CCC ’12. 2012.
- [Mei13] Or Meir. “IP = PSPACE Using Error-Correcting Codes”. In: *SIAM Journal on Computing* 42.1 (2013), pp. 380–403.
- [Mid] *Miden*. <https://github.com/0xPolygonMiden>.
- [NA25] Andrija Novakovic and Guillermo Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. Cryptology ePrint Archive, Report 2025/1187. 2025.
- [NTZ25] Vineet Nair, Justin Thaler, and Michael Zhu. *Proving CPU Executions in Small Space*. Cryptology ePrint Archive, Report 2025/611. 2025.
- [Pol] *Polygon*. <https://polygon.technology>.
- [RR20] Noga Ron-Zewi and Ron Rothblum. “Local Proofs Approaching the Witness Length”. In: *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’20. 2020, pp. 846–857.
- [RR22] Noga Ron-Zewi and Ron D. Rothblum. “Proving as Fast as Computing: Succinct Arguments with Constant Prover Overhead”. In: *Proceedings of the 54th ACM Symposium on the Theory of Computing*. STOC ’22. 2022, pp. 1353–1363.
- [RRR16] Omer Reingold, Ron Rothblum, and Guy Rothblum. “Constant-Round Interactive Proofs for Delegating Computation”. In: *Proceedings of the 48th ACM Symposium on the Theory of Computing*. STOC ’16. 2016, pp. 49–62.
- [RS60] I. S. Reed and G. Solomon. “Polynomial Codes Over Certain Finite Fields”. In: *Journal of the Society for Industrial and Applied Mathematics* 8.2 (1960), pp. 300–304.
- [RW24] Noga Ron-Zewi and Mor Weiss. “Zero-knowledge IOPs Approaching Witness Length”. In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO ’24. 2024, pp. 105–137.
- [Ris] *Risc0*. <https://risc0.com>.
- [Sta] *StarkNet*. <https://www.starknet.io/>.
- [Suc] *Succinct*. <https://succinct.xyz>.
- [TUZ07] Amnon Ta-Shma, Christopher Umans, and David Zuckerman. “Lossless Condensers, Unbalanced Expanders, and Extractors”. In: *Combinatorica* 27.2 (2007), pp. 213–240. DOI: 10.1007/s00493-007-0053-2. URL: <https://www.cs.utexas.edu/~diz/pubs/condenser.pdf>.
- [VSBW13] Victor Vu, Srinath Setty, Andrew J. Blumberg, and Michael Walfish. “A hybrid architecture for interactive verifiable computation”. In: *Proceedings of the 34th IEEE Symposium on Security and Privacy*. Oakland ’13. 2013, pp. 223–237.
- [XZZPS19] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. “Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation”. In: *Proceedings of the 39th Annual International Cryptology Conference*. CRYPTO ’19. 2019, pp. 733–764.
- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. “BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes”. In: *Proceedings of the 44th Annual International Cryptology Conference*. Vol. 14929. CRYPTO ’24. 2024, pp. 138–169.

- [Zei24] Hadas Zeilberger. *Khatam: Reducing the Communication Complexity of Code-Based SNARKs*. Cryptology ePrint Archive, Report 2024/1843. 2024.